Unidad 1: Introducción a la Inteligencia Artificial y a los Sistemas Expertos

Sistemas Expertos

1 Introducción

Definiciones de inteligencia, aprendizaje, inferencia.

Inteligencia se puede describir como la capacidad de percibir o inferir información, y retenerla como conocimiento para aplicarlo en situaciones cambiantes e imprevisibles. Implica razonamiento, interacción con el mundo exterior y aprendizaje.

Aprendizaje es el proceso a través del cual se adquieren y modifican destrezas, conocimientos o conductas con el propósito de llevar a cabo una tarea de un modo más eficiente y eficaz cada vez. El aprendizaje involucra el perfeccionamiento de habilidades y la adquisición de conocimiento.

Inferencia es un mecanismo que permite, a partir del valor de verdad de ciertos hechos (premisas), y con el conocimiento que se posee, conocer el valor de verdad de otros hechos (conclusiones).

La **Inteligencia Artificial** consiste en el desarrollo y la utilización de ordenadores con los que se intenta reproducir los procesos de la inteligencia humana: comprensión del lenguaje, aprendizaje, razonamiento, resolución de problemas, etc.

¿Qué características debe tener un problema para aplicar técnicas de IA?

Problemas en los que se ocupa técnicas de IA:

- No se sabe si tienen solución, y si la tienen, no es única
- Las soluciones son aceptadas como buenas, no como óptimas
- Los problemas varían en el tiempo
- La explosión combinatoria en la resolución se vuelve inmanejable

La inteligencia necesita del conocimiento, pero el conocimiento no es suficiente para obtener inteligencia.

Una **técnica de IA** es un método que utiliza conocimiento representado de una forma tal que:

- Represente generalizaciones (no se necesita representar cada situación individual)
- Sea comprendido por todas las personas que lo proporcionan.
- Pueda modificarse fácilmente (corregir errores y reflejar cambios en el mundo y visión del mismo)
- Pueda usarse en gran cantidad de situaciones aun no siendo totalmente preciso o completo.

- Pueda usarse para ayudar a superar su propio volumen (acotar el rango de posibilidades que deben ser consideradas normalmente).

La inteligencia artificial en realidad no existe. Solamente existe la inteligencia natural: todo lo artificial no es inteligente porque responde a un algoritmo. Un nombre más apropiado para lo que se conoce como Inteligencia Artificial sería Simulación del Comportamiento Inteligente.

Gran parte del trabajo inicial de la IA trataba de concentrarse en las tareas más comunes (habla, visión) pero esta ingenua suposición no es correcta. Si bien las habilidades de un experto necesitan conocimientos que la mayoría no tenemos, con frecuencia es mucho menor que el conocimiento necesario en las tareas más comunes y con frecuencia más fácil de representar y tratar en los programas.

Aunque los sistemas expertos constituyen una de las áreas de investigación en el campo de la IA, la mayor parte de las restantes áreas, si no todas, disponen de una componente de sistemas expertos formando parte de ellas.

2 ¿Qué es un Sistema Experto?

Un **sistema experto** puede definirse como un sistema informático (hardware y software) que simula a los expertos humanos en un área de especialización dada.

Como tal, un sistema experto debería ser capaz de procesar y memorizar información, razonar en situaciones deterministas e inciertas, comunicarse con los hombres y/u otros sistemas expertos, tomar decisiones apropiadas, y explicar por qué se han tomado tales decisiones. Se puede pensar también en un sistema experto como un consultor que puede suministrar ayuda a (o en algunos casos sustituir completamente) los expertos humanos con un grado razonable de fiabilidad.

Ejemplos: diagnóstico de enfermedades, control de tráfico, edición de imágenes.

¿Un SE es inteligente? ¿Por qué si/no?

No. Un sistema se considera inteligente cuando tiene la posibilidad de aprender y en base a este aprendizaje emprender acciones autónoma-flexibles. Según Karanik, la diferencia entre un sistema experto y un sistema inteligente es que **el sistema experto no tiene la capacidad de aprendizaje** (si la tiene es un sistema inteligente).

4 ¿Por qué los Sistemas Expertos?

Razones para utilizar sistemas expertos:

- Hay pocos expertos humanos: personal con poca experiencia puede resolver problemas que requieren un conocimiento de experto.
- El conocimiento de varios expertos humanos puede combinarse.
- El tiempo de respuesta es crítico.
- Se requieren respuestas rápidas y fiables en situaciones en las que los expertos humanos no pueden (problemas muy complejos).
- Realizar operaciones monótonas o de gran precisión.

- Se pueden obtener grandes ahorros: si bien el desarrollo o la adquisición de un sistema experto generalmente es caro, el mantenimiento y el costo de su uso repetido es bajo, dando lugar a una rápida amortización.

5 Tipos de Sistemas Expertos

Los problemas con los que pueden tratar los sistemas expertos pueden ser esencialmente deterministas o esencialmente estocásticos.

Los sistemas expertos que tratan problemas deterministas son conocidos como **sistemas basados en reglas o deterministas**, porque sacan sus conclusiones basándose en un conjunto de reglas utilizando un mecanismo de razonamiento lógico.

En situaciones inciertas, es necesario introducir algunos medios para tratar la incertidumbre. Algunos sistemas expertos usan la misma estructura de los sistemas basados en reglas, pero introducen una medida asociada a la incertidumbre de las reglas y a la de sus premisas. En este caso se pueden utilizar algunas fórmulas de propagación para calcular la incertidumbre asociada a las conclusiones.

Otra medida intuitiva de incertidumbre es la probabilidad, en la que la distribución conjunta de un conjunto de variables se usa para describir las relaciones de dependencia entre ellas, y se sacan conclusiones usando fórmulas muy conocidas de la teoría de la probabilidad.

Los sistemas expertos que utilizan la probabilidad como medida de incertidumbre se conocen como **sistemas expertos probabilísticos o estocásticos** y la estrategia de razonamiento que usan se conoce como razonamiento probabilístico, o inferencia probabilística.

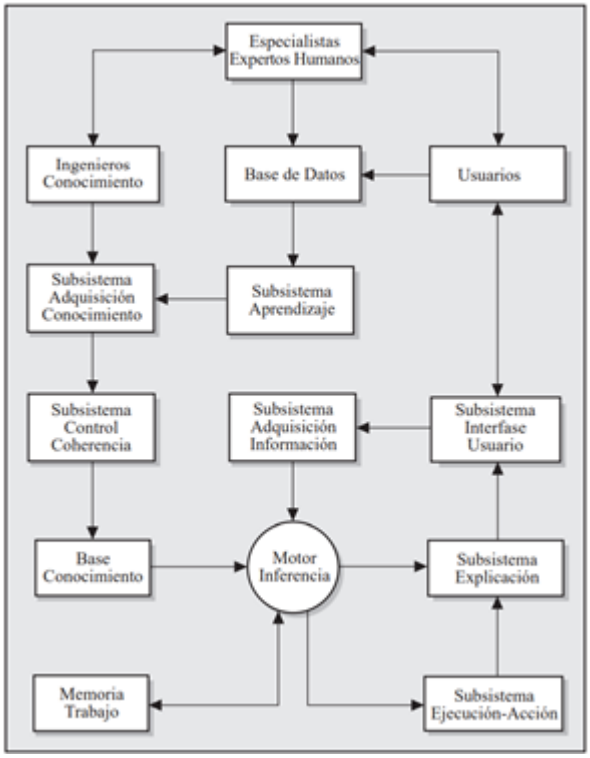
Enumere desde el punto de vista de las entradas y salidas las diferencias entre un SE basado en reglas y otro probabilístico.

Desde el punto de vista de las **entradas** no hay diferencia, para ambos las entradas son hechos o evidencias. En la **BC** los sistemas basados en reglas tienen objetos y reglas que relacionan dichos objetos mientras que los sistemas probabilísticos cuentan con variables, sus probabilidades asociadas y funciones de probabilidad conjunta. La **salida** de los sistemas basados en reglas corresponde al valor de uno o varios objetos mientras que la salida de los sistemas probabilísticos corresponde a un ranking de probabilidades, el cual indica cuál de las situaciones es más probable en base a los hechos proporcionados.

6 Componentes de un Sistema Experto

1. La Componente Humana: un sistema experto es generalmente el resultado de la colaboración de uno o varios expertos humanos especialistas en el tema de estudio y los ingenieros del conocimiento (que trasladan el conocimiento a un lenguaje que el sistema experto pueda entender), con los usuarios en mente.
2. La Base de Conocimiento y la Memoria de Trabajo: los especialistas son responsables de suministrar a los ingenieros del conocimiento una base de conocimiento ordenada y estructurada, y un conjunto de relaciones bien definidas y explicadas. El **conocimiento** se refiere a afirmaciones de validez general tales como reglas, distribuciones de probabilidad, etc. Los **datos** se refieren a los hechos relacionados

con un problema particular (Conocimiento = síntomas, enfermedades y relaciones entre ellos. Datos = síntomas particulares de un paciente). El conocimiento se almacena en la base de conocimiento y los datos se almacenan en la memoria de trabajo.



3. Subsistema de Adquisición de Conocimiento: controla el flujo del nuevo conocimiento que fluye del experto humano a la base de datos. Determina qué nuevo conocimiento se necesita, o si el conocimiento recibido es en realidad nuevo.
4. Control de la Coherencia: controla la consistencia de la base de datos y evita que unidades de conocimiento inconsistentes entren en la misma (conclusiones absurdas o en conflicto como, por ejemplo, situaciones en las que el sistema genera probabilidades mayores que la unidad o negativas). Por otra parte, cuando se solicita información de los expertos humanos, éste subsistema informa sobre las restricciones que ésta debe cumplir para ser coherente con la existente en la base de conocimiento.
5. El Motor de Inferencia: es el corazón de todo sistema experto. Su cometido principal es el de sacar conclusiones aplicando el conocimiento a los datos. Las conclusiones del motor de inferencia pueden estar basadas en conocimiento determinista o conocimiento probabilístico. En los sistemas expertos basados en probabilidad, la propagación de incertidumbre es la tarea principal del motor de inferencia. Esta tarea es tan compleja que da lugar a que ésta sea probablemente la componente más débil de casi todos los sistemas expertos existentes.
6. El Subsistema de Adquisición de Información: si la información inicial es muy limitada y no se pueden sacar conclusiones, el motor de inferencia utiliza el subsistema de adquisición de información para obtener la misma y continuar con el proceso de inferencia hasta que se hayan sacado conclusiones. En algunos casos, el usuario puede suministrar la información requerida para este y otros objetivos. De ello resulta la necesidad de una interface de usuario y de una comprobación de la consistencia de la información suministrada.
7. Interface de usuario: es el enlace entre el sistema experto y el usuario. Debe incorporar mecanismos eficientes para mostrar (conclusiones, razones que expliquen tales conclusiones, explicación de las acciones iniciadas por el sistema, etc.) y obtener información (cuando el motor de inferencia no puede concluir debido a la ausencia de información) de forma fácil y agradable.
8. Subsistema de ejecución de órdenes: es el componente que permite al sistema experto iniciar acciones basadas en las conclusiones sacadas por el motor de inferencia.

9. Subsistema de Explicación: presenta explicaciones de las conclusiones sacadas o las acciones iniciadas por el sistema experto.
10. Subsistema de Aprendizaje: permite aprender y obtener experiencia a partir de los datos disponibles. Por aprendizaje estructural nos referimos a algunos aspectos relacionados con la estructura del conocimiento (reglas, distribuciones de probabilidad, etc.), como el descubrimiento de nuevos síntomas relevantes para una enfermedad. Por aprendizaje paramétrico nos referimos a estimar los parámetros necesarios para construir la base de conocimiento, como la estimación de la frecuencia de aparición de un síntoma para una enfermedad.

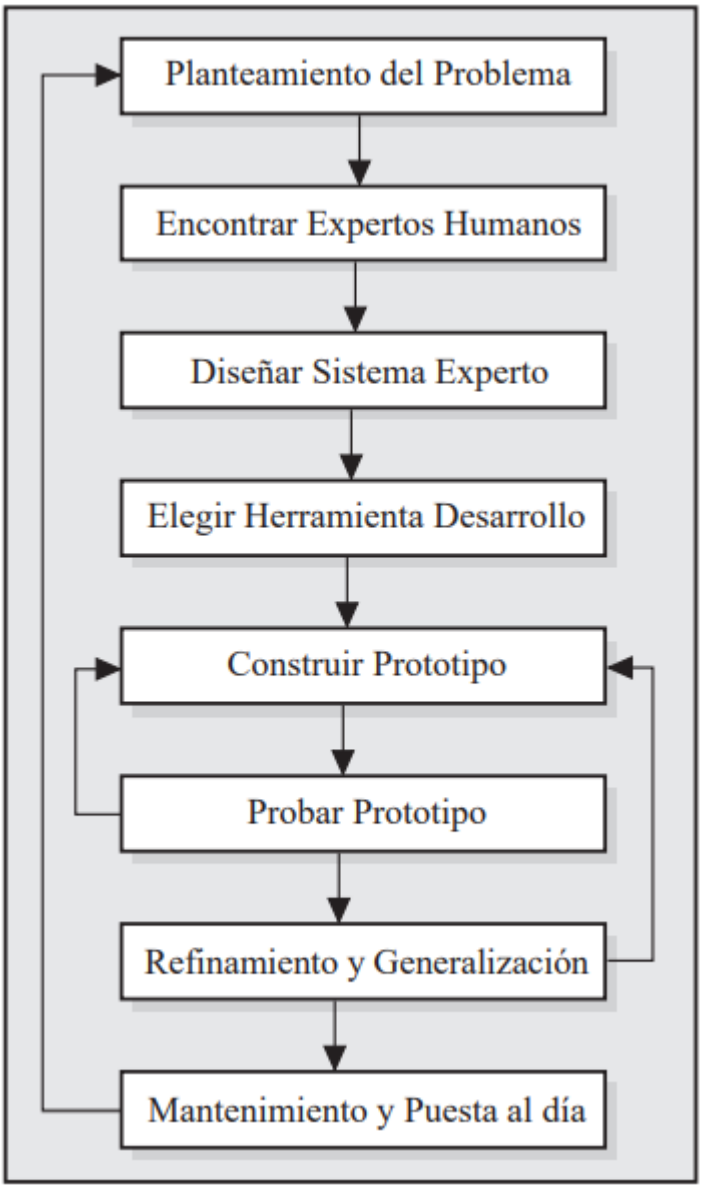
De los componentes antes mencionados puede verse que los sistemas expertos pueden realizar varias tareas, entre ellas:

- Adquisición de conocimiento y la verificación de su coherencia.
- Almacenar (memorizar) conocimiento.
- Preguntar cuando se requiere nuevo conocimiento.
- Aprender de la base de conocimiento y de los datos disponibles.
- Realizar inferencia y razonamiento en situaciones deterministas y de incertidumbre.
- Explicar conclusiones o acciones tomadas.
- Comunicarse con los expertos y no expertos humanos y con otros sistemas expertos.

Las 3 partes fundamentales son:

- La base de conocimiento (BC): conjunto de objetos y conjunto de reglas que establecen las relaciones entre esos objetos.
- El motor de inferencia (MI): técnicas que permiten realizar las inferencias o deducciones.
- Memoria de trabajo (MT): se encarga de guardar resultados parciales y datos que informan sobre la evolución del sistema.

7 Desarrollo de un Sistema Experto



8 Otras Áreas de la IA

1. Representación del conocimiento: se refiere a los mecanismos para representar y manipular la información de manera que los esquemas de representación resultantes permitan una búsqueda o una operación eficiente de los mecanismos de inferencia.
2. Planificación: los métodos de planificación analizan diferentes estrategias para descomponer un problema dado, resolver cada una de sus partes, y llegar a una solución final. La computación paralela da lugar a nuevos problemas que requieren estrategias para dividir las tareas de forma adecuada para resolver muchas partes simultáneamente.
3. Demostración automática de teoremas
4. Juegos automatizados

5. Reconocimiento de patrones: trata de diferentes técnicas de clasificación para identificar los subgrupos, o conglomerados, con características comunes en cada grupo.
6. Reconocimiento de la voz: Los problemas asociados con las diferentes pronunciaciones y tonos de voz son los principales obstáculos que esta disciplina tiene que afrontar.
7. Procesamiento de lenguaje natural: su objetivo consiste en extraer tanta información como sea posible de un texto escrito. El problema inverso es el de la generación de lenguaje, es decir, dotar a los computadores de capacidades para generar sentencias de lenguaje natural.
8. Visión Artificial: el objetivo es usar ordenadores para localizar y reconocer automáticamente objetos en tres dimensiones.
9. Robótica
10. Redes neuronales: la información contenida en una red neuronal se codifica en la estructura de la red y en los pesos de las conexiones. Por tanto, en una situación particular, los pesos de las conexiones tienen que modificarse para reproducir la salida deseada. Esta tarea de aprendizaje se consigue mediante una técnica de aprendizaje por analogía, es decir, el modelo se entrena para reproducir las salidas de un conjunto de señales de entrenamiento con el objetivo de codificar de esta forma la estructura del fenómeno

Espacio de Estados y soluciones

Muchos de los problemas en los que se pueden aplicar técnicas de IA se plantean en términos de encontrar una configuración objetivo a partir de una configuración inicial dada, aplicando transformaciones válidas de acuerdo al universo del problema.

De esta manera, el **espacio de estados** de un problema es un grafo cuyos nodos representan las configuraciones alcanzables (los estados válidos) y cuyos arcos representan las transformaciones posibles (las transiciones de estado). El **espacio de soluciones** consiste en un subgrafo del espacio de estados, que contiene el conjunto de posibles configuraciones y transiciones que llevan de un estado inicial a un estado objetivo.

Agentes Inteligentes

1 Introducción

S* = {{∅}, {s1}, {s2}, {s1, s1}, {s1, s2}, {s1, s2, s2, s1},...} Clausura de Kleene

□(S) = {{∅}, {s1}, {s2}, {s1, s2}} Conjunto potencia

2 ¿Qué son los agentes?

Explique la definición de agentes dada por Wooldridge.

Un **agente** es un sistema informático que está situado en algún entorno, y que es capaz de actuar de forma autónoma en este entorno para cumplir sus objetivos de diseño. Esto significa que un agente recibe percepciones del ambiente mediante sensores, y en base a ello toma decisiones de forma autónoma (es decir, sin intervención de un usuario), seleccionando de las acciones disponibles en su repertorio aquellas a ejecutar para modificar el entorno y cumplir los objetivos para los que fue diseñado.

En general, el ambiente es no determinístico: los agentes no tendrán un control total sobre su entorno, por lo que la misma acción ejecutada dos veces en condiciones aparentemente similares, puede tener efectos muy diferentes.

Capacidad efectorial de los agentes: es su repertorio de acciones disponibles para modificar sus entornos. No todas las acciones se pueden realizar en todas las situaciones, sino que estas tienen pre-condiciones asociadas a ellas.

En términos matemáticos se puede decir que el comportamiento del agente viene dado por la **función del agente**, que proyecta una percepción dada en una acción. La función de un agente artificial se implementará mediante el **programa del agente**, que se ejecuta sobre su arquitectura.

El problema clave que enfrenta un agente es decidir cuál de sus acciones debe realizar para satisfacer mejor sus objetivos de diseño. La complejidad del proceso de toma de decisiones puede verse afectada por una serie de **propiedades ambientales** diferentes:

Ambientes. Características de ambientes en los que operan los agentes.

- **Accesible vs inaccesible**: un entorno accesible es aquel en el que el agente puede obtener información completa, precisa y actualizada sobre el estado del entorno. No es necesario que el agente mantenga un estado interno ya que el sensor le permite tener acceso al estado total del ambiente.
    - Accesible: Ajedrez
    - No accesible: Póker, Loba.
- **Determinista vs no determinista**: un entorno determinista es aquel en el que no hay incertidumbre sobre el estado que resultará de realizar una acción.
    - Determinista: Cubo Rubik.
    - No determinista: Juegos de dados, ruleta.
- **Episódico vs no episódico**: en los entornos episódicos el agente puede decidir qué acción realizar basándose únicamente en el episodio actual; no necesita razonar sobre las interacciones entre este y los episodios futuros.

    - Episódico: Firewall, detector de billetes falsos.
    - No episódico: Truco.
- **Estático vs dinámico**: un entorno estático es aquel que se puede asumir que permanece sin cambios excepto por la realización de acciones por parte del agente. Un entorno dinámico es aquel en el que operan otros procesos y, por tanto, cambia de formas que escapan al control del agente.
    - Estáticos: Torres de Hanoi
    - Dinámicos: Damas, aire acondicionado
- **Discreto vs continuo**: un entorno es discreto si hay un número fijo y finito de acciones y percepciones en él.
    - Discretos: Ta-Te-Ti
    - Continuos: Piloto automático

Dada la clasificación de ambientes de Wooldridge determine la combinación más simple de entorno para una agente.

La combinación de entorno más simple es: accesible, determinista, episódico, estático y discreto.

2.2 Agentes Inteligentes

Explique cuáles son las características implícitas en la definición de comportamiento autónomo flexible de agentes inteligentes.

Un **agente inteligente** es aquel que es capaz de un comportamiento autónomo flexible para cumplir con sus objetivos de diseño. Autónomo significa que realiza acciones sin intervención de un usuario, y con flexible nos referimos a:

- **Reactividad**: los agentes inteligentes son capaces de percibir su entorno y responder de manera oportuna a los cambios que ocurren en él (donde estos cambios pueden afectar los objetivos del agente o los supuestos que sustentan los procedimientos que el agente está ejecutando para lograr sus objetivos).
- **Proactividad**: los agentes inteligentes pueden exhibir un comportamiento dirigido a objetivos al tomar la iniciativa.
- **Habilidad social**: los agentes inteligentes son capaces de interactuar con otros agentes (y posiblemente con humanos).

Construir un sistema que logre un equilibrio efectivo entre el comportamiento dirigido a objetivos y el reactivo es uno de los problemas clave que los diseñadores deben enfrentar: no queremos un sistema que ejecute procedimientos a ciegas para cumplir los objetivos ni un sistema que se la pase reaccionando al medio sin comprometerse con un objetivo.

Además, se entiende a la habilidad social no como la capacidad de intercambiar información, que es algo relativamente sencillo, sino de negociar y cooperar con otros sistemas autónomos para lograr los objetivos.

2.3 Agentes y Objetos

Según Wooldridge, ¿Cuál es la diferencia entre agente y objeto? ¿Un objeto puede tener un comportamiento autónomo flexible?

Los objetos se definen como entidades computacionales que encapsulan algún estado, pueden realizar acciones o métodos en este estado y comunicarse mediante el paso de mensajes. Si bien existen similitudes obvias, también existen diferencias significativas entre agentes y objetos:

- El grado en que los agentes y los objetos son autónomos: se puede pensar que un objeto exhibe autonomía sobre su estado (tiene control sobre él a través de propiedades privadas) pero un objeto no exhibe control sobre su comportamiento.
  Una vez un objeto ha hecho público un método, posteriormente no tiene control sobre si se ejecuta o no ese método. Esto no suele ser un problema porque los objetos de un sistema comparten un objetivo común, pero en muchos tipos de sistemas multiagente, no se puede suponer un objetivo común, ellos deciden por sí mismos si realizar o no una acción a petición de otro agente.
  Explique la afirmación "Objects do it for free, Agents do it for money".
  Los agentes no invocan métodos de otros, sino que les solicitan a otros agentes realizar acciones, pudiendo obtener resultados o no. En los objetos la decisión recae en el objeto que invoca el método mientras que en los agentes dicha decisión recae en el que recibe la petición.
- El modelo de objetos no tiene nada que decir sobre cómo construir sistemas que integren comportamiento autónomo flexible (reactivo, proactivo, social).
- Se considera que cada agente tiene su propio hilo de control, mientras que en el modelo de objeto estándar hay un solo hilo de control en el sistema.

2.4 Agentes y sistemas expertos

Diferencia entre sistema experto y agentes inteligentes según Wooldridge.

La distinción más importante entre agentes y sistemas expertos es que los sistemas expertos no interactúan directamente con ningún entorno: obtienen su información no a través de sensores, sino a través de un usuario que actúa como intermediario. Del mismo modo, no actúan en ningún entorno, sino que dan feedback o consejos a un tercero. Además, generalmente no requerimos que los sistemas expertos sean capaces de cooperar con otros agentes.

3 Arquitecturas abstractas para agentes inteligentes

Consisten en el modelado de la toma de decisiones de un agente como una función abstracta, que de alguna manera logra indicar qué acción realizar, sin discutir cómo se podría implementar esa función.

Sea S = {s1, s2, ...} el conjunto de los posibles estados ambientales y sea el conjunto de acciones A = {a1, a2, ...} la capacidad efectorial de un agente. Luego, un **agente** puede verse de manera abstracta como una función:
action: S* → A
que mapea secuencias de estados ambientales a acciones. Nos referiremos a un agente modelado por una función de esta forma como un **agente estándar**. La idea es que un agente decide qué acción realizar en base a su historia (sus experiencias hasta la fecha).

El **comportamiento de un entorno** se puede modelar como una función:
env: S x A → □(S)

que toma el estado actual del entorno s ∈ S y una acción a ∈ A (realizada por el agente), y los asigna a un conjunto de estados ambientales env(s, a), que podrían resultar de realizar la acción a en el estado s. Si todos los conjuntos en el rango de env tienen un solo elemento (es decir, si el resultado de realizar cualquier acción en cualquier estado es un conjunto que contiene un solo miembro), **entonces el entorno es determinista** y su comportamiento se puede predecir con precisión.

Podemos representar la interacción del agente y el entorno como una historia. Una historia h es una secuencia:
h: s0 --a0→ s1 --a1→ s2 … --au-1→ su
luego la secuencia h representará una posible historia del agente en el ambiente si se cumplen las siguientes condiciones:

$$\forall u \in \mathbb{N}, a_u = action((s_0, s_1, \ldots, s_u))$$
$$\forall u \in \mathbb{N} \text{ such that } u > 0, s_u \in env(s_{u-1}, a_{u-1}).$$

El **comportamiento característico** de un agente es el conjunto de todas las acciones que satisfacen esas dos condiciones.

Si una propiedad se mantiene en todas las historias, puede considerarse una **propiedad invariante** del agente en el ambiente.

Denotaremos con hist(agent, environment) el conjunto de todas las historias del agente en el ambiente. Dos agentes a1 y a2 son **equivalentes en comportamiento con respecto al ambiente** env si h(a1, env) = h(a2, env). Y simplemente **equivalentes en comportamiento** si son equivalentes en comportamiento con respecto a cualquier ambiente.

3.1 Agentes puramente reactivos

Agentes puramente reactivos. Explicación y ejemplo.

Es un tipo de agentes que basan su toma de decisiones completamente en el presente, sin ninguna referencia al pasado, responden directamente a su entorno. Son los más simples.

El comportamiento de un agente puramente reactivo puede representarse con una función action: S → A

Para cada agente puramente reactivo, existe un agente estándar equivalente. Lo contrario, sin embargo, no es cierto.

Ejemplo: termostato de un aire acondicionado.
Hace calor → Prender.
-Hace calor → Apagar

3.2 Percepción

La función de decisión de un agente se puede separar en percepción y acción. La **función ‘see’** captura la habilidad del agente para observar su entorno. La salida de la función see es una percepción.

Sea P un conjunto no vacío de percepciones, luego ‘see’ es una función que mapea los estados del entorno a percepciones.
see: S → P

Y ‘action’ es ahora una función que mapea secuencias de percepciones a acciones.
action: P* → A

Explique y ejemplifique el concepto de estados indistinguibles que plantea Wooldridge en la definición de percepción.

De toda la información existente en el ambiente, los agentes perciben aquella que es necesaria para tomar sus decisiones. Teniendo en cuenta esto, **dos estados del ambiente S, s y s’, son equivalentes** para el agente (s ≡ s’) si se mapean a la misma percepción (see(s) ≡ see(s’)) y, por lo tanto, son indistinguibles para el mismo. Si el número de percepciones distintas es igual al número de estados ambientales diferentes, entonces el agente puede distinguir cada estado; el agente tiene una **percepción perfecta** en el ambiente. En el otro extremo, si el número de percepciones distintas es igual a 1, la capacidad de percepción del agente es inexistente, no puede distinguir entre estados diferentes. **Ejemplo**: para un sistema de riego automatizado que depende de la humedad, dos estados ambientales con la misma humedad y diferente velocidad del viento, provocarán percepciones iguales dado que al agente no le interesa la velocidad del viento (por lo que no será percibida), siendo así estados equivalentes e indistinguibles para el agente.

3.3 Agentes con estado

Explicar agentes con estado.

Representar las acciones a partir de secuencias de estados del ambiente o percepciones no es intuitivo, la idea ahora es considerar agentes que pueden mantener su estado.

Estos agentes tienen una estructura de datos interna, que normalmente se utiliza para registrar información sobre el estado y el historial del entorno. Llamaremos I al conjunto de todos los **estados internos** del agente; el proceso de toma de decisiones se basa, al menos en parte, en esta información.

La función ‘action’ es ahora definida como un mapeo de estados internos a acciones.
action: I → A

Se introduce una nueva función ‘next’ que mapea el estado interno y una percepción a un nuevo estado interno.
next: I x P → I

De esta manera, el agente lleva a cabo un ciclo consistente en: percibir el entorno con la función ‘see’, actualizar su estado interno con la función ‘next’, y seleccionar una acción mediante ‘action’.

Diferencias entre agente puramente reactivo y agente con estado interno.

Mientras que el agente puramente reactivo toma decisiones que involucran únicamente la percepción actual (sin tener en cuenta el pasado), el agente con estado mantiene un estado

interno que refleja la historia de lo que sucede en el ambiente en el que está inmerso y utiliza dicho estado para la toma de decisiones. Para actualizar su estado a partir de las diferentes percepciones, el agente con estado interno tiene una función adicional, la función next. Además, los agentes reactivos son más rápidos que los agentes con estado, que son deliberativos y tienen problemas en ambientes dinámicos si el tiempo de deliberación es muy alto./

4 Arquitecturas concretas para agentes inteligentes

La **arquitectura de un agente** es esencialmente un mapeo de las partes internas de un agente: sus estructuras de datos, las operaciones que se pueden realizar en estas estructuras de datos y el flujo de control entre estas estructuras de datos.

Consideraremos cuatro clases de agentes:

- Agentes basados en lógica: la toma de decisiones se realiza mediante deducción lógica.
- Agentes reactivos: la toma de decisiones se implementa en alguna forma de mapeo directo de la situación a la acción.
- Agentes creencia-deseo-intención (BDI): la toma de decisiones depende de la manipulación de estructuras de datos que representan las creencias, deseos e intenciones del agente.
- Arquitecturas en capas: la toma de decisiones se realiza a través de varias capas de software, cada una de las cuales razona más o menos explícitamente sobre el entorno en diferentes niveles de abstracción.

4.1 Arquitecturas basadas en lógica

Agentes basados en lógica. Explicación y ejemplo.
De un ejemplo de Agente basado en Lógica (distinto al material) y explique su funcionamiento basado en el algoritmo que utiliza el agente

Se puede generar un comportamiento inteligente en un sistema dándole a ese sistema una representación simbólica de su entorno y su comportamiento deseado (fórmulas lógicas), y manipulando sintácticamente esta representación (deducción lógica). Asumimos que su estado interno es una base de datos de fórmulas de lógica de predicados de primer orden, que representa la información que el agente tiene de su entorno.

Sea L el conjunto de sentencias de lógica de primer grado, y sea D = □(L) el conjunto de L bases de datos; es decir, el conjunto de conjuntos de L-fórmulas. Luego, el estado interno de un agente es un elemento de D (un conjunto de L-fórmulas). Llamamos Δ, Δ1, … a los elementos de D. El proceso de toma de decisión de un agente se modela a través de un conjunto de reglas de inferencia, p. Éstas son simples reglas de inferencia lógica. Si se puede probar la fórmula Φ desde la base de datos Δ utilizando únicamente las reglas de inferencia ρ, escribimos Δ ├ρ Φ.

La función de percepción ‘see’ permanece invariable: see: S □ P

La función ‘next’ ahora tiene la forma: next: D x P □ D
La cual mapea una base de datos y una percepción con una nueva base de datos.

La función de selección de la acción, tiene la siguiente forma: action: D □ A
La definición en pseudocódigo de la función es la siguiente:

```
1.    function action(Δ : D) : A
2.    begin
3.          for each a ∈ A do
4.                if Δ ⊢_ρ Do(a) then
5.                      return a
6.                end-if
7.          end-for
8.          for each a ∈ A do
9.                if Δ ⊬_ρ ¬Do(a) then
10.                     return a
11.               end-if
12.         end-for
13.         return null
14.   end function action
```

La función action toma como entrada algún estado interno perteneciente a D y devuelve la acción a realizar.

La idea es que el programador del agente codifique las reglas de deducción y la base de datos de tal manera que si se puede derivar Do(a), entonces a es la mejor acción.

En las líneas 3 a 7 el agente toma cada posible acción a, e intenta probar la condición de la fórmula Do(a) desde su base de datos actual, utilizando sus reglas de inferencia ρ. Si tiene éxito, devuelve la acción a ejecutar a. (se lee: 'Si desde el estado interno se puede derivar "hacer A" usando las reglas RO, entonces'.)

Si el agente falla al probar Do(a), para todas las acciones a, intenta encontrar una acción que sea consistente con las reglas y la base de datos, es decir una acción que no esté explícitamente prohibida: intenta encontrar una acción a tal que no se pueda derivar ¬ Do(a) (líneas 8 a 12). Si no se encuentra ninguna acción, devuelve null.

De esta forma, el comportamiento del agente está determinado por las reglas de inferencia del agente ("su programa") y su base de datos actual (que representa la información que tiene el agente acerca de su ambiente).

La gran **ventaja** de este enfoque son las semánticas lógicas simples y elegantes.

Desventajas:

- La alta complejidad computacional hace cuestionable que estos puedan operar en dominios grandes o en entornos con limitaciones de tiempo: si el entorno cambia más rápido de lo que el agente toma una decisión, la misma puede no ser acertada.
- En algunos entornos, no es simple mapear el estado del ambiente a una percepción simbólica.
- La representación del conocimiento en lógica tradicional puede ser anti-intuitiva e incómoda.

Ejemplo: un agente encargado de mantener lleno un tanque de agua mediante la apertura y cierre de una canilla.

Predicados:

- CargandoAgua

- TanqueLleno

Reglas
- CargandoAgua AND TanqueLleno => hacer(CerrarCanilla)
- -CargandoAgua AND -TanqueLleno => hacer(AbrirCanilla)

4.2 Arquitecturas Reactivas

Explicación y ejemplo de arquitectura de subsunción (distinta al material de estudio). Ejemplo y explicación de agente basado en el modelo de Brooks. Explicar arquitectura reactiva.

La arquitectura incorporada o de subsunción desarrollada por Brooks es la mejor arquitectura conocida de agentes reactivos. La misma tiene dos características definitorias:
- La toma de decisión de un agente se realiza a través de un conjunto de comportamientos; donde cada comportamiento es una función de acción individual, que toma percepciones de entrada y las mapea a una acción a realizar.
- Muchos comportamientos pueden “dispararse” simultáneamente. Obviamente debe haber un mecanismo para elegir entre las diferentes acciones de los comportamientos disparados. Brooks propone ordenar los módulos en una jerarquía, con los comportamientos distribuidos en capas: cuanto más baja sea la capa, su prioridad es mayor (y por lo tanto pueden inhibir a los de capas más altas, que representan comportamientos más abstractos).

La función see permanece sin cambios: see: $S \rightarrow P$

La función de decisión action se realiza a través de un conjunto de comportamientos, junto a una relación de inhibición entre esos comportamientos. Un comportamiento es un par (c,a) donde $c \subseteq P$ es un conjunto de percepciones llamadas condición, y $a \in A$ es una acción. Un comportamiento (c,a) se va a “disparar” cuando el ambiente esté en estado $s \in S$ si $see(S) \in c$. Llamemos $Beh = \{(c,a) \mid c \subseteq P \text{ y } a \in A\}$ el conjunto de tales reglas.

Asociada con el conjunto de reglas de comportamiento del agente $R \subseteq Beh$, está una **relación de inhibición** binaria en el conjunto de comportamientos $\prec \subseteq R \times R$. Escribimos $b1 \prec b2$ si $(b1, b2) \in \prec$, y se lee como “b1 inhibe a b2”, es decir b1 es menor que b2 en la jerarquía, y por lo tanto tendrá prioridad sobre b2. La función action se define como:

```
1.    function action(p : P) : A
2.    var fired : ℘(R)
3.    var selected : A
4.    begin
5.          fired := {(c, a) | (c, a) ∈ R and p ∈ c}
6.          for each (c, a) ∈ fired do
7.                if ¬(∃(c', a') ∈ fired such that (c', a') ≺ (c, a)) then
8.                      return a
9.                end-if
10.         end-for
11.         return null
12.   end function action
```

La misma recibe una percepción y devuelve la acción a ejecutar. Para ello se conforma el conjunto “fired” con todos los comportamientos que se disparan con la percepción de entrada (línea 5). Luego se busca el comportamiento de mayor prioridad (el que inhiba a los demás que se dispararon) y se devuelve su acción a (líneas 6 a 10). Si ningún comportamiento se dispara, se devuelve null.

Ventajas:

- Simplicidad computacional: con este enfoque sencillo la complejidad temporal es de O(n^2) siendo n el mayor entre el número de comportamientos y el número de percepciones.
- Economía (el poder de cómputo requerido es mínimo)
- Robustez ante fallas
- Simplicidad y elegancia

Desventajas:

- Si los agentes no utilizan modelos de su ambiente, entonces deben tener suficiente información disponible en su ambiente local para determinar una acción aceptable.
- Dado que los agentes puramente reactivos toman decisiones basadas en información local, es difícil ver como tal toma de decisión podría considerar la información no local.
- Es difícil ver cómo diseñar los agentes puramente reactivos, de manera que aprendan de la experiencia y mejoren su performance con el tiempo.
- Es muy difícil desarrollar agentes que realicen tareas específicas.
- Mientras que los agentes efectivos se pueden diseñar con un número pequeño de comportamientos (normalmente, menos de diez capas), es mucho más difícil construir agentes que contengan muchas capas: la interacción entre los diversos comportamientos resulta difícil de entender.

Ejemplo: un agente que juega al Ta-Te-Ti

Comportamientos:

1. Si se tienen dos fichas en línea y hay espacio para una tercera entonces colocar la tercera ficha en dicho lugar.

2. Si el oponente tiene dos fichas en línea y hay espacio para una tercera entonces colocar una ficha en dicho lugar.
3. Si el lugar del centro del tablero está vacío entonces colocar una ficha en dicho lugar.
4. Si se tiene una ficha y hay 2 espacios vacíos en línea con ella entonces colocar una ficha en uno de ellos.
5. Si hay un lugar vacío entonces colocar una ficha en dicho lugar.

Luego (1) ≺ (2) ≺ (3) ≺ (4) ≺ (5)

4.3 Arquitecturas BDI (Belief-Desire-Intention o Creencia-Deseo-Intención)

El razonamiento práctico implica dos procesos importantes: decidir qué objetivos queremos conseguir (deliberación), y cómo vamos a conseguir esos objetivos (razonamiento medios-fines).

El funcionamiento de este modelo consiste en analizar la situación y generar un conjunto de alternativas (**deseos**). Luego de generar este conjunto de alternativas, se debe elegir una y comprometerse con ella. La opción elegida se convierte en **intención**, la cual determina las acciones del agente (destinadas a cumplir esa intención) y realimenta el futuro razonamiento práctico del agente (no se realizan acciones que son inconsistentes con esa intención). Esta acción persiste hasta que se cumple, se vuelve inviable o desaparece la razón que da lugar a dicha intención. Además, las intenciones están relacionadas con las **creencias** a futuro: si se tiene una intención, también se debería creer en que es posible que la misma se cumpla en el futuro.

Un problema clave es encontrar un buen balance entre el comportamiento proactivo y reactivo:

- Un agente que no se detiene lo suficiente a reconsiderar sus intenciones, seguirá intentando lograrlas incluso después de que esté claro que no se pueden lograr o que ya no hay razón para lograrlas.
- Un agente que constantemente reconsidera sus intenciones puede dedicar tiempo insuficiente a trabajar realmente para lograrlas, y por lo tanto corre el riesgo de nunca lograrlas.

Sin embargo, se debe tener en cuenta que los ambientes que cambian más rápido requieren que el agente reconsidere sus intenciones más seguido.

El estado de un agente BDI en cualquier momento es una tupla (B, D, I) y sus componentes son:

1. Un conjunto de creencias actuales (Bel), que representa la información que el agente tiene sobre su entorno actual.
2. Un conjunto de opciones actuales (Des), que representan posibles cursos de acción disponibles para el agente.
3. Un conjunto de intenciones (Int), que representan el enfoque actual del agente.
4. Una función de revisión de creencias (brf), que toma una entrada perceptiva y las creencias actuales del agente, y sobre la base de estas, determina un nuevo conjunto de creencias. brf: □(Bel) x P → □(Bel)
5. Una función de generación de opciones (options) que determina las opciones disponibles para el agente (sus deseos), sobre la base de sus creencias actuales y

sus intenciones actuales. Esta es responsable de decidir cómo cumplir las intenciones. options: □(Bel) x □(Int) → □(Des)

6. Una función de filtro (filter), que representa el proceso de deliberación del agente y que determina sus intenciones sobre la base de sus creencias, deseos e intenciones actuales. Esta función debe desechar las intenciones que ya no son alcanzables o cuyo costo de alcanzarlas es superior a la ganancia que proporcionan, retener las intenciones que aún no fueron alcanzadas pero son beneficiosas, y adoptar nuevas intenciones para alcanzar las actuales o descubrir nuevas oportunidades. filter: □(Bel) x □(Des) x □(Int) → □(Int)
7. Una función de selección de acción (execute), que determina una acción a realizar sobre la base de las intenciones actuales. execute: □(Int) → A

La función de decisión del agente BDI es: action: P □ A y se define como:

```
1.  function action(p : P) : A
2.  begin
3.      B := brf(B, p)
4.      D := options(D, I)
5.      I := filter(B, D, I)
6.      return execute(I)
7.  end function action
```

La línea 4 debe ser: D:=options(B, I)

Ventajas:

- Es intuitivo
- Proporciona una clara descomposición funcional

Desventajas:

- Es muy difícil saber cómo implementar eficientemente estas funciones.

En una arquitectura BDI:

- Las creencias se actualizan con las percepciones y creencias anteriores (V)
- Las intenciones se actualizan con intenciones anteriores, deseos y creencias. (V)
- Las acciones se toman en base a los deseos. (F).
- Las percepciones se actualizan a partir de las creencias actuales y las percepciones del agente. (F)
- Las intenciones se actualizan a partir de las creencias, los deseos y las acciones que ha tomado anteriormente el agente. (F)
- Las intenciones no se modifican hasta que los objetivos se alcanzan. (F)

Agentes basados en lógica: diferencia con agente reactivo y con BDI.

Diferencias con agentes reactivos:

- El agente basado en lógica mantiene y actualiza un estado interno y el reactivo no.
- El comportamiento del agente reactivo no depende de la deducción lógica sino de que la percepción satisfaga la condición de un comportamiento.

- El agente reactivo tiene una jerarquía de comportamientos en la que los de mayor prioridad inhiben a los demás debido a que múltiples comportamientos pueden dispararse con una percepción mientras que el agente basado en lógica se diseña de manera que sólo se pueda aplicar una acción.

Diferencias con BDI:

- El estado interno de un agente BDI es más complejo y está conformado por una tupla (B,D,I) mientras que el basado en lógica contiene objetos con sus valores de verdad.
- Mientras que el mecanismo de decisión del agente basado en lógica consiste en ejecutar aquella acción que se pueda derivar con las reglas dadas, el BDI tiene en cuenta las creencias, deseos e intenciones para comprometerse con una intención y ejecutar las acciones necesarias.
- El agente BDI es mucho más deliberativo mientras que en el basado en lógica es el programador quien se debe plantear todo lo que puede pasar y representarlo en las reglas.

4.4 Arquitecturas en capas

En estas arquitecturas los distintos subsistemas están dispuestos en una jerarquía de capas que interactúan entre sí con el propósito de que un agente sea capaz de tener comportamiento reactivo y proactivo a la vez.

Podemos encontrar dos tipos de flujo de control en la arquitectura por capas:

Capas horizontales

En las arquitecturas de capas horizontales las capas de software están conectadas directamente al sensor de entrada y a la acción de salida. Cada capa en sí misma actúa como un agente: recibe una percepción y produce sugerencias en cuanto a qué acción realizar.
La mayor ventaja es su simplicidad conceptual: si necesitamos un agente que exhiba n tipos diferentes de comportamiento, entonces implementamos n capas distintas.
Sin embargo, dado que las capas están compitiendo entre sí para generar sugerencias de acción, existe el peligro de que el comportamiento total del agente no sea coherente. Para asegurar que sea consistente, generalmente se incluye una función mediadora, que toma la decisión sobre qué capa tiene el “control” del agente en un instante determinado. Esto implica un gran problema de diseño: si hay n capas y cada una es capaz de sugerir m acciones diferentes, hay m^n interacciones que considerar.

Capas verticales

En las arquitecturas de capas verticales, tanto el sensor de entrada como la acción de salida son tratados, cada uno, de cerca por al menos una capa.
Podemos subdividirlas en arquitecturas de un paso y de dos pasos. En las arquitecturas de un paso, el control fluye secuencialmente a través de cada capa, hasta que la última capa genera la acción de salida. En las arquitecturas de dos pasos, la información fluye hacia arriba de la arquitectura y el control luego fluye hacia abajo. En ambos casos la complejidad de las interacciones es menor: entre n capas hay n-1 interfaces, si cada una es capaz de sugerir m acciones, hay m^2 (n-1) interacciones posibles. Sin embargo, debido a que el control debe pasar por todas las capas, no es tolerante a fallas: si falla alguna capa, afecta en gran medida el desempeño del agente.

5 Lenguajes de programación de agentes

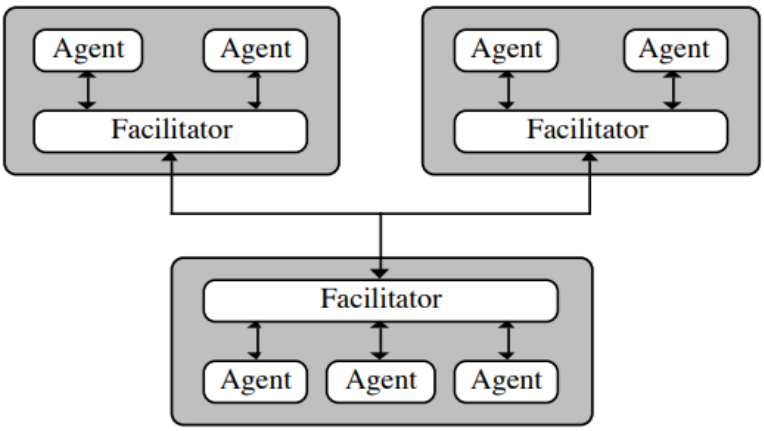
La idea clave es la de programar directamente a los agentes en términos de modelos mentales (como BDI) como un mecanismo de abstracción para representar las propiedades de sistemas complejos.

Agentes con comunicación directa y asistida.

En la comunicación directa los agentes manejan su propia coordinación mientras que en la comunicación asistida los agentes dependen de programas especiales para lograr la coordinación. La ventaja de la comunicación directa es que no se basa en la existencia, capacidades o sesgos de ningún otro programa. Las desventajas son el costo (debido a que hacer un broadcast de solicitudes o especificaciones y que este sea procesado por los demás agentes en un entorno grande es muy costoso) y la complejidad de la implementación (ya que cada agente debe negociar la comunicación con los demás agentes).

Agentes Federados.

Una alternativa popular a la comunicación directa que elimina sus dos desventajas es organizar a los agentes en lo que se llama un sistema federado. Esto consiste en conformar una jerarquía en la que los agentes no se comunican directamente entre sí. En cambio, se comunican solo con los programas del sistema llamados facilitadores, y son los facilitadores quienes se comunican entre sí.



¿En un sistema multiagentes es fundamental el uso de la ontología para la comunicación? ¿Para qué sirve la ontología?

Si, es fundamental la ontología. La misma define un conjunto de significados específicos sobre un vocabulario, respecto a un dominio determinado, para que los agentes puedan realizar la misma interpretación sobre los mensajes que se envían.

Según la definición de Agencia ¿Un sistema experto puede considerarse Agente?

Agencia: se refiere al grado de autonomía del agente, es decir, la capacidad que este posee para tomar decisiones y realizar acciones sin la intervención del usuario.

Unidad 2: Representación del conocimiento y planificación

Sistemas basados en reglas (SBR)

Razonamiento - Generalidades

Se entiende por razonamiento al proceso mental de inferir una conclusión a partir de un conjunto de premisas. El razonamiento o inferencia no añade nuevo conocimiento (esto solo se hace mediante el aprendizaje).

- Razonamiento lógico, formal o deductivo: el razonamiento es válido cuando la verdad de las premisas implica necesariamente la verdad de la conclusión.
- Razonamiento no-lógico, informal o inductivo: se generaliza para todos los elementos de un conjunto la propiedad observada en un número finito de casos (el problema es que aunque algo se dé muchas veces, no significa que en el futuro va a volver a darse, por lo que aceptar la conclusión tiene que ver con probabilidades).

1 Introducción

Las reglas deterministas constituyen la más sencilla de las metodologías utilizadas en sistemas expertos. La base de conocimiento contiene el conjunto de reglas que definen el problema, y el motor de inferencia obtiene las conclusiones aplicando la lógica clásica a estas reglas.

2 La base de conocimiento

En los SE intervienen 2 elementos fundamentales:

- **Datos**: son los hechos conocidos para un problema particular. Se almacenan en la memoria de trabajo y son de naturaleza no permanente, dinámica.
- **Conocimiento**: es el conjunto de relaciones entre los objetos dentro del dominio del problema. Sea para un determinado problema: A y B → C el conocimiento está en la relación de implicancia. La relación que une los elementos es el conocimiento del dominio particular. En situaciones deterministas, las relaciones entre un conjunto de objetos se representan mediante un conjunto de reglas. La información almacenada en la base de conocimiento es de naturaleza permanente y estática, no cambia de una aplicación a otra, a menos que se incorporen al sistema experto elementos de aprendizaje.

Regla: es una afirmación lógica que relaciona dos o más objetos. Incluye la premisa y la conclusión, que consisten de expresiones lógicas con una o más afirmaciones conectadas mediante operadores lógicos.

Algunos sistemas imponen ciertas restricciones a las reglas con el propósito de facilitar su tratamiento sin perder generalidad:

- No permitir en la premisa el operador lógico o
- Limitar las conclusiones a expresiones lógicas simples (aquellas que contienen sólo una afirmación objeto-valor, sin conectores)

3 El motor de inferencia

El motor de inferencia obtiene nuevas conclusiones o hechos utilizando datos y conocimiento. Las conclusiones pueden ser simples (resultan de una regla simple) o compuestas (resultan de más de una regla).

Para obtener conclusiones simples se utiliza **Modus Ponens**: si la premisa es verdadera, la conclusión también es verdadera (si A → B y A=V, entonces B=V) pero si la premisa es falsa no se puede afirmar nada de la conclusión y **Modus Tollens**: si la conclusión es falsa, también lo es la premisa (si A → B y B=F, entonces A=F) pero si la conclusión es verdadera no se puede afirmar nada de la premisa.

Para obtener conclusiones compuestas se utilizan los siguientes mecanismos:

Explique el mecanismo de resolución. Ejemplo.

3.1 Mecanismo de Resolución

Consta de las siguientes etapas:

1. Las reglas son sustituidas por expresiones lógicas equivalentes.
2. Estas expresiones lógicas se combinan en otra expresión lógica.
3. Esta última expresión se utiliza para obtener la conclusión.

Ejemplo: Sea A=V y dadas R1: A → B y R2: B → C:

1. Teniendo en cuenta que X→Y es equivalente a -X o Y, se las transforma en R1: -A o B y R2: -B o C
2. Dichas reglas se pueden combinar R3: (-A o B) y (-B o C) y, teniendo en cuenta que B aparece con diferentes signos en ambas partes de la conjunción, se obtiene -A o C.
3. Se utiliza la regla -A o C en conjunto con los hechos para obtener la conclusión. De esta manera, dado que A=V, se concluye que C=V.

Explique el método de encadenamiento de reglas en SBR.

3.2 Encadenamiento de reglas

Esta estrategia puede utilizarse cuando las premisas de ciertas reglas coinciden con las conclusiones de otras.

1. Asignar a los objetos sus valores conocidos.
2. Ejecutar cada regla de la base de conocimiento y concluir nuevos hechos si es posible.
3. Repetir el paso 2 hasta que no se puedan obtener nuevos hechos.

Ejemplo: sean A=V y D=F, dadas R1: A → B, R2: C → D, R3: B y -C → E:
R1: V → B, R2: C → F
Por R1 (Modus Ponens): B = V. Por R2 (Modus Tollens): C = F
R3: V y -F → E
Por R3 (Modus Ponens): E = V

Explicar y ejemplificar encadenamiento de reglas orientado a un objetivo.

PRACTICAR

3.3 Encadenamiento de reglas orientado a un objetivo

Requiere del usuario seleccionar una variable objetivo. Luego, el algoritmo navega a través de las reglas en búsqueda de una conclusión para la variable objetivo. Si no se obtiene ninguna conclusión con la información existente, entonces el algoritmo fuerza a preguntar al usuario en busca de nueva información sobre los elementos que son relevantes para llegar a una conclusión sobre el objetivo. Su funcionamiento es el siguiente:

1. Asignar a los objetos sus valores conocidos. Marcar todos los objetos cuyo valor ha sido asignado. Si el nodo objetivo está marcado, ir al paso 7; en otro caso:
    a. Designar como objetivo inicial el objetivo en curso y marcarlo.
    b. Hacer ObjetivosPrevios = ∅.
    c. Designar todas las reglas como activas.
2. Encontrar una regla activa que incluya el objetivo en curso y ninguno de los objetos en ObjetivosPrevios. Si no se encuentra una regla, ir al paso 5.
3. Ejecutar la regla referente al objetivo en curso. Si concluye, asignar el valor concluido al objetivo en curso e ir al paso 6.
4. Si todos los objetos de la regla están marcados, declarar la regla como inactiva e ir al paso 2; en otro caso:
    a. Añadir el objetivo en curso a ObjetivosPrevios.
    b. Designar uno de los objetos no marcados en la regla como el objetivo en curso y marcarlo.
    c. Ir al paso 2.
5. Si el objetivo en curso es el mismo que el objetivo inicial, ir al paso 7; en otro caso, preguntar al usuario por el valor del objetivo en curso. Si se da un valor, asignar al objeto el valor dado.
6. Si el objetivo en curso no es el mismo que el objetivo inicial, designar el objetivo previo como objetivo en curso, eliminarlo de ObjetivosPrevios, e ir al paso 2.
7. Devolver el valor del objetivo en curso si es conocido.

Ejemplo: sean A=V y D=F, dadas R1: A → B, R2: C → D, R3: B y -C → E. Sea E el nodo objetivo:

Paso 1: Se asignan los valores A=V y D=F, y se marcan. Como el nodo objetivo no está marcado:

1. Se designa a M como objetivo en curso y se lo marca. ObjetosMarcados={A, D, E}
2. ObjetivosPrevios = ∅
3. ReglasActivas={1, 2, 3}

Paso 2: Se busca una regla que incluya el objetivo en curso E, encontrándose la Regla 3.
Paso 3: La regla 3 no puede concluir ya que B y C tienen valores desconocidos.
Paso 4: Los objetos B y C no están marcados, por lo que

1. ObjetivosPrevios = {E}
2. Se elige el objeto B como objetivo en curso y se lo marca. ObjetosMarcados={A, D, E, B}
3. Se va al paso 2.

Paso 2: Se busca una regla que incluya el objetivo en curso B encontrándose la regla 1.
Paso 3: Se ejecuta la regla 1 obteniéndose B=V. Se va al paso 6.
Paso 6: Se designa a E como objetivo en curso. ObjetivosPrevios=∅. Se va al paso 2.
Paso 2: Se busca una regla que incluya el objetivo en curso E, encontrándose la Regla 3.

Paso 3: La regla 3 no puede concluir ya que C tiene valor desconocido.
Paso 4: El objeto C no están marcados, por lo que

4. ObjetivosPrevios = {E}
5. Se elige el objeto C como objetivo en curso y se lo marca. ObjetosMarcados={A, D, E, B, C}
6. Se va al paso 2.

Paso 2: Se busca una regla que incluya el objetivo en curso C encontrándose la regla 2.
Paso 3: Se ejecuta la regla 2 obteniéndose C=F. Se va al paso 6.
Paso 6: Se designa a E como objetivo en curso. ObjetivosPrevios=∅. Se va al paso 2.
Paso 2: Se busca una regla que incluya el objetivo en curso E, encontrándose la Regla 3.
Paso 3: Se ejecuta la regla 3 obteniéndose E=V. Se va al paso 6.
Paso 6: El objetivo en curso es el objetivo inicial por lo que se va al paso 7.
Paso 7: Se devuelve el valor E=V.

En encadenamiento de reglas orientado a un objetivo, ¿cómo queda la pila de objetivos previos si se llega al valor de verdad del objetivo?
Siempre queda vacía

Explique qué son las reglas activas en el encadenamiento orientado a objetivo
Las reglas activas son aquellas que aún pueden usarse para concluir algo relevante para la variable objetivo. Una regla se marca como inactiva cuando todos sus objetos son objetivos (en curso o previos) o tienen valores conocidos.

3.4 Compilación de reglas

Se comienza con un conjunto de datos y se trata de alcanzar algunos objetivos. Cuando ambos, datos y objetivos, se han determinado previamente, las reglas pueden ser compiladas, es decir, pueden escribirse los objetivos en función de los datos para obtener las llamadas ecuaciones objetivo.

Ejemplo: Sean A=V y D=F, dadas R1: A → B, R2: C → D, R3: B y -C → E. Sea E el nodo objetivo:

Ecuaciones:
R1: B = A
R2: D = C
R3: E = B y -C = A y -D

Regla resultante:
A y -D → E

4 Control de la Coherencia

El objetivo del control de la coherencia consiste en ayudar al usuario a no dar hechos inconsistentes y evitar que entre en la base de conocimiento cualquier tipo de conocimiento contradictorio. El control de la coherencia debe hacerse controlando la coherencia de las reglas y de los hechos.

Tipos de incoherencia en Sistemas basados en reglas.

4.1 Coherencia de reglas

Explique coherencia de reglas.

Un **conjunto de reglas** se denomina **coherente** si existe, al menos, un conjunto de valores de todos los objetos que producen conclusiones no contradictorias. En consecuencia, un conjunto coherente de reglas no tiene por qué producir conclusiones no contradictorias para todos los posibles conjuntos de valores de los objetos, con uno es suficiente.

Es suficiente con realizar la comprobación de la coherencia de las reglas solo una vez, tras ser introducida cada regla.

¿Cuándo el conjunto de reglas que compone una BC es incoherente? Dé un ejemplo de un sistema de reglas incoherentes con 4 elementos, A,B,C,D.

Un conjunto de reglas es incoherente cuando todos los conjuntos de valores de los objetos producen contradicciones. Ejemplo:
R1: A → B R2: -A → -B R3: B → -A R4: C → D R5: D → -C R6: -B → C

¿Cuándo una regla es incoherente?:

- **Todos los conjuntos de valores de los objetos producen contradicciones.**
- Un conjunto de todos los valores de los objetos producen contradicciones.
- Un conjunto de todos los valores de los objetos no producen contradicciones.
- Ninguna de las anteriores.

4.2 Coherencia de hechos

Los datos suministrados por los usuarios también deben ser consistentes entre sí y con el conjunto de reglas de la base de datos. Por ello, el sistema no debe aceptar hechos que contradigan el conjunto de reglas o el conjunto de hechos existente en cada instante del proceso.

Se dice que **un valor a para el objeto A no es factible** si las conclusiones obtenidas al hacer A = a contradicen cualquier combinación de valores del resto de los objetos. La eliminación automática de valores no factibles de la lista de valores posibles de sus correspondientes objetos asegura la coherencia de hechos.

5 Explicando Conclusiones

Las reglas activas durante el proceso realizado por el motor de inferencia (las que han concluido) forman la base del mecanismo de explicación, que es regulado por el subsistema de explicación. En los sistemas expertos basados en reglas, es fácil dar explicaciones de las conclusiones obtenidas. El motor de inferencia obtiene conclusiones basándose en un conjunto de reglas y, por lo tanto, conoce de qué regla proviene cada conclusión. Por ello, el sistema puede dar al usuario la lista de hechos concluidos junto con las reglas que se han utilizado para obtenerlos.

Componentes y su relación en SBR. Dar ejemplo.

Los componentes fundamentales de un SBR son la base de conocimiento (BC), la memoria de trabajo (MT) y el motor de inferencia (MI). Para el funcionamiento de un SE es necesario que interactúen los 3 componentes: los conocimientos se almacenan en la BC, los datos en la MT, y el MI emplea el conocimiento para realizar inferencias a partir de los datos y obtener así conclusiones..

Reglas de asociación

Introducción

Las reglas de asociación son una clase importante de regularidades en los datos. Su objetivo es encontrar todas las relaciones de co-ocurrencia, llamadas asociaciones, entre elementos de datos. Se explora un conjunto de transacciones para determinar la probabilidad de que los objetos contenidos allí se relacionen entre sí. La aplicación clásica de reglas de asociación es la canasta de supermercado:
Queso → Cerveza [Soporte=10%, Confianza=80%]
El 10% de los clientes compran queso y cerveza juntos, y los que compran queso también compran cerveza el 80% del tiempo.

Una característica importante con estos métodos es la secuencialidad de las acciones, es decir, el orden en que se realiza cada acción: en la minería de reglas de asociación, no se considera la secuencia en la que se compran los artículos.

Conceptos básicos

Sea I = {i1, i2, …, in} el conjunto de todos los ítems.

Sea T = {t1, t2, …, tn} el conjunto de todas las transacciones donde cada transacción ti es un conjunto de ítems de manera que ti ⊆ I.

Una **regla de asociación** es una implicación de la forma:
X → Y, donde X ⊂ I, Y ⊂ I, X ∩ Y = ∅

La inclusión de X en I e Y en I no es amplia porque sino no se cumpliría que la intersección entre X e Y sea un conjunto vacío.

X (o Y) es un conjunto de ítems llamado itemset.

Una transacción t1 perteneciente a T contiene o cubre un conjunto de items X si ese conjunto de ítems es un subconjunto de dicha transacción. ti cubre a X si X ⊂ ti.

Soporte y confianza: definición, para que se realizan estas medidas, ejemplos.

El conteo de soporte de X en T (llamado X.count) es el número de transacciones en T que contienen X.

El **soporte** de una regla X → Y es el porcentaje de transacciones en T que contienen X ∪ Y. Determina qué tan frecuente una regla es aplicable en el conjunto de transacciones T. Es la probabilidad estimada P(X U Y).
Sea n el número de transacciones en T: $soporte = \frac{(X \cup Y).count}{n}$

La **confianza** de una regla X → Y es el porcentaje de transacciones en T que contienen X también contienen Y. Determina la predictibilidad de la regla. Es la probabilidad estimada P(Y/X). $confianza = \frac{(X \cup Y).count}{X.count}$

Explicar algoritmo Apriori (generación de itemsets y generación de reglas)

Algoritmo Apriori

El objetivo es encontrar todas las reglas de asociación en T cuyos valores de soporte y confianza sean mayores o iguales a los establecidos por el usuario. El algoritmo trabaja en 2 etapas:

1. Genera el conjunto de itemsets frecuentes (utiliza minsup)
2. Genera las reglas de asociación confiables del conjunto de itemsets frecuentes (utiliza minconf)

Explicar la propiedad de cierre hacia abajo y para qué sirve.

Para generar conjuntos de ítems frecuentes el algoritmo hace una poda utilizando la propiedad **downward closure**: si un conjunto de ítems tiene un valor mínimo de soporte, todos los subconjuntos no vacíos también tienen, al menos, el mismo valor de soporte. De esta manera, si un k-itemset candidato tiene un subconjunto que no está en los k-1-itemsets frecuentes, se descarta ya que no cumplirá con el minsup, ahorrándose dicho cálculo.

Generación del conjunto de itemsets frecuentes F

Los ítems deben estar ordenados en **orden lexicográfico** para evitar repeticiones (ya que no se distingue el orden en que se realizan las acciones). En cada paso del algoritmo se generan los conjuntos de ítems frecuentes de tamaño 1, 2, …, m (m-itemsets frecuentes) que superan el soporte mínimo establecido.

El algoritmo para la generación de ítems frecuentes recibe el conjunto de transacciones T y devuelve el conjunto de 1..m-itemsets frecuentes F:

```
Algorithm Apriori(T)
1   C1 ← init-pass(T);                          // counts the occurrences of each item
2   F1 ← {f | f ∈ C1, f.count/n ≥ minsup};      // n is the no. of transactions in T
3   for (k = 2; Fk−1 ≠ ∅; k++) do               // subsequent passes over T
4     Ck ← candidate-gen(Fk−1);
5     for each transaction t ∈ T do             // scan the data once
6       for each candidate c ∈ Ck do
7         if c is contained in t then
8           c.count++;
9       endfor
10    endfor
11    Fk ← {c ∈ Ck | c.count/n ≥ minsup}
12  endfor
13  return F ← ⋃k Fk;
```

Se carga C1 con todos los ítems y su cantidad de ocurrencias en las transacciones (línea 1), se carga F1 con los ítems de C1 cuyo soporte es mayor o igual al minsup (línea 2). Luego, para cada k desde 2 hasta que Fk-1 sea vacío:

1. Se generan los k-itemsets candidatos y se los guarda en Ck (línea 4).
2. Se cuentan las ocurrencias de cada candidato en las transacciones (líneas 5 a 10).
3. Se guarda en Fk los itemsets de Ck que cumplen con el minsup (línea 11).

Finalmente, se devuelve el conjunto F con todos los 1..m-itemsets que cumplen con el minsup (línea 13).

La función candidate-gen recibe un conjunto de k-1-itemsets frecuentes y devuelve un conjunto de k-itemsets candidatos.

¿En qué parte del algoritmo Apriori para reglas de asociación se utiliza la propiedad downward closure?

```
Function candidate-gen(F_{k-1})
1   C_k ← ∅;                                  // initialize the set of candidates
2   forall f_1, f_2 ∈ F_{k-1}                 // find all pairs of frequent itemsets
3     with f_1 = {i_1, ... , i_{k-2}, i_{k-1}}  // that differ only in the last item
4     and f_2 = {i_1, ... , i_{k-2}, i'_{k-1}}
5     and i_{k-1} < i'_{k-1} do               // according to the lexicographic order
6       c ← {i_1, ..., i_{k-1}, i'_{k-1}};    // join the two itemsets f_1 and f_2
7       C_k ← C_k ∪ {c};                      // add the new itemset c to the candidates
8       for each (k-1)-subset s of c do
9         if (s ∉ F_{k-1}) then               // downward clousure
10          delete c from C_k;
11      endfor
12  endfor
13  return C_k;                               // return the generated candidates
```

Se inicializa Ck con el conjunto vacío (línea 1) y para cada par de (k-1)-itemsets frecuentes en orden lexicográfico que difieren solo en el último elemento:

1. Se combinan dichos itemset respetando el orden lexicográfico, es decir, el último elemento del segundo itemset se añade al final del primero (línea 6).
2. Se añade el nuevo itemset c al conjunto de candidatos Ck (línea 7)
3. Si algún (k-1)-subconjunto de c no se encuentra en Fk-1, se elimina c de Ck ya que no superará el soporte mínimo por downward closure (líneas 8 a 11)

Finalmente, se devuelve el conjunto de k-itemsets candidatos Ck.

Dados:
t1: Beef, Chicken, Milk
t2: Beef, Cheese
t3: Cheese, Boots
t4: Beef, Chicken, Cheese
t5: Beef, Chicken, Clothes, Cheese, Milk
t6: Chicken, Clothes, Milk
t7: Chicken, Milk, Clothes

F1: {{Beef}:4, {Cheese}:4, {Chicken}:5, {Clothes}:3, {Milk}:4}
C2: {{Beef, Cheese}, {Beef, Chicken}, {Beef, Clothes}, {Beef, Milk}, {Cheese, Chicken}, {Cheese, Clothes}, {Cheese, Milk}, {Chicken, Clothes}, {Chicken, Milk}, {Clothes, Milk}}
F2: {{Beef, Chicken}:3, {Beef, Cheese}:3, {Chicken, Clothes}:3, {Chicken, Milk}:4, {Clothes, Milk}:3}

C3: {{Chicken, Clothes, Milk}} //se descarta {Beef, Chicken, Cheese} por downward clousure
F3: {{Chicken, Clothes, Milk}:3}

- Teóricamente es un algoritmo exponencial O(2^m) siendo m la cantidad de ítems en I. Sin embargo, el algoritmo explota el alto valor de soporte mínimo y la dispersión en los datos para que la minería sea posible y eficiente.
- El algoritmo puede escalar hasta grandes conjuntos de datos, ya que no carga toda la información en memoria.
- Utiliza la técnica de level-wise search, es decir, que puede detenerse en cualquier momento y no revisar todo el espacio de búsqueda.
- Dados T, minsup y minconf, el conjunto de reglas generadas es único.

Generación de las reglas de asociación

Explicación del funcionamiento del algoritmo genRules()

Para generar reglas para cada conjunto de ítems f, se trabaja con todos los subconjuntos no vacíos de f (α). Para cada subconjunto α se genera una regla (f-α) → α si $confianza = \frac{f.count}{(f-\alpha).count} \geq minconf$. Dado que independientemente del α, el soporte de f es el mismo, solo se debe calcular la confianza.

El algoritmo para la generación de reglas de asociación recibe el conjunto de 1..m-itemsets frecuentes F e imprime todas las reglas de asociación que cumplen con la confianza mínima.

Algorithm genRules(F) // F is the set of all frequent itemsets
1 **for** each frequent k-itemset f_k in F, $k \geq 2$ **do**
2 output every 1-item consequent rule of f_k with confidence ≥ *minconf* and support ← $f_k.count$ / n // n is the total number of transactions in T
3 $H_1 \leftarrow$ {consequents of all 1-item consequent rules derived from f_k above};
4 ap-genRules(f_k, H_1);
5 **endfor**

Para cada k-itemset frecuente fk (con k>2), primero se generan todas las reglas con un ítem como consecuente que cumplan la confianza mínima. Luego, con esos consecuentes se forma el conjunto H1, que se usa para generar H2..Hm (con k > m) mediante ap-genRules, imprimiendo en cada caso las reglas con 2..m consecuentes que cumplen la confianza mínima.

ap-genRules recibe un k-itemset frecuente fk y un conjunto Hm de m-consecuentes y genera las reglas con m+1-consecuentes que cumplan con la confianza mínima.

```
Procedure ap-genRules(f_k, H_m)        // H_m is the set of m-item consequents
1   if (k > m + 1) AND (H_m ≠ ∅) then
2     H_{m+1} ← candidate-gen(H_m);
3     for each h_{m+1} in H_{m+1} do
4       conf ← f_k.count / (f_k − h_{m+1}).count;
5       if (conf ≥ minconf) then
6         output the rule (f_k − h_{m+1}) → h_{m+1} with confidence = conf and
          support = f_k.count / n;    // n is the total number of transactions in T
7       else
8         delete h_{m+1} from H_{m+1};
9     endfor
10    ap-genRules(f_k, H_{m+1});
11  endif
```

Si k>m+1 y el conjunto Hm no es vacío:

1. Se generan los consecuentes candidatos Hm+1 a partir de Hm usando la función candidate-gen.
2. Para cada (m+1)-consecuente hm+1 se calcula la confianza de la regla (fk - hm+1) → hm+1. Si es mayor o igual a minconf se imprime la regla, sino se elimina a hm+1 de Hm+1
3. Se llama recursivamente al procedimiento para generar las reglas con m+2...k-1 consecuentes que cumplen con la confianza mínima.

Para el 3-itemset frecuente (F3):

Regla 1: Chicken, Clothes → Milk [conf = 3/3]

Regla 2: Chicken, Milk → Clothes [conf = 3/4] X

Regla 3: Clothes, Milk → Chicken [conf = 3/3].

Se descarta la regla 2 por no cumplir con la confianza mínima (80%)

H1 = {{Chicken}, {Milk}}

H2 = {{Chicken, Milk}}

Regla 4: Clothes → Milk, Chicken [conf = 3/3]

Redes Semánticas y Marcos

1 Introducción

En representación de conocimientos se puede hablar de tres modelos distintos:

- El **modelo conceptual** que es una representación de los conocimientos de un dominio, independientemente de cómo se implementen, utilizando estructuras no computables que modelizan el problema y la solución en un dominio concreto.
- El **modelo formal** que es una representación "semiinterna" o "semicomputable" de los conocimientos de un dominio. Este modelo formal se ha de obtener a partir del modelo conceptual.
- El **modelo computable** que hace que el modelo formal sea totalmente operativo, y que está formado por una base de conocimientos, un motor de inferencias y una serie de estrategias de control.

Formalizar consiste en representar simbólicamente los conocimientos de un dominio utilizando alguno de los formalismos de representación de conocimientos existentes, organizarlos de acuerdo a algún modelo de diseño y determinar los métodos de inferencia adecuados para manejar eficiente y efectivamente los conocimientos representados.

Cuando un IC aborda un nuevo problema, una de las decisiones que tiene que tomar es determinar el formalismo de representación más adecuado para la tarea que le corresponde realizar al sistema. En sistemas pequeños y sencillos, lo normal es que solamente se utilice un formalismo para representar el conocimiento; pero a menudo se combinan dos o tres formalismos de modo que se complementen unos a otros para lograr una representación que se ajuste de la forma más natural posible a los conocimientos del dominio. Cada uno de los formalismos existentes tiene asociadas unas técnicas básicas de inferencia.

Se puede realizar la siguiente clasificación:

- **Formalismos basados en conceptos**: representan las principales clases o entidades del dominio, sus propiedades, y los posibles valores que puede tomar cada propiedad. En este caso, se pueden distinguir entre las ternas Objeto-Atributo-Valor y los Marcos.
- **Formalismos basados en relaciones**: centran su atención en las relaciones que aparecen entre los conceptos o entidades del dominio. Los más importantes son: la Lógica, las Redes Semánticas y la Teoría de la Dependencia Conceptual.
- **Formalismos basados en acciones**: describen los conocimientos del dominio como un conjunto de acciones básicas. Los principales formalismos de este tipo son los Sistemas de Producción y los Guiones.

El formalismo de las **Redes Semánticas** consiste en un grafo orientado formado por nodos y arcos unidireccionales, ambos etiquetados. Los nodos representan conceptos y los arcos relaciones entre conceptos.

El formalismo de **Marcos** es una estructura de datos para representar estereotipos. Cada marco está formado por un nombre y un conjunto de propiedades. Los valores de cada propiedad se describen en el marco utilizando un conjunto de facetas. Los marcos se unen unos con otros mediante relaciones. La relación subclase-de permite crear jerarquías de conceptos por especialización y proporciona un camino para la herencia de propiedades. Las

principales características que han hecho de los marcos el formalismo de representación más utilizado cuando los conocimientos del domino están basados en conceptos son: la organización de los conceptos que intervienen en el problema en jerarquías o taxonomías de conceptos, y la posibilidad de representar conocimientos procedimentales y declarativos en los marcos de una forma integrada.

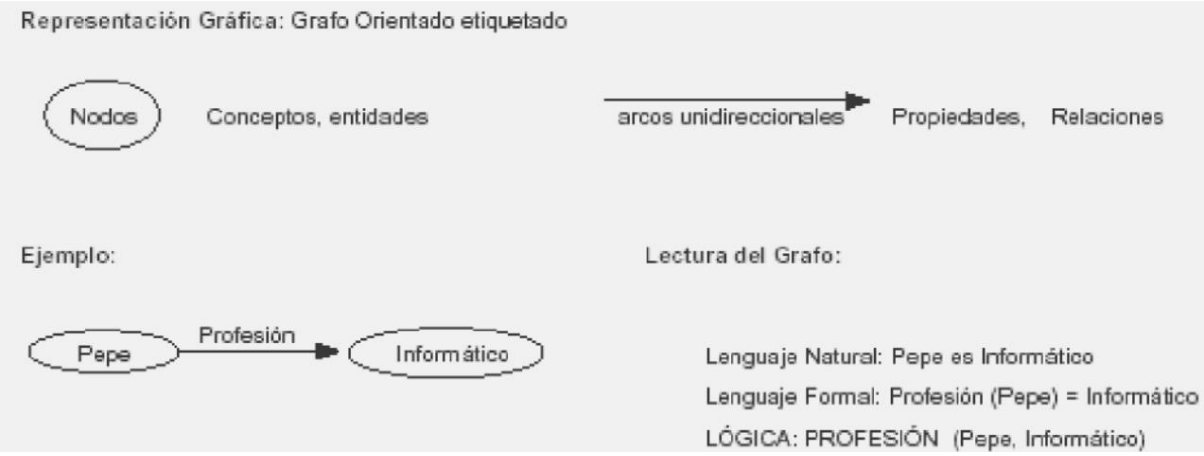
2. Redes semánticas

Concepto de redes semánticas y los tipos de relaciones que hay.

Una red semántica es un grafo orientado formado por nodos etiquetados, que representan conceptos e instancias, y arcos unidireccionales etiquetados, que representan relaciones entre conceptos o instancias.

Relaciones:

- Asociación: describen o relacionan entidades. Ej: la Profesión de Pepe es Informático
- Generalización (subclase-de): relaciona una clase con otra más general. Ej: Hombre es una subclase de Persona
- Instancia: liga un objeto concreto con su tipo genérico. Ej: Pepe es un Hombre
- Agregación (parte-de): liga un objeto con sus componentes. Ej: los Ojos son parte de la Cara



2.1 Representación de conocimiento

Se puede utilizar cualquier etiqueta para dar nombre a cualquier nodo o arco. Los principales problemas de este formalismo de representación son la falta de una terminología adecuada y universalmente aceptada, y de una semántica uniforme y precisa. Sin embargo, su gran atractivo gráfico y su intuitiva interpretación las hace rápidamente comprensibles y utilizadas en la formalización de bases de conocimiento.

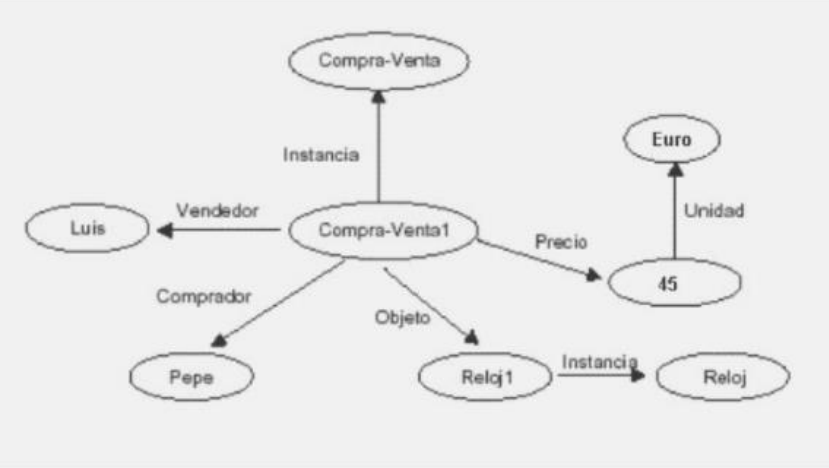
La base de la representación de conocimientos en las redes semánticas consiste en modelar los conocimientos relativos a un objeto o concepto mediante pares atributo-valor. Los pares se representan en el grafo orientado de la siguiente manera: el nodo origen (Pepe) es el objeto o concepto para el cual se definen los pares atributo-valor, los arcos que parten de dicho nodo (profesión, color-pelo, etc.) son los atributos del par, y los nodos destinos (informático, rubio, etc.) representan los valores de los atributos.

Los arcos en las redes conceptuales se agrupan en dos categorías:

- **Arcos descriptivos**: describen entidades y conceptos (profesión, color-pelo).
    - Arcos que relacionan dos entidades independientes ya existentes (el arco profesión)
    - Arcos utilizados para definir una nueva entidad (los nodos Edad1 y Edad2 son nuevos conceptos que representan la edad de Pepe y de Luis, y que se definen por sus relaciones con dichos nodos).
- **Arcos estructurales**: enlazan las entidades formando la arquitectura o estructura de la red (subclase-de, instancia y parte-de). A diferencia de los arcos descriptivos, la semántica de los arcos estructurales es independiente de los conocimientos del dominio que se está representando. No obstante, el IC puede definir, a medida, tantas etiquetas estructurales como crea oportuno. Estos procesos básicos asociados a arcos estructurales se pueden definir de la siguiente forma:
    - La generalización pone en relación una clase con otra más general. Las propiedades definidas en los nodos generales se heredan a los nodos específicos (Pepe por ser Hombre y por ser Persona tiene Cabeza).
    - El arco instancia liga un objeto concreto con su tipo genérico (Pepe es un Hombre).
    - La agregación liga un objeto con sus componentes (la Cara forma parte de la Cabeza).

2.2 Representación de predicados no binarios

Los predicados de aridad distinta a dos se pueden representar también en redes semánticas: el predicado de aridad uno HOMBRE(Pepe) equivaldría a INSTANCIA(Pepe, Hombre). Cuando se utilizan predicados de aridad tres o superior, un nuevo objeto representa al predicado de aridad mayor que dos, y nuevos predicados binarios describen las relaciones entre este nuevo objeto y sus argumentos. Por ejemplo, al representar el predicado COMPRA-VENTA(Pepe, Luis, Reloj1, 45, euros) en una red semántica, se crearía un nodo que representa la compra-venta (compra-venta1) y cinco predicados (vendedor, comprador, objeto, precio, unidad) que representan las relaciones con las cinco piezas de información.

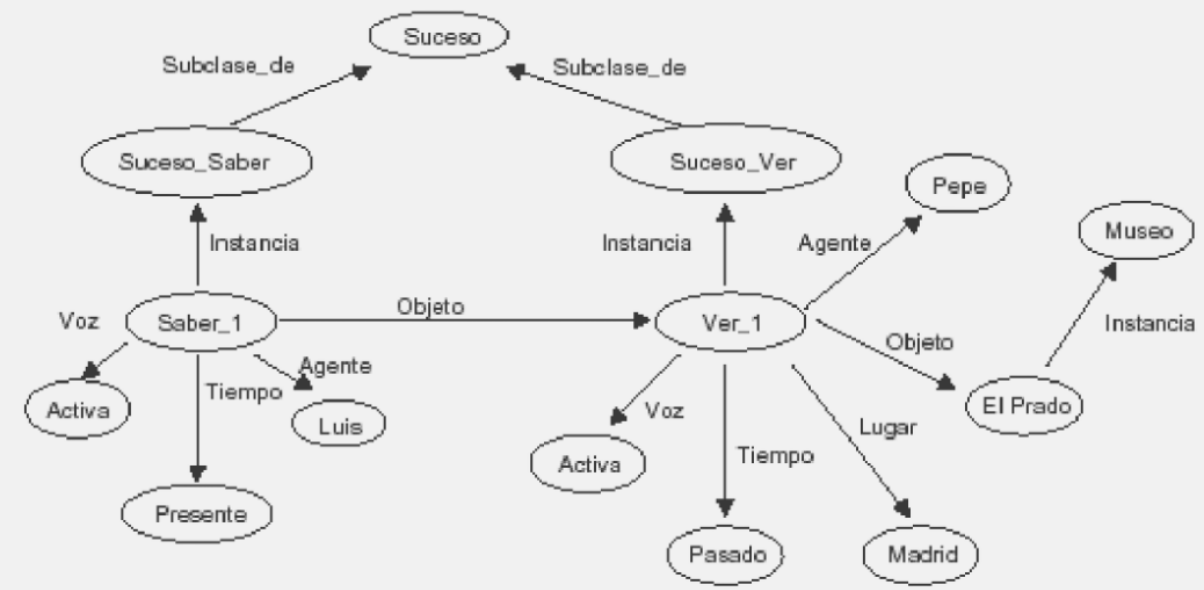


2.3 Representación de acciones

La gramática de casos se basa en que toda proposición contenida en una sentencia tiene una estructura formada por un verbo y una o más frases nominales que se relacionan con el verbo

mediante un conjunto de casos (agente, contra-agente, objeto, resultado, instrumento, origen, propósito, lugar, tiempo y sujeto). La modalidad, por su parte, hace referencia a características que presenta el verbo (tiempo y voz).

Para representar afirmaciones que no se refieren a atributos de los objetos, sino a acciones y eventos en una red semántica, se utiliza la gramática de casos. Ej: “Luis sabe que Pepe vio el Prado en Madrid”



2.4 Representación de conocimiento disjunto

Conceptos o situaciones disjuntos pueden representarse en la red semántica utilizando la etiqueta disjunto.



Sin embargo, existen unos arcos especiales que permiten representar estos conocimientos de manera más sencilla:

- S : Subconjunto.
- SD : Subconjunto disjunto.
- E : Elemento.
- ED : Elemento disjunto.



3 Inferencia de conocimiento en redes semánticas

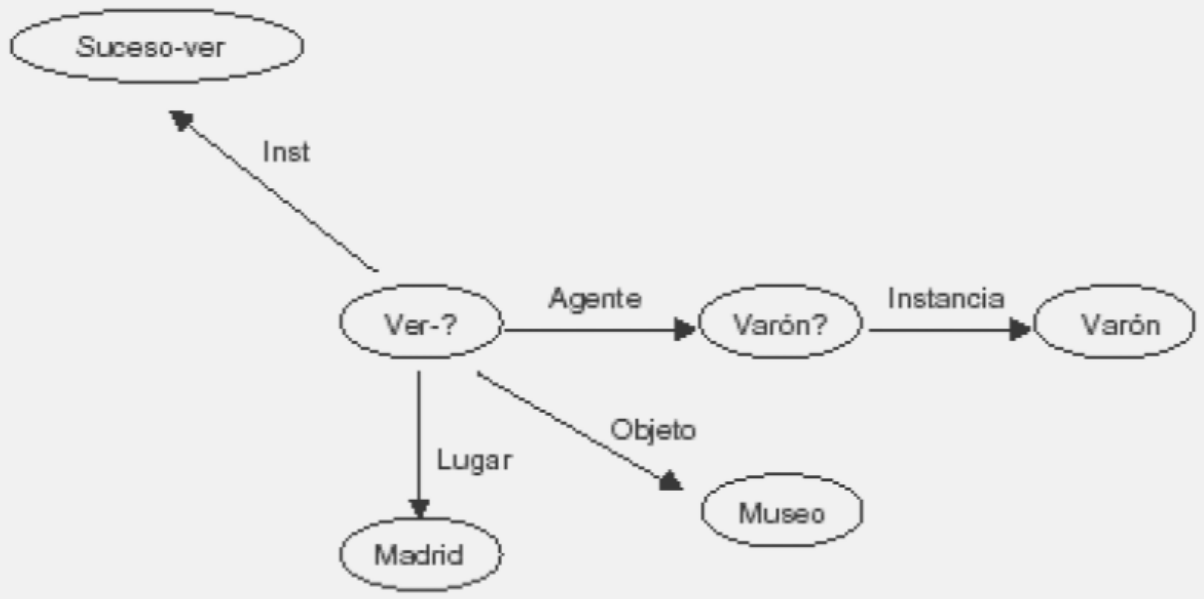
La eficacia del razonamiento en las redes depende de los procedimientos que trabajan con la semántica de sus arcos. Las técnicas más empleadas son la equiparación y la herencia de propiedades.

3.1 Equiparación

Un fragmento de red, apunte o consulta se equipara con una red semántica, si el apunte se puede asociar con un fragmento de la red semántica.

Dada la consulta "¿Existe algún varón que viera un museo en Madrid?", los pasos a seguir en el proceso de equiparación son:

1. Se construye un apunte que responda a la pregunta en cuestión utilizando los criterios seguidos en la construcción de la red semántica. El apunte está formado por un conjunto de nodos constantes (datos conocidos de la pregunta: varón, Madrid, museo, suceso-ver), nodos variables (valores que se requieren y, por consiguiente, son desconocidos, Ver-? y Varón?) y arcos etiquetados (instancia, agente, lugar, objeto) que unen nodos constantes y nodos variables entre sí.
2. Se coteja o superpone el apunte sobre la red semántica en la base de conocimientos.
3. Los nodos variables se ligan a los nodos constantes de la red hasta encontrar una equiparación perfecta. Con este fin, en el ejemplo, se busca en la red un nodo situación del que parta un arco Lugar hacia un nodo Madrid, y un arco Instancia hacia el nodo Suceso-Ver. Cuando esto ocurre, se liga la variable Ver-? del apunte con el nodo situación de la red, Ver-1, y el nodo Varón? del apunte con el nodo constante de la red Pepe. Dado que en el apunte se conoce que se trata de un varón, es necesario comprobar en la red que desde Pepe parte un arco Instancia hacia el nodo Varón.
4. La respuesta a la consulta es el fragmento de red semántica con los valores con los que se rellenan los nodos variables. En el ejemplo: Ver-? = Ver-1 y Varón? = Pepe. Si no hubiera ninguna equiparación del apunte con la red semántica, la respuesta dada por la técnica de equiparación sería "No se ha encontrado un varón que haya visto un museo en Madrid".

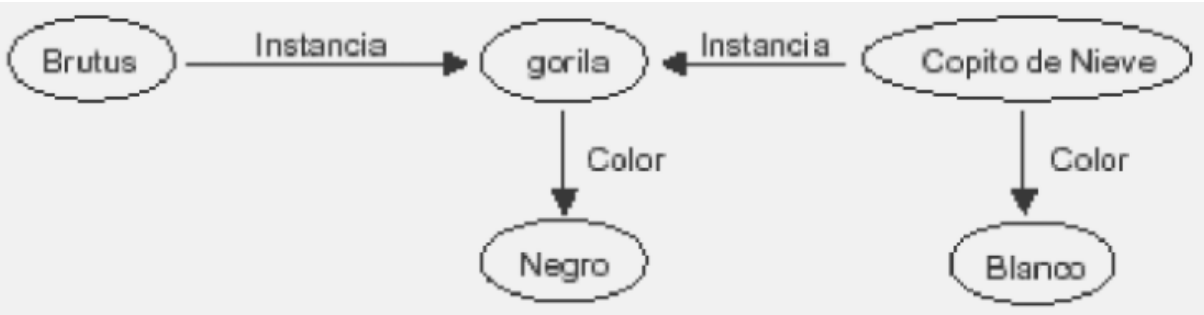


3.2 Herencia de propiedades

El concepto de herencia de propiedades tiene su fundamento teórico en la regla del modus ponens. La herencia de propiedades permite que nodos específicos de una red accedan a las propiedades definidas en otros nodos utilizando los arcos Instancia y Subclase-de, favoreciendo así la compartición de propiedades entre diferentes nodos y evitando la repetición de propiedades en la base de conocimiento.

Supóngase que se quiere determinar la veracidad de la sentencia “Dumbo es de color gris”. En primer lugar se debe localizar el nodo Dumbo, y después se debe buscar si desde dicho nodo sale un arco con la etiqueta Color. Al no existir el susodicho arco en Dumbo, el sistema buscará arcos Instancia que partan desde dicho nodo hacia algún otro nodo. En este caso, existen dos arcos con destino Elefante y Macho, y el motor de inferencias elegirá alguna de las dos alternativas. Si elige el nodo Elefante, como desde Elefante parte el arco buscado hacia el nodo Gris, entonces el motor de inferencias devolverá que la sentencia “Dumbo es de color gris” es cierta. Pero si se elige el nodo Macho, al no encontrarse la propiedad en él, utilizando el arco Subclase-de se buscará si dicha etiqueta se encuentra en sus superclases. Dado que en Animal no se encuentra la propiedad y que en Ser-Vivo tampoco, entonces se retrocederá y se empezará a explorar el resto de alternativas.

La herencia de propiedades trabaja muy bien con propiedades que presentan excepciones en sus valores. La distribución de las propiedades en la red permite que se herede el valor de la propiedad del nodo más cercano al nodo que sirvió como punto de partida en la inferencia. Por ejemplo, si se pregunta por el color de Brutus, utilizando la herencia de propiedades se deduce que “Brutus es de color Negro”. Sin embargo, si se pregunta por el color de Copito de Nieve, el valor que se obtiene es Blanco, porque desde Copito de Nieve sale un arco etiquetado con color hacia el nodo Blanco.



4 Marcos

Los marcos son una estructura de datos que representa situaciones estereotipadas construidas sobre situaciones similares ocurridas anteriormente, permitiendo así aplicar a situaciones nuevas los conocimientos de situaciones, eventos y conceptos previos. Los conocimientos que se expresan en los marcos son conocimientos declarativos del dominio.

Los marcos organizan los conocimientos del dominio en árboles, también llamados jerarquías, o en grafos, ambos construidos por especialización de conceptos generales en conceptos más específicos. Las técnicas de inferencia utilizadas para razonar con los conocimientos de la base de conocimientos son: equiparación, para clasificar entidades en una jerarquía; herencia simple y herencia múltiple, para compartir propiedades que están distribuidas en la jerarquía de conceptos o en el grafo, respectivamente; y valores activos y métodos para representar la conducta del sistema.

4.1 Representación de conocimiento

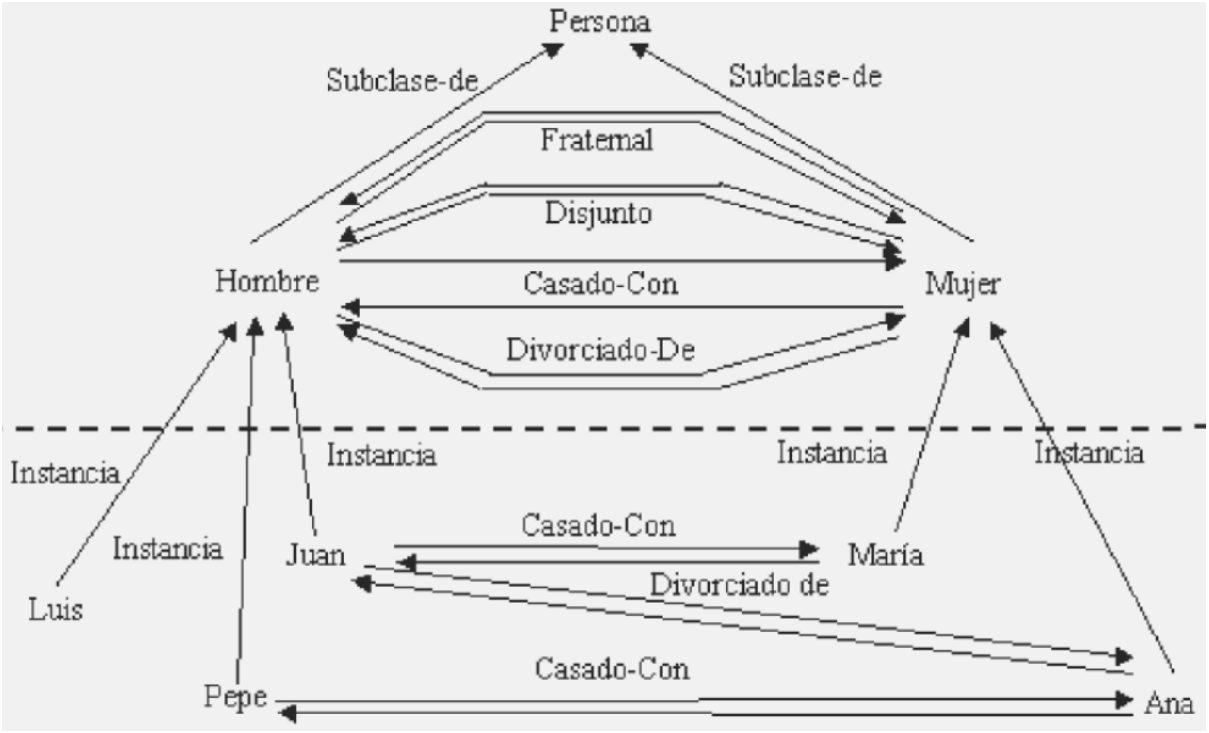
Elementos de frames.

Se utilizan **marcos** para representar conceptos o elementos, **relaciones** para expresar dependencias entre conceptos, **propiedades** para describir cada concepto, y **facetas** para expresar de múltiples formas los valores con los que se puede rellenar cada propiedad.

4.1.1 Representación de conceptos e instancias

Básicamente, existen dos tipos de marcos:

- **Marcos clase:** se utilizan para representar conceptos, clases o situaciones genéricas descritos por un conjunto de propiedades, unas con valores y otras sin valores asignados, que son comunes al concepto, clase o situación que el marco representa.
- **Marcos instancia:** pueden considerarse como la representación en el dominio real de una clase determinada. Estos marcos instancia deben estar relacionados, como mínimo, con un marco clase. Además, han rellenado la mayoría de sus propiedades con valores específicos; es decir, con información concreta del elemento, instancia o individuo que representan. El resto de propiedades que no estén almacenadas en él, las hereda de los marcos clase de los cuales son instancias.



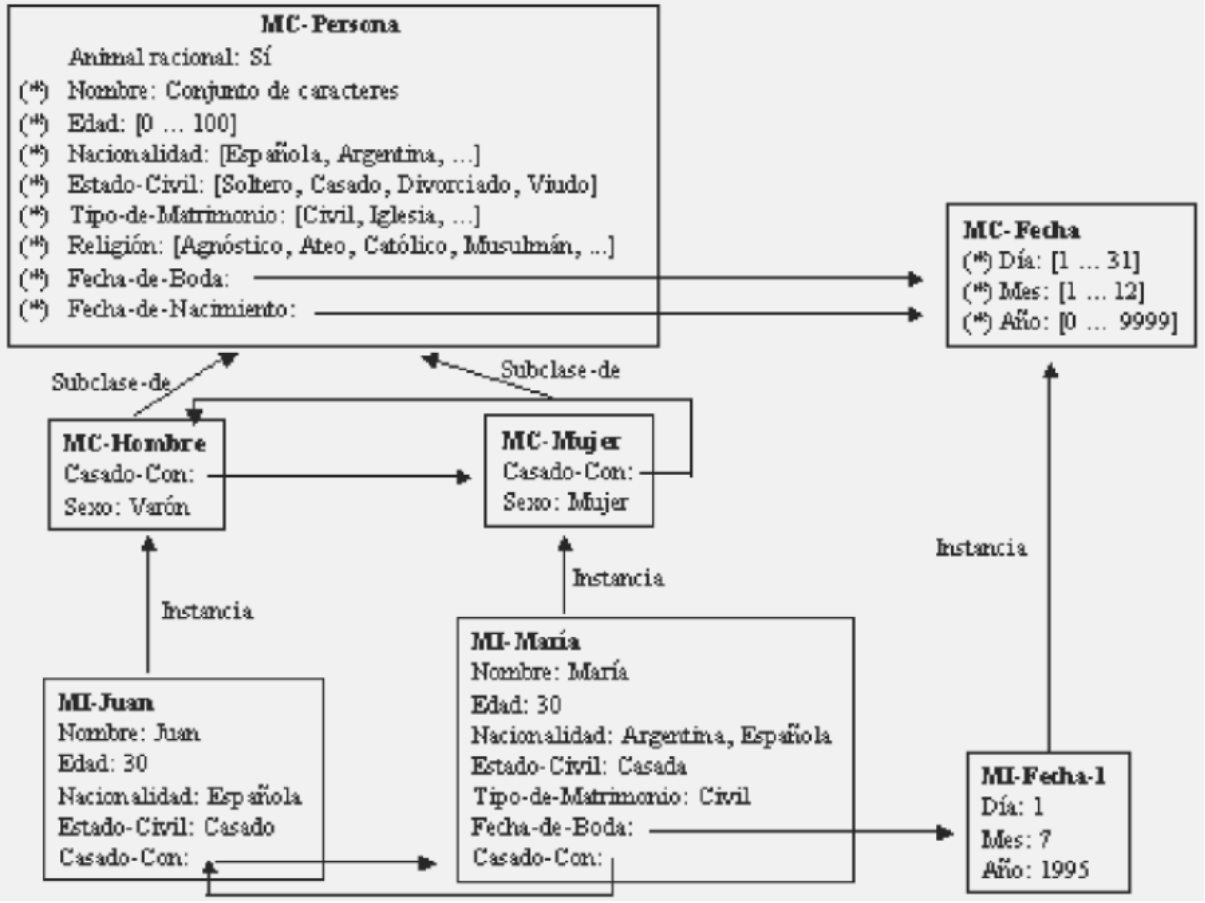
4.1.2 Representación de relaciones entre conceptos

Al igual que ocurría en el formalismo de las redes semánticas, hay ciertas relaciones que son independientes del dominio, y que reciben el nombre de **relaciones estándar** (subclase-de e instancia, y sus respectivas relaciones inversas llamadas superclase-de y representa). Sin embargo, también existen otras relaciones llamadas **relaciones no estándar**, para representar relaciones "a medida" entre conceptos de un dominio (fraternal -representa que dos conceptos son hermanos y, por consiguiente, tienen que tener el mismo padre-, disjunto -las clases representadas por ambos marcos no tienen ningún elemento en común- y no-disjunto -pueden tener instancias comunes-). Dentro de las relaciones no estándares, podemos definir relaciones a medida o **ad-hoc**, como pertenece, casado-con, divorciado-de, conectado- a, etc. Cuando se definen relaciones ad-hoc entre dos marcos instancia, se debe comprobar que dicha relación haya sido definida entre las clases y que los marcos instancia que se desea conectar son instancias de dichas clases.

4.1.3 Representación de las propiedades de los conceptos

Las **propiedades de clase** representan atributos o características genéricas de un concepto o clase. Se definen y rellenan en el marco clase, y toman siempre el mismo valor en todos los elementos o instancias de la clase.

Las **propiedades de instancia**, aunque se definen en el marco clase y son comunes a todas las instancias del marco clase, se rellenan en cada instancia con valores concretos que dependen del elemento de la clase que se esté representando. Gráficamente, si la propiedad va precedida del símbolo "*" se trata de una propiedad de instancia.



4.1.4 Representación de facetas de propiedades

Identificadas las relaciones y distribuidas las propiedades en los marcos clase, el IC pasa a describir las facetas de cada una de las propiedades. Las facetas expresan de múltiples formas los valores con los que se puede rellenar cada propiedad. Permiten modelar características de las propiedades y relaciones en los marcos clase. Declarativamente, una propiedad se define especificando un puntero o un tipo de datos (lógico, carácter, número, entero, real, conjunto restringido de valores numéricos, caracteres, etc.); procedimentalmente, se define utilizando un procedimiento o una regla.

Las facetas, independientemente de que se rellenen con valores, punteros, o procedimientos, se clasifican en las siguientes tres categorías:

- Facetas que definen propiedades de clase, de instancia y relaciones: se especifican en cualquier propiedad o relación.
    - Tipo ranura: establece el tipo de datos con el que se rellenará la propiedad o relación (si se trata de relaciones, la relación se definirá siempre en el marco clase origen de la relación, tendrá como nombre el de la relación, y se deberá especificar que se trata de un marco).
    - Cardinalidad mínima: número mínimo de valores con los que se rellena la ranura, siempre que ésta se rellene.
    - Cardinalidad máxima: número máximo de valores con los que se puede rellenar la ranura.
    - Multivaluada: establece si la propiedad puede tener más de un valor o no.
- Facetas que define propiedades de clase y relaciones

    - Propiedad general: se rellena con las propiedades de clase que se definieron en la faceta tipo ranura como un tipo de datos.
- Facetas que definen propiedades de instancia: se establecen para cada una de las propiedades de instancia definidas en un marco clase.
    - Valores Permitidos: conjunto de valores válidos que puede tomar la propiedad de instancia (un tipo de datos, un rango de valores o un puntero a un marco clase)
    - Valores por Omisión: valores que debe tomar la propiedad de instancia en un marco instancia si no se conoce de forma explícita otro valor.
    - Si necesito: procedimiento o regla que se ejecuta al solicitar el valor de una propiedad de instancia en un marco instancia y ser desconocido dicho valor (usado para solicitar el valor al usuario, comprobar restricciones, obtener valores en forma dinámica o determinar dinámicamente el rango de valores).
    - Si Modifico: procedimiento que se ejecuta al modificar un valor de una propiedad de un marco instancia. La ejecución de este procedimiento puede añadir, modificar y borrar valores de otras ranuras, disparando así sus procedimientos asociados.
    - Si Añado: procedimientos que se ejecutan al introducir un valor en una propiedad de un marco instancia que estaba vacía.
    - Si Borro: procedimiento que se ejecuta al borrar un valor en una propiedad de un marco instancia.

El motor de inferencias usa las facetas para mantener la integridad semántica de los datos, es decir, para comprobar que los valores introducidos en las propiedades realmente pertenecen al tipo especificado, en la obtención de valores, y en la asignación de valores por defecto.

4.2 Criterios de diseño

- Se debe favorecer la compartición de propiedades de clase y de instancia entre marcos.
- Se debe evitar representar conocimientos redundantes.
- En cada marco clase debe haber una propiedad de clase que identifique a los elementos de dicha clase.
- Las propiedades deben tener o aportar significado a la representación.
- Debido al carácter local de las propiedades, se pueden tener propiedades repetidas con el mismo nombre en diferentes marcos clase.
- En caso necesario, se pueden redefinir las propiedades (de clase e instancia) en marcos clase más específicos.
- Si se desea conocer información o propiedades de las propiedades de un marco, entonces dichas propiedades deben ser definidas como marcos.
- No se deben mezclar conceptos.
- En un marco instancia se pueden rellenar, o no, todas las propiedades de instancia definidas en los marcos clase con los que está conectado.
- Las propiedades de instancia se rellenan con un valor concreto.
- No se pueden utilizar en las instancias propiedades que no se hayan definido en los marcos clase.

5 Inferencia de conocimiento en SBM

Las tres técnicas que utilizan los marcos, de forma integrada o de manera individualizada, son:

- La equiparación para clasificar entidades en la BC.
- La herencia de propiedades para compartir propiedades entre marcos.
- Los valores activos para representar la conducta del sistema y mantener la integridad de los datos almacenados.

5.1 Equiparación

Esta técnica es especialmente útil en sistemas que clasifican o que se enfrentan a situaciones parecidas a otras que ocurrieron anteriormente. Conocidos los valores de un conjunto de propiedades que describen parcialmente un marco pregunta, la técnica de equiparación clasifica el marco pregunta en el árbol o en el grafo que representa los conceptos del dominio. Para ello, debe encontrar los marcos clase (uno o varios) de la BC que describen más consistentemente al marco pregunta. Encontrados los marcos clase, el marco pregunta se convierte en un marco instancia de dichos marcos clase.

1. Selección de marcos candidatos:
    - Si el tipo del marco pregunta es conocido, se puede seleccionar el marco clase en el que se ha definido el tipo, y todos los marcos clase en los que éste se ha especializado.
    - Si el tipo de la nueva entidad es desconocido, se eligen aquellos marcos clase en los que, como mínimo, se encuentre definida una propiedad conocida en el marco pregunta.
2. Cálculo del valor de equiparación (VE) de la nueva entidad en cada una de las clases seleccionadas: el VE es una medida que informa del grado de idoneidad de la equiparación que se va a realizar. El método de cálculo del VE variará de unas aplicaciones a otras. Por ejemplo, si los valores de la propiedad en la nueva entidad no coinciden o no están incluidos en los valores almacenados en la clase, y estas propiedades son esenciales, entonces el VE de la nueva entidad con la clase puede ser cero. Pero si las propiedades no son esenciales, la opción podría ser disminuir el VE. El IC, según sean las características de la aplicación, elaborará una fórmula para el cálculo del VE.
3. Decisión de con qué marcos clase se equiparará la nueva entidad: si el VE es lo suficientemente alto y el marco es lo suficientemente específico, entonces el sistema no buscará otros marcos e instanciará la nueva entidad convirtiéndola en un marco instancia. Sin embargo, si el VE no es lo suficientemente alto, o el marco no es lo suficientemente específico, entonces la técnica tendrá que identificar marcos relevantes. Esto puede realizarse de varias maneras (ascender a lo largo de la jerarquía hasta encontrar una equiparación relevante, comenzar en el marco raíz y seleccionar el marco con el mejor VE, buscar marcos relacionados utilizando las relaciones Fraternal, Disjunto, No-Disjunto y/o relaciones ad-hoc).

5.2 Herencia de propiedades

Explique cuál es el rol de la herencia en los sistemas de frames y cómo se implementa dicho mecanismo.

La herencia se utiliza para compartir propiedades en los marcos, evitando repeticiones, y como mecanismo de inferencia. Se puede diferenciar entre herencia simple, que se aplica a BC en forma de árbol, y herencia múltiple, que se aplica a BC en forma de grafo.

- **Herencia simple (árbol)**: se aplica a una BC formalizada en marcos cuando sólo existe un único camino que une el marco instancia con el nodo raíz de la jerarquía.
    1. Se busca la propiedad en el marco instancia. Si se encuentra, se devuelven sus valores. En caso contrario, se accede al marco clase padre utilizando la relación instancia.
    2. Se busca la propiedad en el marco clase. Si se encuentra la propiedad, se devuelven sus valores. En otro caso, se utiliza la relación subclase-de para acceder al marco clase padre. Este paso se repite mientras el padre no sea el marco raíz del árbol.
    3. Se busca la propiedad en el marco raíz. Si se encuentra, se devuelven sus valores. Si no, la técnica debe responder que con los conocimientos almacenados en la BC es imposible proporcionar una respuesta. La ejecución de un procedimiento asociado a la faceta Si Necesito, o la herencia de un valor por omisión, permitirán, en última instancia, responder a la pregunta. Nótese que en caso de estar definidas ambas facetas, debe existir una estrategia de control que ayude al SBM a decidir si ejecuta un procedimiento o si toma el valor por omisión.
- **Herencia múltiple (grafo)**: se aplica a una BC formalizada en marcos si existen varios caminos que unen los marcos instancia con el nodo raíz de la jerarquía. Dado que un hijo puede heredar propiedades de varios padres, la dificultad se encuentra en determinar el orden en el cual las clases se deben explorar: si los padres tienen propiedades con distinto nombre nunca habrá conflicto pero si los padres tienen propiedades con el mismo nombre, el valor de la propiedad que hereda el marco hijo dependerá de la técnica de búsqueda que se utilice para recorrer el grafo.
    - Búsqueda en profundidad. Esta técnica explora en profundidad todos los posibles caminos que van desde el marco instancia al marco raíz del SBM. Criterios que pueden usarse: recorrer el grafo de izquierda a derecha, usar el criterio de exhaustividad para evitar la búsqueda reiterada de propiedades en marcos clase ya analizados, buscar la propiedad en una clase sólo si previamente se ha buscado en todas sus subclases (para evitar tomar valores de clases generales cuando hay excepciones en clases más específicas). Un algoritmo que cumple los tres requisitos anteriores es el procedimiento de ordenación topológica que almacena en una lista el orden en el que se recorrerán todos los marcos clase accesibles desde un marco instancia concreto.
    - Búsqueda en amplitud o Longitud del camino: recorre el grafo por niveles que están a igual distancia del marco instancia. Problemas: si el grafo contiene propiedades con excepciones deberá razonar en presencia de ambigüedades y de conocimientos no especializados, y la longitud de un camino puede no corresponder con el nivel de generalidad de una clase si hay clases generales que no se han especializado con el mismo nivel de detalle que sus clases hermanas
    - Distancia “inferencial”: la condición necesaria y suficiente para que la clase1 esté más cercana a la clase2 que a la clase3 es que la clase1 tenga un camino

de inferencia a través de la clase2 hacia la clase3; en otras palabras, si la clase2 está entre la clase1 y la clase3. Sin embargo, al no permitir comparar marcos no conectados, el orden introducido por la distancia “inferencial” es un orden parcial y, por consiguiente, se pueden heredar valores contradictorios definidos en ramas que no están conectadas.

5.3 Valores activos

Además de la estructura declarativa de los marcos, también existe un aspecto dinámico o procedimental de los mismos (demonios, valores activos o disparadores). En muchos sistemas, estos procedimientos son el mecanismo principal para realizar razonamientos que no están cubiertos por las otras dos técnicas explicadas anteriormente.

Los procedimientos pueden simular encadenamiento hacia adelante teniendo demonios dirigidos por eventos, que ejecutan procedimientos antes de almacenar o borrar valores en las propiedades de un marco instancia y están asociados a las facetas Si Añado, Si Modifico y Si Borro, y encadenamiento hacia atrás teniendo demonios dirigidos por metas, que están asociados a la faceta Si Necesito, y deducen valores de propiedades a partir de valores almacenados en otras propiedades.

Árboles de decisión

1. Conceptos básicos

Un conjunto de datos utilizados en un problema de clasificación es un conjunto de registros que se describen mediante un conjunto de atributos: A = {A1, A2, …, A|A|} (donde |A| denota el número de atributos) y, asociado a ellos, un atributo más que es la clase a la que pertenece cada dato (etiqueta de clase): C = {c1, c2, …, c|C|}. Dado un dataset D, el objetivo es producir una **función de clasificación/predicción** (también llamada clasificador, modelo de clasificación o modelo predictivo) que relacione los valores de los atributos de A y las clases de C.

La idea es, a partir de los datos, poder construir un modelo que, dada una **nueva** instancia, pueda predecir a qué clase pertenece. Este tipo de actividad se conoce como aprendizaje supervisado y está caracterizado porque los datos tienen asociados la clase a la que pertenecen.

Para la construcción de los modelos que permiten predecir casos futuros hay dos conjuntos derivados del conjunto de datos que son fundamentales: el conjunto de entrenamiento (se utiliza para definir el modelo, o sea, el que se utiliza en el proceso de aprendizaje o ajuste de parámetros del modelo) y el conjunto de test (una vez que se tiene el modelo, se utiliza este conjunto para evaluar su precisión). No hay diferencia en las características de estos conjuntos: dado un conjunto de datos etiquetados se lo suele dividir, por ejemplo ¾ se utilizan para entrenar el algoritmo y ¼ restante para probar que el algoritmo responde de la manera esperada a las nuevas situaciones, comparando el resultado del algoritmo con el conocido.

Precisión = Número de clasificaciones correctas / número total de casos de test.

El proceso de definición de un modelo preciso puede ser representado como: Información de entrenamiento → Algoritmo de aprendizaje → Modelo → Información de test → Precisión

2. Inducción utilizando árboles de decisión

Los árboles de decisión son uno de los modelos más utilizados y precisos para la clasificación de instancias utilizando aprendizaje supervisado. Tienen 2 tipos de nodos: los nodos de decisión (especifican algún test sobre un atributo) y los nodos hoja que indican la clase.

El algoritmo más representativo de árboles de decisión es **C4.5** desarrollado por Quinlan. La idea de los árboles de decisión es representar en una estructura jerárquica todos los posibles casos del conjunto de entrenamiento para luego, ante la presentación de una nueva instancia, determinar a qué clase pertenece. Tienen como gran ventaja que son fáciles de entender.

Un árbol de decisión se construye particionando el conjunto de entrenamiento de manera tal de encontrar subconjuntos lo más puros posibles. Un **subconjunto puro** es aquel que contiene solo ejemplos de entrenamiento de una sola clase.

Ahora bien, dado un conjunto de entrenamiento, existen muchos árboles que lo representan. Por cuestiones prácticas, los mejores árboles son aquellos lo más simples y precisos posible. Por lo general un árbol simple es más preciso, y es más fácil de comprender para los usuarios.

Los valores (x/y) en los nodos hoja indican que de los y ejemplos de entrenamiento que llegaron a ese nodo hoja, x tienen la clase indicada en dicho nodo hoja. En la realidad, en general x<y siendo **x/y el concepto de confianza** visto en reglas de asociación (donde x es el conteo de soporte -(X U Y).count-). De hecho, un árbol de decisión puede convertirse en un conjunto de reglas si-entonces donde cada camino desde la raíz a una hoja forma una regla (los nodos de decisión forman el antecedente, y el nodo hoja el consecuente). Sin embargo, **los árboles de decisión encuentran solo un subconjunto de reglas** basadas en los datos, las cuales son suficientes para realizar una clasificación de los mismos. Own_house = false, Has_job = true → Class = Yes [sup=3/15, conf=3/3]

Una propiedad importante de los árboles de decisión es que los **caminos** que se generan son **mutuamente excluyentes** (los ejemplos de entrenamiento están cubiertos por un solo camino) y **exhaustivos** (no hay ejemplos sin cubrir).

Al no existir un único árbol para representar todo el conjunto de datos, determinar cuál es el más adecuado (simple y preciso) se trata de un problema NP completo, por lo que se requiere algún tipo de heurística para tratarlo. La estrategia más utilizada es **divide y conquista**, que recursivamente produce particiones del conjunto de datos para construir el árbol: al principio, todos los ejemplos están en la raíz; a medida que el árbol crece, los ejemplos son subdivididos recursivamente.

2.1 Algoritmo

Explicar algoritmo c4.5

El algoritmo es recursivo y recibe como entrada el conjunto de datos D, el conjunto de atributos A y el árbol (al principio vacío) T. Como salida se tiene el árbol de decisión almacenado en T.

```
. Algorithm decisionTree(D, A, T)
1    if D contains only training examples of the same class c_j ∈ C then
2       make T a leaf node labeled with class c_j;
3    elseif A = ∅ then
4       make T a leaf node labeled with c_j, which is the most frequent class in D
5    else  // D contains examples belonging to a mixture of classes. We select a single
6          // attribute to partition D into subsets so that each subset is purer
7       p_0 = impurityEval-1(D);
8       for each attribute A_i ∈ A (={A_1, A_2, ..., A_k}) do
9          p_i = impurityEval-2(A_i, D)
10      endfor
11      Select A_g ∈ {A_1, A_2, ..., A_k} that gives the biggest impurity reduction,
           computed using p_0 − p_i;
12      if p_0 − p_g < threshold then    // A_g does not significantly reduce impurity p_0
13         make T a leaf node labeled with c_j, the most frequent class in D.
14      else                             // A_g is able to reduce impurity p_0
15         Make T a decision node on A_g;
16         Let the possible values of A_g be v_1, v_2, ..., v_m. Partition D into m
              disjoint subsets D_1, D_2, ..., D_m based on the m values of A_g.
17         for each D_j in {D_1, D_2, ..., D_m} do
18            if D_j ≠ ∅ then
19               create a branch (edge) node T_j for v_j as a child node of T;
20               decisionTree(D_j, A−{A_g}, T_j)    // A_g is removed
21            endif
22         endfor
23      endif
24   endif
```

En las líneas 1 a 4 se tratan los 2 casos base que ponen fin a la recursividad: se crea un nodo hoja cuando todos los ejemplos de entrenamiento tienen la misma etiqueta de clase o cuando no hay más atributos por los que clasificar (en este caso el nodo hoja se etiqueta con la clase más frecuente). En caso contrario, se selecciona un atributo para particionar D.

Se calcula el valor de impureza (entropía) del conjunto D y se lo guarda en p0. Luego se calcula el valor de impureza del mismo conjunto respecto a cada atributo Ai en A y se lo guarda en pi. Se selecciona (usando la ganancia de información p0-pi) el nodo que produce la mayor reducción de impureza (Ag), es decir, el que mejor divide los datos para clasificarlos luego.

Sea threshold un valor mínimo de impureza a partir del cual no es significativa la separación, si la reducción de impureza es menor a dicho valor, se crea un nodo hoja con la etiqueta de clase más frecuente. Sino, se crea un nodo decisión, particionando D en subconjuntos Dj de acuerdo al atributo de clasificación Ag. Para cada uno de esos subconjuntos se crea un nodo y se invoca recursivamente el algoritmo pasando como parámetros el subconjunto Dj, los restantes atributos en A (sin Aj) y el nodo creado Tj.

2.2. Funciones de impureza

Ganancia de información y tasa de ganancia de información, comparación de ambos. Explicar para qué sirven la entropía, la ganancia y la tasa de ganancia en los árboles

Dos de las funciones de impureza más utilizadas en árboles de decisión son la Ganancia de Información y la Tasa de Ganancia de Información. Ambas están basadas en la función de entropía extraída de la teoría de la información.

La entropía de un conjunto D es la sumatoria de la probabilidad de cada clase multiplicada por el logaritmo de la probabilidad de esa clase. La misma mide la impureza o el desorden del conjunto de datos, siendo su valor máximo cuando los elementos están equitativamente distribuidos en las clases y su valor mínimo (0) cuando todos los elementos pertenecen a una sola clase.

Ganancia de información. Explicación y ejemplo.

Ganancia de información

La ganancia de información mide la reducción de impureza o desorden. Para obtenerla, primero se calcula la entropía de todo el conjunto de datos D (esto hace impurityEval-1):

$$entropy\,(D) = -\sum_{j=1}^{|C|} P(c_j)\,.\,log_2(P(c_j))$$

Luego se calcula la entropía que resultaría de particionar D según el atributo Ai (esto hace impurityEval-2) en m subconjuntos:

$entropy_{A_i}\,(D) = \sum_{j=1}^{m} \frac{|D_j|}{|D|}\,.\,entropy(D_j)$ donde Dj es el subconjunto con el valor j del atributo Ai

Finalmente se calcula la ganancia de información:

$$gain(D, A_i) = entropy\,(D) - entropy_{A_i}\,(D)$$

Tasa de ganancia de información

En un caso extremo, si tomáramos el id como nodo de clasificación, todos sus subconjuntos serían puros y la ganancia de información sería la mayor pero el árbol no serviría para predecir un nuevo elemento que tenga un id distinto. Para evitar que se seleccionen atributos con una gran cantidad de valores posibles se utiliza la tasa de ganancia de información:

$gainRatio(D, A_i) = \frac{gain(D,A_i)}{-\sum_{j=1}^{m} (\frac{|D_j|}{|D|}.log_2(\frac{|D_j|}{|D|}))}$ donde m es el número de posibles valores de Ai

De esta manera, si Ai tiene muchos valores posibles el denominador (llamado split info) será grande, dificultando que Ai sea seleccionado.

3. Manejo de atributos continuos

Explicar manejo de atributos continuos en árboles de decisión

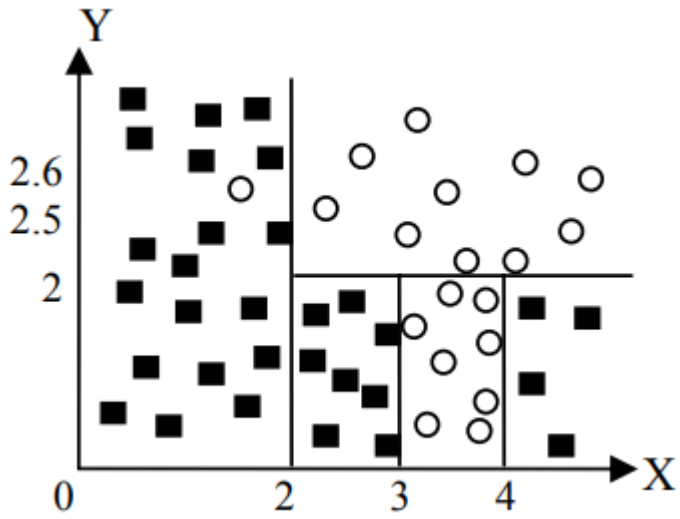
Para datos continuos se divide el rango de valores de un atributo Ai en intervalos; luego, cada intervalo se puede considerar como si fuera un valor discreto.

Se puede dividir Ai en cualquier cantidad de intervalos pero generalmente 2 son suficientes. De esta manera, se busca la división que produzca la mayor ganancia: si bien el atributo es continuo, la cantidad de valores diferentes en Ai es finita (y tomar diferentes valores entre 2 de ellos como punto de división de los intervalos no altera la clasificación), por lo que se puede ordenar dichos valores y mover los límites de clasificación calculando para cada uno la entropía. Además, no se remueve el atributo Ag una vez seleccionado ya que un intervalo se puede volver a dividir, por lo que el mismo atributo continuo puede aparecer varias veces en un camino del árbol.

El manejo de atributos continuos impacta claramente en la eficiencia del algoritmo: en casos discretos la complejidad del algoritmo es lineal (dependiendo directamente de |D|), mientras que para casos continuos es |D|.log|D|

4. Otros problemas

Sobreajuste: el proceso puede dar como resultado árboles muy profundos y muchas hojas de árboles pueden cubrir muy pocos ejemplos de entrenamiento. Si usamos un árbol de este tipo para clasificar conjuntos de prueba, la precisión puede ser muy baja ya que el árbol de decisiones no generaliza bien los datos. Este fenómeno se llama sobreajuste (un clasificador f1 sobreajusta los datos si logra mayor precisión en los datos de entrenamiento que otro clasificador f2, pero menor precisión en los datos de prueba). Para reducir el sobreajuste en el contexto del aprendizaje del árbol de decisiones, realizamos la poda del árbol. Podar el árbol después de su construcción (post-poda) es más eficaz que la pre-poda (detención temprana en la generación del árbol) ya que se tiene una idea más clara de qué ramas no serán útiles. La idea general de la poda posterior es estimar el error de cada nodo del árbol. Si el error estimado para un nodo es menor que el error estimado de su subárbol extendido, entonces el subárbol se poda.
Ej de sobreajuste: para x<2 se realiza una poda obteniendo:



Poda de reglas: señalamos anteriormente que un árbol de decisiones se puede convertir en un conjunto de reglas. De hecho, C4.5 también elimina las reglas para simplificarlas y reducir el sobreajuste. Primero, el árbol se convierte en un conjunto de reglas. Luego la poda de reglas se realiza eliminando algunas condiciones para hacer las reglas más cortas y menos reglas (después de podar algunas reglas pueden volverse redundantes). En la mayoría de los

casos, la poda da como resultado un conjunto de reglas más precisas, ya que es menos probable que las reglas más cortas se sobreajusten a los datos de entrenamiento. La poda también se llama generalización, ya que hace que las reglas sean más generales. Un punto a tener en cuenta es que después de podar el conjunto de reglas resultante puede que ya no sea mutuamente excluyente y exhaustivo: se necesita un orden de las reglas para garantizar que al clasificar un caso de prueba sólo se aplicará una regla y una clase predeterminada, que es generalmente la clase mayoritaria, para cuando un caso de prueba no satisface las condiciones de ninguna regla.

Manejo de valores de atributos faltantes: en muchos conjuntos de datos prácticos, algunos valores de atributos faltan. Hay muchas formas de abordar el problema: completar cada valor faltante con el valor especial "desconocido" o el valor más frecuente del atributo si el atributo es discreto y la media de los valores del atributo si es continuo. El algoritmo del árbol de decisiones en C4.5 adopta otro enfoque: en un nodo de árbol, distribuye el ejemplo de entrenamiento con el valor faltante a cada rama del árbol proporcionalmente de acuerdo con la distribución de los ejemplos de entrenamiento que tienen valores para el atributo.

Manejo de la distribución de clases sesgada: en muchas aplicaciones, las proporciones de datos para diferentes clases pueden ser muy diferentes. Por ejemplo, la proporción de casos de intrusión en redes informáticas es extremadamente pequeña (<1%) en comparación con los casos normales. La aplicación directa del algoritmo del árbol de decisiones para la clasificación o predicción de intrusiones no suele ser eficaz. El árbol de decisión resultante a menudo consta de un solo nodo hoja "normal", que es inútil para la detección de intrusiones. Una forma de abordar el problema es sobremuestrear los ejemplos de intrusión para aumentar su proporción. Otra solución es clasificar los nuevos casos de acuerdo con la probabilidad de que sean intrusiones. Los usuarios humanos pueden entonces investigar los casos mejor clasificados.

¿Qué atributo va en la raíz de un árbol de decisión?

En la raíz va aquel atributo que produzca la mayor ganancia de información, es decir, aquel que separe al conjunto de datos en subconjuntos lo más puros posibles.

¿Cómo se determina el nodo raíz en un árbol de decisión?

- Atributo con mayor cantidad de valores.
- Atributo que aporta más entropía.
- Atributo que aporte menos ganancia.
- Ninguna de las anteriores.

Al finalizar la ejecución del algoritmo C4.5:

- **¿Pueden quedar nodos hoja impuros? Justifique su respuesta**
- **¿El árbol generado puede tener solamente un nodo (el raíz)? Justifique su respuesta**

Sí, pueden quedar nodos hoja impuros. Esto sucede en 2 de los 3 casos que ponen fin a la recursividad del algoritmo:

- Cuando el conjunto de atributos A es vacío en el paso actual. Es decir, ya no hay más atributos con los cuales particionar el conjunto de datos.

- Cuando la reducción de impureza que aporta la partición del conjunto de datos D según los diferentes valores del atributo Ag (atributo seleccionado para efectuar la partición por aportar esta la mayor ganancia de información) es menor a un valor threshold preestablecido. El mismo indica el valor de reducción de impureza a partir del cual no tiene sentido seguir particionando los datos.

En ambos casos se etiqueta al nodo hoja con la clase más frecuente en el conjunto de datos de dicho paso (es decir, de los datos que llegaron a ese nodo).

Sí, el árbol podría tener un solo nodo si todos los datos en el conjunto D inicial pertenecieran a la misma clase, etiquetándose el nodo con esa clase. Este corresponde al tercer caso que pone fin a la recursividad de los mencionados anteriormente y es la primera verificación que efectúa el algoritmo.

Planning

La idea básica de planning consiste en dividir un problema complejo en un subconjunto de problemas que se resuelven independientemente y sus soluciones se combinan para lograr una solución al problema completo.

1. Un agente de planificación simple

Cuando el estado global es accesible, un agente puede utilizar las percepciones proporcionadas por el entorno para construir un modelo completo y correcto del estado global actual. Luego, dado un objetivo, puede llamar a un algoritmo de planificación adecuado (al que llamaremos IDEAL-PLANNER) para generar un plan de acción.

Luego, el agente puede ejecutar los pasos del plan, una acción a la vez. Asumimos la existencia de una función ESTADO-DESCRIPCIÓN, que toma una percepción como entrada y devuelve una descripción del estado inicial en el formato requerido por el planificador. y una función MAKE-GOAL-QUERY, que se utiliza para preguntar a la base de conocimientos cuál debería ser la próxima meta.

El agente interactúa con el entorno de una manera mínima: utiliza sus percepciones para definir el estado inicial y, por lo tanto, la meta inicial, pero a partir de entonces simplemente sigue los pasos del plan que ha construido.

2. De resolución de problemas a planificación

La resolución de problemas y la planificación difieren en la forma de representar estados, acciones y objetivos. El hecho de que se empiece por el estado inicial y se intente avanzar mediante la aplicación de operadores aumenta la dificultad del proceso. Por lo tanto, se necesita de una manera más flexible de estructurar las decisiones, de manera que se trabaje en la parte del problema que tiene más probabilidades de resolverse con la información actual.

Con el objetivo de poder manejar la explosión combinatoria que se genera al inicio de la resolución del problema la **planificación** utiliza **tres conceptos clave**:

1) La apertura de la representación de estados, objetivos y acciones: los estados y objetivos se representan con conjuntos de literales y las acciones mediante descripciones lógicas de las pre y post condiciones. Esto permite conectar los estados y las acciones haciendo coincidir estos literales. Además, no se pierde tiempo probando acciones que no son relevantes en ese momento.
2) Libertad para añadir acciones al plan donde sea necesario y no en una secuencia incremental empezando desde el estado inicial: no necesariamente existe una relación entre el orden de planificación y el orden de ejecución. Cuando se toman decisiones obvias al principio de la construcción del plan, se evita elevar el factor de ramificación correspondiente a elecciones futuras y se evita tener que retroceder en el proceso debido a decisiones arbitrarias.
3) Divide y vencerás: la mayoría de las partes son independientes de la mayoría de las otras partes. Esto es importante a la hora de resolver situaciones complejas, dividiendo la meta en sub-metas y generando sub-planes que pueden ser resueltos de manera más sencilla. Para resolver todo el problema se agrupan todos los planes.

Esto puede hacerse siempre y cuando la interacción entre los planes sea mínima. Falla si el costo de unir los planes es elevado.

3. Planificación en Cálculo de situación

Un problema de planificación es representado en cálculo de situación mediante sentencias lógicas que describen las 3 partes principales de un problema:

- Estado inicial
- Estado objetivo
- Operadores: un conjunto de descripciones de acciones.

Para que la planificación sea práctica, debemos hacer dos cosas:

- Restringir el lenguaje con el que definimos los problemas. Con un lenguaje restrictivo, hay menos soluciones posibles para buscar.
- Utilizar un algoritmo de propósito especial llamado planificador en lugar de un demostrador de teoremas de propósito general para buscar una solución.

4. Representación básica para planificación

El lenguaje utilizado actualmente para representar estados y acciones es STRIPS. Este lenguaje conserva la expresividad de las representaciones de estados y acciones, además de ser adecuado para la utilización de la mayoría de los algoritmos de planificación. Este tipo de representación permite seleccionar las sub-metas que van a ser resueltas en un momento dado.

Los **estados** se representan como conjunciones de literales en los que no hay funciones. La descripción de un estado no necesariamente tiene que estar completa. Por convención, si un literal está ausente en la representación del estado se lo considera como presente pero su valor dependerá del contexto (por lo general es la negación). Los **objetivos** también se representan mediante conjunción de literales (pueden contener variables).

En STRIPS los **operadores** tienen tres elementos:

1. La especificación de la acción. ACCION: Ir(y)
2. La condición previa: conjunción de literales que indica lo que debe ser verdadero antes de poder aplicar el operador. PRECOND: En(x) ^ Ruta(x, y)
3. El efecto: conjunción de literales que indica el cambio efectuado después de aplicar el operador. EFECTO: En(y) y -En(x)

Un **operador con variables** se conoce como **esquema de operador**, porque no corresponde a una sola acción ejecutable sino a una familia de acciones, una para cada instanciación diferente de las variables. Por lo general, solo se pueden ejecutar operadores completamente instanciados.

Un operador o es **aplicable** a un estado s si se pueden concretizar las variables que contiene o de manera que todas las precondiciones de o sean válidas en s.
Si existen los literales: En(casa), Ruta(casa, supermercado)
Se puede aplicar: Ir(supermercado)
Resultado: En(supermercado), -En(casa), Ruta(casa, supermercado)

Los métodos de búsqueda se pueden considerar como planificadores: sería un planificador de **espacio de estados** porque busca en el espacio de todas las situaciones posibles y un **planificador de progresión** porque busca hacia adelante desde la situación inicial hasta la situación objetivo. El principal problema con este enfoque es el alto factor de ramificación y, por lo tanto, el enorme tamaño del espacio de búsqueda. Una manera de reducir el factor de ramificación es partiendo del objetivo y buscar hacia atrás intentando satisfacer las condiciones del estado inicial, lo que se conoce como **planificador de regresión**. Esto presenta problemas cuando se debe alcanzar un conjunto de objetivos y no solo uno.

Una alternativa es buscar a través del **espacio de planes** en lugar del espacio de estados. Se inicia con un **plan parcial** (plan incompleto que no resuelve el problema) que puede completarse mediante operadores de refinamiento y modificación. Un **operador de refinamiento** toma un plan parcial y le añade restricciones. Algunos planificadores crean planes alternativos potencialmente incorrectos que se depuran usando operadores de modificación para obtener el plan definitivo. Una buena representación de planes asegura poder hacer una búsqueda eficiente en el espacio de dichos planes.

Explicar compromiso mínimo. Dar un ejemplo.

Muchos planificadores usan el principio de **compromiso mínimo**, el cual consiste en comprometerse a resolver algo y solo tomar decisiones respecto a ello, dejando las otras decisiones para resolverlas después. Esto tiene como propósito evitar tomar malas decisiones que luego implicarán una vuelta atrás, pudiendo dejar sin especificar el orden de ciertos pasos. Ej: para hacer una torta un planificador debe tomar la decisión de comprar harina y la de comprar huevos, pero en ese punto no requiere tomar decisiones sobre en qué orden comprarlos.

Diferencia entre planificador de orden parcial POP y de orden total POT

Un planificador que puede representar planes en los que algunos pasos están ordenados entre sí y otros pasos están desordenados se denomina **planificador de orden parcial**. La alternativa es un **planificador de orden total**, en el que los planes consisten en una simple lista de pasos. Un plan totalmente ordenado que se deriva de un plan P mediante la adición de restricciones de orden se denomina **linealización de P**. A los planes que tienen todas las variables enlazadas a una constante se los conoce como planes totalmente concretizados.

Un plan se define como una estructura de datos con 4 componentes:

- Un conjunto de pasos (cada paso es un operador)
- Un conjunto de restricciones de ordenamiento Si $\prec$ Sj
- Un conjunto de restricciones de enlace de variables
- Un conjunto de vínculos causales Si --(c)→ Sj (Si logra c para Sj). Los vínculos causales expresan el propósito de los pasos en el plan, especifican que el paso Si satisface la precondición c del paso Sj.

Una **solución** es un plan completo y consistente.

Un **plan completo** es aquél en donde cada precondición de cada paso se logra con la ejecución de algún otro paso.

Si logra una precondición c de Sj si:
1) Si ≺ Sj y c ∈ Efectos(Sj)
2) ∄ Sk / -c ∈ Efectos(Sk) y Si ≺ Sk ≺ Sj
para alguna linealización del plan

Un **plan consistente** es aquel en el que no hay contradicciones en las restricciones de ordenamiento o enlace.
Existe una contradicción si Si ≺ Sj y Sj ≺ Si, o bien, v=A y v=B (para la misma variable se instancian dos constantes diferentes A y B). Estas propiedades son transitivas, por lo que también existe una contradicción si S1 ≺ S2, S2 ≺ S3 y S3 ≺ S1.

Planning: ¿Cuándo se produce una amenaza? ¿Cómo se resuelve?

Una amenaza consiste en un paso que puede eliminar la condición protegida de un vínculo causal. Es decir, se produce una amenaza cuando la postcondición de un paso elimina la precondición ya lograda de otro paso con el que no hay restricciones de orden ni vínculos causales; es decir, no hay un orden de ejecución definido entre ellos. Las mismas se resuelven añadiendo restricciones de orden para que ocurran antes (degradación) o después (promoción) del vínculo protegido.

5. Algoritmo de planificación de orden parcial (POP)

POP es un algoritmo no determinístico que usa elegir y fallar en lugar de bucles explícitos. Se debe notar que la selección del paso y la condición a satisfacer en SELECCIONAR-SUBMETA no es una razón para el backtracking, ya que eventualmente todas las precondiciones deben ser consideradas y el orden en que se manejan estas no afecta al conjunto de planes posibles (ni a la posibilidad de encontrar una solución).

Algoritmo POP. Explicación.

POP es un planificador de regresión que recibe como entradas el estado inicial, el objetivo y los operadores y devuelve el plan a ejecutar. A partir del estado inicial y los objetivos arma el plan parcial mínimo y entra en un bucle hasta que el plan sea solución, extendiendo el plan en cada paso:

1. Se invoca SELECCIONAR-SUBMETA: recibe como entrada el plan y selecciona algún paso existente Sneed que tenga una precondición c sin cumplir, devolviendo Sneed y c.
2. Se invoca SELECCIONAR-OPERADOR: recibe como entradas el plan, los operadores, Sneed y c, y selecciona un paso Sadd (nuevo o existente) que logra la precondición c. Si no se encuentra ningún operador, falla. Sino, guarda el vínculo causal Sadd→Sneed y su restricción de orden Sadd≺Sneed. Si Sadd es un nuevo paso (obtenido de los operadores), se lo añade a la lista de pasos existentes y se guarda la restricción de orden Iniciar≺Sadd≺Terminar.
3. Se invoca RESOLVER-AMENAZAS: recibe como entrada el plan y resuelve cualquier amenaza Sthreat hacia los vínculos causales existentes (Si--c→Sj) promoviéndolas (añadiendo la restricción de orden Sj≺Sthreat) o degradándolas (Sthreat≺Si). Si no se puede resolver alguna amenaza (plan inconsistente), el algoritmo falla.

```
function POP(initial, goal, operators) returns plan

  plan ← MAKE-MINIMAL-PLAN(initial, goal)
  loop do
      if SOLUTION?(plan) then return plan
      S_need, c — SELECT-SUBGOAL(plan)
      CHOOSE-OPERATOR(plan, operators, S_need, c)
      RESOLVE-THREATS(plan)
  end
```

```
function SELECT-SUBGOAL(plan) returns S_need, c

  pick a plan step S_need from STEPS(plan)
      with a precondition c that has not been achieved
  return S_need, c
```

```
procedure CHOOSE-OPERATOR(plan, operators, S_need, c)

  choose a step S_add from operators or STEPS(plan) that has c as an effect
  if there is no such step then fail
  add the causal link Sadd --c--> S_need to LINKS(plan)
  add the ordering constraint S_add ≺ S_need to ORDERINGS(plan)
  if Sadd is a newly added step from operators then
      add S_add to STEPS(plan)
      add Start ≺ S_add ≺ Finish to ORDERINGS(plan)
```

```
procedure RESOLVE-THREATS(plan)

  for each S_threat that threatens a link S_i --c--> S_j in LINKS(plan) do
      choose either
          Promotion: Add S_threat ≺ S; to ORDERINGS(plan)
          Demotion: Add S_j ≺ S_threat to ORDERINGS(plan)
      if not CONSISTENT(plan) then fail
  end
```

6. Planificación con operadores parcialmente instanciados

El algoritmo visto no maneja el enlace de las variables. Con esto en mente, en RESOLVE-THREATS, ¿un operador que tiene el efecto At(x) se considera una amenaza para la condición -At(Home)? Actualmente podemos distinguir entre amenazas y no amenazas, pero esta es una posible amenaza. Hay tres enfoques principales para hacer frente a posibles amenazas:

- **Resolver ahora con una restricción de igualdad**: modificar RESOLVE-THREATS para que resuelva todas las amenazas posibles tan pronto como sean reconocidas. Por ejemplo, cuando el planificador elige el operador que tiene el efecto At(x), agregaría una vinculación como x = HWS para asegurarse de que no amenaza -At(Home).
- **Resolver ahora con una restricción de desigualdad**: ampliar el lenguaje de las restricciones de vinculación de variables para permitir la restricción x≠Home. Esto tiene la ventaja de ser un compromiso más bajo (no requiere una elección arbitraria para el valor de x) pero es un poco más complicado de implementar, porque las rutinas de unificación que hemos usado hasta ahora tratan con igualdades, no con desigualdades.
- **Resolver más tarde**: la tercera posibilidad es ignorar las posibles amenazas y solo tratarlas cuando se conviertan en amenazas necesarias. Es decir, RESOLVE-THREATS no consideraría At(x), como una amenaza para -At(Home). Pero si la restricción x=Home se añadiera alguna vez al plan, la amenaza se resolvería (mediante promoción o degradación). Este enfoque tiene la ventaja de ser de bajo compromiso, pero tiene la desventaja de que es más difícil decidir si un plan es una solución.

Cuando aparecen operadores parcialmente instanciados en los planes, el criterio para las soluciones debe refinarse un poco. Una solución se definió como un plan parcial de modo que todas las linealizaciones estén garantizadas para lograr el objetivo. Con operadores parcialmente instanciados, también necesitamos asegurarnos de que todas las instanciaciones logren el objetivo. Por lo tanto, ampliamos la definición de logro para un paso en un plan de la siguiente manera:

Un paso Si logra una precondición c del paso Sj si (1) Si $\prec$ Sj y Si tiene un efecto que necesariamente se unifica con c; y (2) no hay ningún paso Sk tal que Si $\prec$ Sk $\prec$ Sj en alguna linealización del plan y Sk tenga un efecto que posiblemente se unifica con -c.

CHOOSE-OPERATOR crea el Si que logra (1), y RESOLVE-THREATS se asegura de que (2) se satisfaga promoviendo o degradando posibles amenazas.

```
procedure CHOOSE-OPERATOR(plan, operators, S_need, c)

    choose a step S_add from operators or STEPS(plan) that has c_add as an effect
            such that u = UNIFY(C, c_add, BINDINGS(plan))
    if there is no such step
        then fail
    add u to BINDINGS(plan)
    add S_add --c--> S_need to LINKS(plan)
    add S_add ≺ S_need to ORDERINGS(plan)
    if S_add is a newly added step from operators then
        add S_add to STEPS(plan)
        add Start ≺ S_add ≺ Finish to ORDERINGS(plan)
```

```
procedure RESOLVE-THREATS(plan)

    for each S_i --c--> S_j in LINKS(plan) do
        for each S_threat in STEPS(plan) do
            for each c' in EFFECT(S_threat) do
                if SUBST(BINDINGS(plan), c) = SUBST(BINDINGS(plan), ¬c') then
                    choose either
                        Promotion: Add S_threat ≺ S_i to ORDERINGS(plan)
                        Demotion: Add S_j ≺ S_threat to ORDERINGS(plan)
                if not CONSISTENT(plan)
                    then fail
            end
        end
    end
```

Luego de la ejecución del algoritmo POP se obtiene:

- Secuencias de pasos para resolver un problema determinado. (Correcta)

¿Qué es y cómo se implementa la descomposición jerárquica en planning?

La descomposición jerárquica consiste en dividir un problema complejo en subconjuntos de problemas hasta que estos pueden ser resueltos por el agente en cuestión. Se descomponen jerárquicamente los operadores, construyendo subplanes para cada operador. Cada uno de estos subplanes se resuelve y sus soluciones se combinan para lograr una solución al problema completo, teniendo en cuenta que los efectos de un subplan sean precondiciones de otros.

Unidad 3: Búsqueda en grandes espacios de soluciones

Problemas, espacios de problema y búsqueda

Para construir un sistema que resuelva un problema específico, es necesario realizar estas cuatro acciones:

1. Definir el problema con precisión: la definición debe incluir especificaciones precisas tanto sobre la o las situaciones iniciales corno sobre las situaciones finales que se aceptarían como soluciones al problema.
2. Analizar el problema: algunas características de gran importancia pueden tener un gran efecto sobre la conveniencia o no de utilizar las diversas técnicas que resuelven el problema.
3. Aislar y representar el conocimiento necesario para resolver el problema.
4. Elegir las mejores técnicas que resuelvan el problema y aplicarlas al problema particular.

1. Definición del problema mediante una búsqueda en un espacio de estados

La representación corno espacio de estados forma la base de la mayoría de los métodos de IA que se discuten aquí. Su estructura se corresponde con la estructura de la resolución de problemas por dos importantes razones:

1. Permite definir formalmente el problema, mediante la necesidad de convertir alguna situación dada en la situación deseada usando un conjunto de operaciones permitidas.
2. Permite definir el proceso de resolución de un problema como una combinación de técnicas conocidas y búsqueda, la técnica general de exploración en el espacio intenta encontrar alguna ruta desde el estado actual hasta un estado objetivo. La búsqueda es un proceso de gran importancia en la resolución de problemas difíciles para los que no se dispone de técnicas más directas.

Para poder producir una descripción formal de un problema debe hacerse lo siguiente:

1. Definir un espacio de estados que contenga todas las configuraciones posibles de los objetos más relevantes. Es, por supuesto, posible definir este espacio sin tener que hacer una enumeración de todos y cada uno de los estados que contiene.
2. Identificar uno o más estados que describan situaciones en las que comience el proceso de resolución del problema. Éstos se denominan estados iniciales.
3. Especificar uno o más estados que pudieran ser soluciones aceptables del problema. Estos estados se denominan estados objetivo.
4. Especificar un conjunto de reglas que describan las acciones (operadores) disponibles. Para realizarlo se debe meditar acerca de los siguientes temas:
    - ¿Qué suposiciones presentes en la descripción informal del problema no están expresadas como tales?
    - ¿Qué generalidad deben tener las reglas?
    - ¿Qué cantidad del trabajo necesario para resolver el problema debe ser preprocesado y representado en forma de reglas?

De esta forma, el problema puede resolverse con el uso de las reglas en combinación con una estrategia apropiada de control para trasladarse a través del espacio del problema hasta encontrar una ruta desde un estado inicial hasta un estado objetivo. La búsqueda es, sin embargo, un mecanismo general que puede utilizarse cuando no se conoce otro método más directo.

2. Los sistemas de producción

Debido a que la búsqueda es el núcleo de muchos procesos Inteligentes, es adecuado estructurar los programas de IA de forma que se facilite describir y desarrollar el proceso de búsqueda. Los sistemas de producción proporcionan tales estructuras. Un **sistema de producción** consiste en un conjunto de reglas compuestas por una parte izquierda que determina la aplicabilidad de la regla, y una parte derecha que describe la operación que se lleva a cabo si se aplica la regla.

2.1. Estrategias de control

Hasta ahora se ha ignorado por completo la cuestión de cómo se decide **qué regla hay que aplicar** durante el proceso de búsqueda de la solución de un problema (ya que con frecuencia es posible aplicar más de una regia -y algunas veces ninguna- cuando sus partes izquierdas coinciden con el estado actual). Esto tiene un impacto crucial en la rapidez, y tal vez en la posibilidad, de resolución del problema. Una buena estrategia de control debe cumplir dos requisitos:

1. **Causar algún cambio**: las estrategias de control que no causan cambio de estado nunca alcanzan la solución.
2. **Ser sistemática**: este requisito se corresponde con una necesidad de cambio global (en el curso de varios pasos) tanto como de cambio local (en el curso de un paso sencillo).

Algoritmos de búsqueda sistemática:

1. Algoritmo de Búsqueda primero en amplitud:

1. Crear una variable llamada LISTA-NODOS y asignarle el estado inicial.
2. Hasta que se encuentre un estado objetivo o LISTA-NODOS esté vacía, hacer:
    a. Si LISTA-NODOS está vacía, terminar. Sino, eliminar el primer elemento de LISTA-NODOS y llamarlo E.
    b. Para cada regla que se empareje con el estado descrito en E hacer:
        i. Aplicar la regla para generar un nuevo estado.
        ii. Si el nuevo estado es un estado objetivo, terminar y devolver este estado.
        iii. En caso contrario, añadir el nuevo estado al final de LISTA-NODOS

2. Algoritmo de Búsqueda primero en profundidad:

1. Si el estado inicial es un estado objetivo, terminar y devolver un éxito.
2. En caso contrario, hacer lo siguiente hasta que se marque un éxito o un fracaso.
    a. Generar un sucesor, E, del estado inicial. Si no existen más sucesores, marcar un fracaso.
    b. Llamar a la Búsqueda primero en profundidad con E como estado inicial.

c. Si se devuelve un éxito, marcar un éxito. En caso contrario, continuar con el ciclo.

Ambos algoritmos son **exhaustivos**: exploran todo el espacio hasta llegar a la solución, sin hacer una poda de algún espacio que se considere que no es conveniente.

Ventajas de la Búsqueda primero en amplitud:

- La búsqueda primero en amplitud no queda atrapada explorando callejones sin salida. Esto se contrapone con la búsqueda primero en profundidad en la que se puede seguir una ruta infructuosa durante mucho tiempo, y quizás para siempre, antes de acabar en un estado sin sucesores. Esto es particularmente un problema en la búsqueda primero en profundidad si hay ciclos, a no ser que se tenga un cuidado especial en verificar tales situaciones.
- Si existe una solución, la búsqueda primero en amplitud garantiza que se logre encontrarla. Además, si existen múltiples soluciones, se encuentra la solución mínima. En cambio, en la búsqueda primero en profundidad es posible encontrar una solución larga en alguna parte del árbol, cuando puede existir otra mucho más corta en alguna parte inexplorada del mismo.

Ventajas de la Búsqueda primero en profundidad:

- Necesita menos memoria ya que solo se almacenan los nodos del camino que se sigue en ese instante. Esto contrasta con la búsqueda primero en amplitud, en la que debe almacenarse todo el árbol que haya sido generado hasta ese momento.
- Si se tiene suerte (o si se tiene cuidado en ordenar los estados alternativos sucesores), la búsqueda primero en profundidad puede encontrar una solución sin tener que examinar gran parte del espacio de estados. En el caso de la búsqueda primero en amplitud deben examinarse todas las partes del árbol de nivel n antes de comenzar con los nodos de nivel n + 1. Esto es particularmente relevante en caso de que existan varias soluciones aceptables (la búsqueda primero en profundidad acaba al encontrar una de ellas).

En un problema con alto factor de ramificación, una estrategia consistente en simplemente explorar todas las opciones colapsa rápidamente. Este fenómeno se denomina explosión combinatoria y, para combatirla, es necesaria una nueva estrategia de control.

¿Cuándo termina Primero en Amplitud? ¿Y primero en profundidad? Si hay solo una solución. ¿Cual algoritmo la encuentra primero: primero en profundidad o primero en amplitud?

En ambos casos, si existe una solución, finalizan cuando encuentran esta. Si no existe solución, finalizan cuando recorren todo el espacio de estados. Si hay una solución, ambos algoritmos la encuentran. Quien la encuentre primero dependerá de donde esté ubicada esta solución. Si existe más de una solución, primero en amplitud encuentra la óptima mientras que primero en profundidad no garantiza esto.

Condiciones a tener en cuenta para aplicar la búsqueda primero en profundidad

- No deben existir ciclos en el problema o se debe establecer un mecanismo para evitarlos ya que sino el algoritmo puede quedar atrapado en un bucle.

- Se desea encontrar una solución que no necesariamente debe ser mínima. Se puede encontrar una solución mínima pero este algoritmo no lo garantiza.
- No deben existir ramas infinitas, de manera que el algoritmo se quede atrapado en ellas.

2.2. Búsqueda heurística

Con el fin de resolver problemas complicados con eficiencia, con frecuencia es necesario comprometer los requisitos de movilidad y sistematicidad, y construir una estructura de control que no garantice encontrar la mejor respuesta pero que casi siempre encuentre una buena solución. De esta forma, surge la idea de heurística. Una **heurística** es una técnica que aumenta la eficiencia de un proceso de búsqueda, posiblemente sacrificando demandas de completitud. Algunas heurísticas ayudan a guiar el proceso de búsqueda sin sacrificar ninguna demanda de completitud que el proceso haya podido tener previamente. Otras (muchas de las mejores) pueden ocasionalmente causar que una buena ruta sea pasada por alto. Pero, en promedio, mejoran la calidad de las rutas que exploran, Al usar buenas heurísticas se pueden esperar buenas (aunque posiblemente no óptimas) soluciones para problemas difíciles en un tiempo menor al exponencial.

La heurística del vecino más próximo es un ejemplo de una buena heurística de propósito general válida para varios problemas combinatorios. Consiste en seleccionar en cada paso la alternativa localmente superior.

Motivos por los cuales usar heurísticas:
- Sin las heurísticas, se estaría desesperadamente enredado en una explosión combinatoria.
- Normalmente no se necesita una solución óptima, con frecuencia una buena aproximación es adecuada.
- Si bien las aproximaciones que se logran con una heurística pueden no ser muy buenas en los peores casos, estos peores casos raramente ocurren en el mundo real.
- Intentar comprender por qué funciona una heurística o por qué no lo hace normalmente sirve para profundizar en la comprensión del problema.

Existen dos formas fundamentales de **incorporar conocimiento** heurístico específico del dominio, a un proceso de búsqueda basado en reglas:
1. En las mismas reglas. Por ejemplo, las reglas de un sistema para jugar al ajedrez podrían no sólo describir el conjunto de movimientos legales, sino incluso un conjunto de movimientos "sensatos" determinados por el programador de las regias.
2. Como una función heurística que evalúa los estados individuales del problema y determina su grado de "deseabilidad".

Una **función heurística** es una correspondencia entre las descripciones de estados del problema hacia alguna medida de deseabilidad, normalmente representada por números. El programa que utilice los valores de la función tendrá que minimizar o maximizar según sea adecuado. En general, existe un compromiso entre el coste de evaluación de una función heurística y el ahorro de tiempo de búsqueda que proporciona la función: cuanto mejor es la heurística, más cuesta calcularla.

2.3. Características del problema

La búsqueda heurística incluye gran variedad de técnicas específicas, cada una de las cuales es particularmente efectiva para una pequeña clase de problemas. A fin de poder elegir el método más apropiado (o una combinación de métodos) para un problema en particular, es necesario analizar diversas cuestiones:

2.3.1. ¿Puede descomponerse el problema?
Existen problemas donde los subproblemas no son independientes, sino que interactúan y tales interacciones deben ser consideradas a fin de llegar a una solución para la totalidad del problema (por ej. el mundo de bloques). En otros donde sí se puede (por ej. una operación matemática), se hace lo siguiente: en cada paso se verifica si el problema en el que se trabaja se puede resolver directamente. Si se puede, se devuelve inmediatamente la respuesta. Sino, se verifica si se puede descomponer el problema en otros más simples. Si se puede, se crean estos problemas y se llama recursivamente al algoritmo con ellos.

2.3.2. ¿Pueden deshacerse o Ignorarse pasos hacia una solución?
En este sentido existen 3 clases de problemas:

- Ignorables (por ej., la demostración de teoremas), en los que pueden ignorarse pasos dados
- Recuperables (por ej., el 8-puzzle), en los que pueden deshacerse pasos dados.
- No recuperables (por ej., el ajedrez), en los que no pueden deshacerse pasos dados.

La recuperabilidad de un problema juega un papel importante en la determinación de la complejidad de la estructura de control necesaria para resolver el problema.

2.3.3. ¿Es predecible el universo?

- Para resolver los problemas de consecuencia cierta (aquellos donde cada vez que se hace un movimiento, se sabe exactamente qué ocurrirá), se puede utilizar la planificación para generar una secuencia de operadores que garantizan llegar a una solución.
- Para los problemas de consecuencia incierta, lo mejor que se puede hacer es investigar distintos planes y utilizar las probabilidades de las consecuencias que se derivan de su elección para resaltar el que tenga la más alta probabilidad estimada de llegar a una solución. Para resolver estos problemas, es necesario tener en cuenta que un proceso de revisión de planes se lleve a cabo cuando el plan se realice y proporcione la necesaria realimentación.

Las dos últimas características interactúan de una forma interesante: una forma de resolver los problemas irrecuperables es planificando una solución completa antes de embarcarse en la implementación del plan. Sin embargo, este proceso de planificación sólo es útil en los problemas de consecuencia-cierta. Así, uno de los tipos de problemas más difíciles de resolver son los irrecuperables de consecuencia-incierta.

2.3.4. Una solución adecuada ¿es absoluta o relativa?
Los problemas de “el mejor camino” son, por lo general, más complicados de computar que los problemas de “algún camino”. Los problemas de algún camino se resuelven frecuentemente en un tiempo razonable mediante heurísticas que sugieran rutas adecuadas para explorar. Para ciertos problemas del mejor camino, sin embargo, no pueden usarse

heurísticas que puedan omitir la mejor solución, por lo que se requiere una búsqueda mucho más exhaustiva.

2.3.5. ¿La solución es un estado o una ruta?
La respuesta a esta cuestión nos indica si va a ser necesario almacenar el camino del proceso de resolución del problema o si basta con almacenar el resultado.

2.3.6. ¿Cuál es el papel del conocimiento?
Existen problemas en los que mucho conocimiento sólo es necesario para acotar la búsqueda (por ej., el ajedrez), y aquéllos en los que el conocimiento es estrictamente necesario para llegar a una solución (por ej., la comprensión de lenguaje natural).

2.3.7. ¿Necesita la tarea interaccionar con una persona?
Se puede hacer una distinción entre dos tipos de problemas:

- Los solitarios, en los que a la computadora se le da una descripción del problema y esta proporciona una respuesta sin ningún tipo de comunicación ni necesidad de explicaciones del proceso de razonamiento.
- Los conversacionales, en los que existe una comunicación intermedia entre el hombre y la computadora, bien para proporcionar una ayuda adicional a la máquina o para que la computadora proporcione información al usuario.

Esta decisión tiene gran importancia a la hora de elegir el método de resolución del problema.

2.4. Características de los sistemas de producción

- Sistemas de producción monótonos: la aplicación de una regla nunca prevé la posterior aplicación de otra regla que podría haberse aplicado cuando se seleccionó la primera. En los sistemas de producción no monótonos, esto no se cumple.
- Sistemas de producción parcialmente conmutativos: si una determinada aplicación de una secuencia de reglas transforma el estado x en el estado y, entonces alguna permutación permitida de estas reglas, también transforma el estado x en el estado y

Un sistema de producción conmutativo es aquel que es a la vez monótono y parcialmente conmutativo.

Desde un punto de vista formal, no existe relación entre tipos de problemas y tipos de sistemas de producción debido a que un problema que puede ser resuelto con un sistema de producción, puede ser resuelto con uno conmutativo (la clase más restrictiva) y, por lo tanto, con cualquier sistema de producción. Sin embargo, el sistema conmutativo puede ser tan inmanejable que sea prácticamente inútil, por lo que, desde un punto de vista práctico, definitivamente existe relación entre tipos de problemas y los tipos de sistemas de producción que se prestan de forma natural a representar estos problemas.

Los sistemas parcialmente conmutativos y monótonos son adecuados para resolver problemas ignorables, donde no se vuelve hacia atrás cuando se descubre que se ha seguido un camino incorrecto. Si bien con frecuencia es adecuado implementar estos sistemas con vuelta atrás para garantizar una búsqueda sistemática, la base de datos actual que representa el problema no necesita regenerarse.

Los sistemas parcialmente conmutativos y no monótonos se adecuan a aquellos problemas en los que se realizan cambios pero éstos son reversibles, y en los que el orden de las

operaciones no es crítico. Éste es normalmente el caso de los problemas de manipulación física como el mundo de bloques. Estos sistemas tienden a alcanzar muchos estados individuales duplicados durante el proceso de búsqueda.

Los sistemas de producción que no son parcialmente conmutativos son adecuados para problemas en los que se producen cambios irreversibles, donde es importante tomar las decisiones correctas la primera vez. Por ejemplo, un proceso que produzca un compuesto químico.

2.5. Algunos aspectos sobre el diseño de programas de búsqueda

Todos los procesos de búsqueda pueden tratarse como un recorrido sobre una estructura en forma de árbol en el que cada nodo representa un estado del problema y cada enlace representa una relación entre dichos estados.

El proceso de búsqueda en árboles hace que si aparece varias veces el mismo nodo se repita cada vez su estructura de sucesores. Esto ocurre porque el espacio de búsqueda puede representarse mejor mediante un grato que mediante un árbol. Sin embargo, en los grafos aparece el problema de los ciclos. Al tratar los procesos de búsqueda como grafos de búsqueda en lugar de árboles de búsqueda, se disminuye el esfuerzo empleado en explorar los mismos caminos varias veces. Sin embargo, es necesario un esfuerzo adicional cada vez que se genera un nodo para ver si se había generado con anterioridad.

Los procedimientos de búsqueda sobre grafos son especialmente indicados para sistemas de producción parcialmente conmutativos en los que dado un conjunto de operaciones se producirá el mismo resultado cambiando el orden de aparición de las operaciones.

Algoritmo: Control de los nodos duplicados

1. Examinar el conjunto de nodos creados hasta el momento para ver si el nuevo nodo ya existe.
2. Si no existe, simplemente añadirlo al grafo como si fuera un árbol.
3. Si ya existe, hacer lo siguiente:
    a. Colocar el nodo que ha surgido en el punto que se corresponde con su sucesor en lugar de con el del nodo nuevo. El nodo nuevo simplemente se desecha.
    b. Si se está manteniendo constancia del mejor camino para cada nodo, verificar si el nuevo camino es mejor o peor que el antiguo. Si es peor, no hacer nada. Si es mejor, almacenar el camino nuevo corno la ruta correcta a utilizar para el nodo y propagar el correspondiente cambio en el coste a lo largo de los nodos sucesores cuando sea necesario.

Dado un árbol de profundidad dT y factor de ramificación bT, suponga que la única solución está en el nodo de más a la derecha en el nivel dT. ¿Qué algoritmo es peor en tiempo para encontrar la solución? (suponga que ambos expanden los nodos primero por la izquierda). Justifique su respuesta.

No hay diferencias: en esas condiciones, tanto el algoritmo de búsqueda primero en profundidad como el de búsqueda primero en amplitud encuentran la solución en el último nodo luego de explorar todo el árbol.

Técnicas de búsqueda heurística

1. Generación y prueba

Algoritmo: Generación y prueba

1. Generar una posible solución. Para algunos problemas, esto significa generar un objetivo particular en el espacio del problema. Para otros, supone más bien generar un camino a partir de un estado inicial.
2. Verificar si realmente el objetivo elegido es una solución comparándolo con el objetivo final o comparando el camino elegido con el conjunto de estados objetivo aceptables.
3. Si se ha encontrado la solución. terminar. Sino, volver al paso 1.

El método de generación y prueba puede funcionar de forma que genere las soluciones de forma aleatoria, pero esto no garantiza que se pueda encontrar alguna vez la solución.

Si se generan las posibles soluciones de forma sistemática, es un procedimiento de búsqueda primero en profundidad ya que las soluciones completas deben generarse antes de que se comprueben.

Entre estos dos extremos existe un punto medio en donde el proceso de búsqueda actúa de forma sistemática, pero, a partir de algún indicio, se descartan caminos que dan la impresión de que por ellos no se llega a la solución.

2. Escalada

El método de la escalada es una variante del de generación y prueba en el que existe realimentación a partir del procedimiento de prueba que se usa para ayudar al generador a decidirse por cuál dirección debe moverse en el espacio de búsqueda, a partir de una función heurística que proporcione una estimación de lo cercano que se encuentra un estado al estado objetivo.

Se utiliza frecuentemente cuando se dispone de una buena función heurística para evaluar los estados, pero cuando no se dispone de otro tipo de conocimiento provechoso.

2.1 Escalada simple

Algoritmo: Escalada simple

1. Evaluar el estado inicial. Si también es el estado objetivo, devolverlo y terminar. En caso contrario, continuar con el estado inicial corno estado actual.
2. Repetir hasta que se encuentre una solución o hasta que no queden nuevos operadores que aplicar al estado actual:
    a. Seleccionar un operador que no haya sido aplicado con anterioridad al estado actual y aplicarlo para generar un nuevo estado.
    b. Evaluar el nuevo estado.
        i. Si es un estado objetivo, devolverlo y terminar.
        ii. Si no es un estado objetivo, pero es mejor que el estado actual, convertirlo en el estado actual.
        iii. Si no es mejor que el estado actual, continuar con el bucle.

La principal diferencia que existe entre este algoritmo y el de generación y prueba, consiste en el uso de una función de evaluación como una forma de introducir conocimiento específico de la tarea realizada en el proceso de control. La utilización de este conocimiento es lo que hace a este y a otros métodos de este capítulo, métodos de búsqueda heurística. Para que el algoritmo pueda funcionar, es necesario proporcionar una definición precisa del término mejor.

2.2 Escalada por la máxima pendiente

Una variación útil del método de escalada simple consiste en considerar todos los posibles movimientos a partir del estado actual y elegir el mejor de ellos como nuevo estado.

Algoritmo: Escalada por la máxima pendiente

1. Evaluar el estado inicial. Si también es el estado objetivo, devolverlo y terminar. En caso contrario, continuar con el estado inicial como estado actual y SUCC (siendo SUCC el mejor estado hasta el momento).
2. Repetir hasta que se encuentre una solución o hasta que una iteración completa no produzca un cambio en el estado actual:
    a. Para cada operador posible del estado actual hacer lo siguiente:
        i. Aplicar el operador y generar un nuevo estado.
        ii. Evaluar el nuevo estado. Si es un estado objetivo, devolverlo y terminar. Si no, compararlo con SUCC. Si es mejor, asignar a SUCC este nuevo estado. Si no es mejor, dejar SUCC como está.
    b. Si SUCC es mejor que el estado actual, hacer que el estado actual sea SUCC.

Existe un compromiso entre el tiempo necesario para seleccionar un movimiento (más largo para una escalada por la máxima pendiente) y el número de movimientos necesarios para alcanzar una solución (mayor para una escalada simple), que debe ser considerado al decidir qué método sería mejor para un problema en particular.

2.3 Problemas

Tanto la escalada simple como la de máxima pendiente pueden no encontrar un estado objetivo, y en cambio encontrar un estado del que no sea posible generar nuevos estado mejores que él. Esto ocurre si el programa se topa con:

- **Máximo local**: es un estado que es mejor que todos sus vecinos, pero que no es mejor que otros estados de otros lugares. En un máximo local, todos los movimientos producen estados peores. Son particularmente frustrantes porque frecuentemente aparecen en las cercanías de una solución. En este caso se denominan estribaciones.
- **Meseta**: es un área plana del espacio de búsqueda en la que un conjunto de estados vecinos posee el mismo valor por lo que no es posible determinar la mejor dirección a la que moverse haciendo comparaciones locales.
- **Cresta**: es un tipo especial de máximo local, en el cual por el camino actual se llega a un punto más alto, pero los sucesores inmediatos producen una solución peor que hacen que sea inalcanzable.

Métodos para tratar de superar los anteriores problemas:

- Volver hacia atrás e intentar seguir un camino diferente. Este método es especialmente adecuado para superar máximos locales.

- Realizar un gran salto en alguna dirección para intentar buscar en una nueva parte del espacio de búsqueda (si la única regla aplicable describe pequeños pasos, aplicarla varias veces en la misma dirección). Este método está especialmente indicado para superar mesetas.
- Aplicar dos o más reglas antes de realizar la evaluación. Esto se corresponde con movimientos en varias direcciones a la vez. Este método es especialmente bueno para superar las crestas.

La escalada es un método local, que decide cuál va a ser el siguiente movimiento atendiendo únicamente a las consecuencias "inmediatas" que va a tener esa elección, en lugar de explorar exhaustivamente todas las consecuencias.

3. Búsqueda primero el mejor

Representa una forma de combinar las ventajas que presentan la búsqueda primero en amplitud y la búsqueda primero en profundidad en un solo método.

En cada paso del proceso se selecciona el nodo más prometedor que se haya generado hasta ese momento mediante una función heurística. Lo normal es que la forma de funcionar se parezca un poco a la búsqueda primero en profundidad. Sin embargo, si no se encuentra una solución, la rama empezará a parecer menos prometedora que otras por encima de ella y que se habían ignorado. En ese caso, una rama que previamente se había ignorado aparecerá ahora como la más prometedora y comenzará su exploración. Sin embargo, la vieja rama no se olvida y la búsqueda puede volver a eIla en el momento en que sea de nuevo el camino más prometedor.

Sea ABIERTOS el conjunto de nodos que se han generado y a los que se les ha aplicado la función heurística, pero que aún no se han generado sus sucesores.

Algoritmo: Búsqueda primero el mejor

1. Comenzar con ABIERTOS conteniendo sólo el estado inicial.
2. Hasta que se llegue a un objetivo o no queden nodos en ABIERTOS hacer:
    a. Tomar el mejor nodo de ABIERTOS.
    b. Generar sus sucesores.
    c. Para cada sucesor hacer:
        i. Si no se ha generado con anterioridad, evaluarlo, añadirlo a ABIERTOS y almacenar a su padre.
        ii. Si ya se ha generado antes, cambiar el padre si el nuevo camino es mejor que el anterior. En este caso, se actualiza el coste empleado para alcanzar el nodo y a los sucesores que pudiera tener.

Algoritmo A*

4. Algoritmo A*

Sea CERRADOS el conjunto de nodos ya examinados y sea f’ una función de estimación igual a la suma de g y h' donde g es una medida del coste para ir desde el estado inicial hasta el nodo actual (no es una estimación) y h’ una medida estimada del coste de ir desde un nodo dado a una solución.

Algoritmo A*

1. Empezar con ABIERTOS conteniendo sólo el nodo inicial. Poner el valor g de ese nodo a 0, su valor h' al que corresponda (f=h'+0=h'). Inicializar CERRADOS corno una lista vacía.
2. Repetir el siguiente procedimiento hasta que se encuentre un nodo objetivo: si no existen nodos en ABIERTOS, informar del fallo. En caso contrario, tomar aquel nodo de ABIERTOS que tenga el menor valor de f'. Llamarle MEJORNODO, quitarlo de ABIERTOS y colocarlo en CERRADOS. Si MEJORNODO es un objetivo, salir e informar de la solución. En caso contrario, generar los sucesores de MEJORNODO, pero sin colocar MEJORNODO apuntando a ellos (antes debe mirarse si alguno de ellos ya ha sido generado). Para cada SUCESOR, hacer lo siguiente:
    a. Poner a SUCESOR apuntando a MEJORNODO (estos enlaces hacia atrás hacen posible que se recupere el camino una vez que se ha encontrado la solución).
    b. Calcular g(SUCESOR) = g(MEJORNODO) + el coste de ir desde MEJORNODO hasta SUCESOR.
    c. Si SUCESOR está contenido en ABIERTOS (es decir, si ya ha sido generado pero no procesado), se denomina a ese nodo VIEJO, se añade VIEJO a la lista de los sucesores de MEJORNODO. Para ver si cuesta menos llegar hasta VIEJO a través de su padre actual o hasta SUCESOR a través de MEJORNODO, se comparan sus valores g. Si VIEJO tiene menor o igual coste, no es necesario hacer nada, Si SUCESOR tiene menor coste, entonces se hace que el enlace paterno de VIEJO apunte hacia MEJORNODO, y que se grabe el nuevo camino óptimo en g(VIEJO) y que se actualice f'(VIEJO).
    d. Si SUCESOR no se encontraba en ABIERTOS, mirar si se encuentra en CERRADOS. Si es así, llamar VIEJO al nodo de CERRADOS y añadirlo a la lista de sucesores de MEJORNODO. Mirar si el nuevo camino es mejor que el viejo tal y como se hizo en el paso 2(c) y actualizar apropiadamente el enlace paterno y los valores de g y f'. Si se acaba de encontrar un camino mejor a VIEJO, se debe propagar la mejora a los sucesores de VIEJO. Esto es algo delicado: VIEJO apunta a sus sucesores, estos a los suyos y así sucesivamente. Para propagar el nuevo coste hacia abajo, se puede hacer una búsqueda en profundidad del árbol, empezando en VIEJO, cambiando el valor g de cada nodo (y por tanto su valor f'), terminando cada rama cuando se alcanza o bien un nodo sin sucesores, o bien un nodo para el que ya se ha encontrado un camino mejor: el enlace paterno de cada nodo apunta hacia atrás a su mejor predecesor conocido, conforme hay una propagación a un nodo siguiente, se debe comprobar si su predecesor apunta al nodo del cual se viene. Si lo hace así, la propagación debe continuar. Sino, en su valor de g ya aparece el mejor camino del que forma parte. Pero es posible que al propagar el nuevo valor de g hacia abajo, el camino que se está siguiendo pueda volverse mejor que el camino a través del antecesor actual. Por tanto, deben compararse los dos. Si el camino a través del antecesor actual todavía es mejor, debe detenerse la propagación. Si el camino a través del cual se propaga es ahora mejor, se actualiza el antecesor y se continúa con la propagación.

e. Si SUCESOR no estaba ya en ABIERTOS o en CERRADOS, ponerlo en ABIERTOS y añadirlo a la lista de sucesores de MEJORNODO. Calcular f'(SUCESOR) = g(SUCESOR) + h'(SUCESOR).

Si solo es relevante alcanzar el objetivo sea de la forma que sea, se puede definir g como 0, de forma que siempre se elija el nodo que parezca más cercano al objetivo final. Si lo que se desea es encontrar un camino con el menor número de pasos posibles, se hace que el coste aplicado para pasar de un nodo a otro sea constante, normalmente 1.

Si h' hace una estimación perfecta de h, entonces el algoritmo A* converge inmediatamente hacia el objetivo. Si, por otro lado, el valor de h' es siempre 0, la función g será la que controle la búsqueda. Si el valor de g es también 0, la estrategia de búsqueda será aleatoria. Si el valor de g es constante, la búsqueda será primero en amplitud.

Si se puede garantizar que **h' nunca sobrestima a h**, el algoritmo A* **garantiza encontrar el camino óptimo** hasta el objetivo, si es que existe. En la mayoría de los problemas reales, la única forma de garantizar que h' nunca sobrestima a h es hacer que su valor sea siempre 0. Un corolario de esto es: Si h' raramente sobrestima a h más de δ, entonces el algoritmo A' raramente encuentra un solución cuyo coste sea más de δ mayor que el coste de la solución óptima.

Sea ni un nodo objetivo no óptimo y sea Vopt el costo de la solución óptima:
f'(ni) = g(ni) + h'(ni) = g(ni) > Vopt
Dado un nodo nj que forma parte del camino óptimo, si h'(nj) no sobreestima a h(nj):
f'(nj) = g(nj) + h'(nj) ≤ Vopt
Por ende queda demostrado que: f'(nj) ≤ Vopt < f'(ni). Y, por lo tanto, A* encuentra el camino óptimo.

El algoritmo puede simplificarse para aplicarse a árboles haciendo que no se compruebe si un nuevo nodo se encuentra en ABIERTOS o CERRADOS, lo que hace más rápida la generación de nodos, pero puede ocurrir que la búsqueda se desarrolle varias veces por el mismo camino si los nodos se repiten con frecuencia.

En un problema de búsqueda donde hay varios caminos de solución, si se utiliza A* con h'(n)=0 y sabiendo que g(n)=1 (para todo n)

- Puede ocurrir que el algoritmo no encuentre la mejor solución debido a que el costo acumulado se encuentre en cada iteración.
- Puede ocurrir que el algoritmo no encuentre la mejor solución debido a que algunos nodos están sobrestimados.
- **El costo total del camino solución será igual a la cantidad de tramos**
- Ninguna de las anteriores

A* con h`=0 y g≠0 (tampoco constante)

- El algoritmo garantiza encontrar la solución en un menor número de tramos
- **Se encuentra la solución con el menor costo**
- El algoritmo no garantiza encontrar la solución con el menor número de tramos
- Ninguna es correcta

A* con f'= 0 y g=3 ¿¿f' o h'??

- No se puede asegurar que encuentre la solución en menor cantidad de tramos
- **Encuentra la solución en menor cantidad de tramos**
- Se comporta como escalada simple
- Ninguna de las anteriores

¿Qué sucede si todos los nodos están sobrestimados en A*?

- En búsqueda heurística si h' = 0
- Se encuentra la solución más corta en cantidad de tramos.
- **No se garantiza que se encuentre el camino óptimo.**
- Se encuentra el camino óptimo.
- Ninguna de las anteriores.

Explique por qué al sobrestimar h no se garantiza encontrar el camino óptimo en A*

Porque de esa manera se puede encontrar un camino no óptimo cuyo costo es menor al estimado f' de algún nodo del camino óptimo (debido a la sobrestimación de h), encontrándose la solución antes de que se explore el nodo que lleva a la solución óptima.

Para resolver un problema de búsqueda se ha diseñado una heurística que está sobreestimada. ¿Es posible que el algoritmo A* no encuentre un camino entre el estado inicial y el estado final? Suponga que sólo hay un estado inicial y un estado final y que hay solo un camino entre ellos. Justifique y demuestre para todos los casos

No, eso no es posible. Si existe una solución, el algoritmo A* la encuentra, solo que cuanto peor es la heurística, más exhaustiva es la búsqueda. La única condición de fallo del algoritmo es que no existan nodos en ABIERTOS por explorar y la única forma de que esto pase es que se exploren todos los nodos y ninguno de ellos tenga sucesores, por lo que no existiría un camino desde el estado inicial al estado final.

Dado que h' sobrestima a h y teniendo estado inicial, estado final y varios caminos al estado final. ¿Es posible que A* no encuentre un camino a la solución?

No, eso no es posible. Si existe una solución, el algoritmo A* la encuentra. Lo que no se garantiza en esas condiciones (debido a la sobrestimación de h) es que encuentre la solución óptima.

Comparación entre búsqueda primero en amplitud y A* con h' realizando estimación perfecta de h y g=1:

- A* encuentra el camino de menor costo.
- Primero en amplitud encuentra el camino de menor costo.
- A* encuentra el camino de menor longitud en cantidad de tramos.
- Primero en amplitud encuentra el camino de menor longitud en cantidad de tramos.
- **Todas son correctas.**

Sobreestimación y subestimación en A*

Si h' está sobrestimada no se garantiza encontrar la solución óptima. Si está subestimada sí pero se pierde más tiempo que si estuviera perfectamente estimada ya que en este último caso el algoritmo converge directamente a la solución.

Si no está seguro de que la heurística que ha diseñado es adecuada para encontrar el camino óptimo de solución, ¿qué debe hacer para garantizar que el algoritmo de A* devuelva dicho camino? Justifique su respuesta.

A* garantiza encontrar la solución óptima si h' no sobrestima a h. La única manera de asegurarse de eso es hacer h'=0 para todos los casos, de manera que el algoritmo encuentre el camino solución con el menor costo.

Si h´ subestima a h puede pasar:

- Que el algoritmo no encuentre el camino con menor costo del origen al destino.
- Que el algoritmo encuentre la solución pero que no sea el camino con menor costo.
- Que el costo real acumulado esté subestimado.
- **Ninguna de los anteriores.**

¿Cuál es la condición de fin de A*?

Mostrar un ejemplo de A* donde se vea claramente el problema de que h' sobreestima a h.

Algoritmos genéticos

1. Introducción

Un Algoritmo Genérico simula la evolución de una población de individuos mediante un proceso iterativo aplicado sobre un conjunto de estructuras. Cada estructura está compuesta de características que definen la aptitud del individuo en un entorno. La población de estructuras evoluciona de generación en generación mediante la combinación de sus integrantes, la mutación de algunas características elegidas al azar, y la selección de aquellas estructuras que se mostraron más aptas.

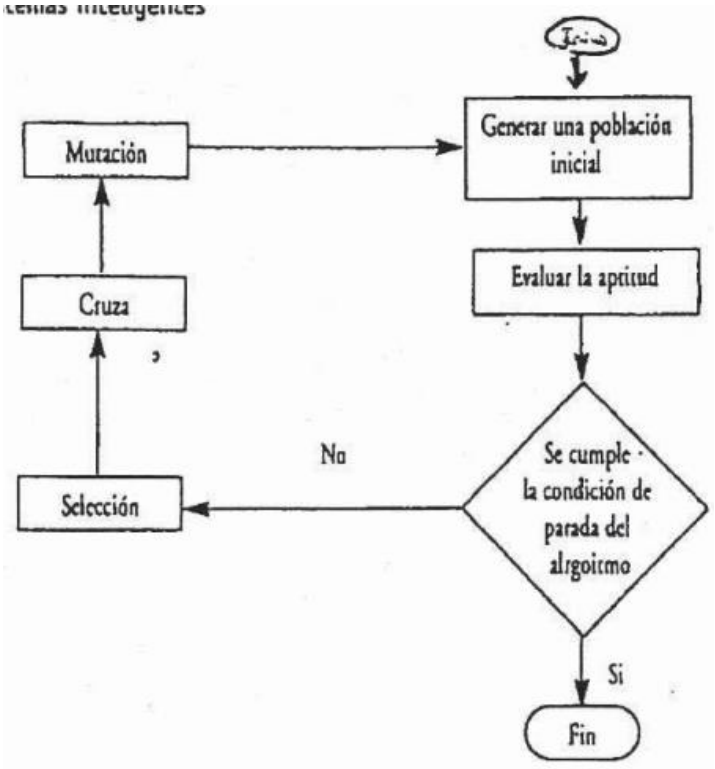
Desde el punto de vista de la resolución de problemas, un individuo representa una solución posible a un problema dado. La aptitud del mismo es una medida de cuán buena es la solución para dicho problema. El objetivo del AG es entonces buscar una "buena" solución al problema.

1.1 Algoritmo simple

Previo a la aplicación de un AG se deben definir dos elementos: la representación de las soluciones candidatos en individuos de una población y la construcción de una función que medirá la aptitud de cada uno de los individuos.

La generación de la población inicial se realiza habitualmente creando individuos en forma aleatoria. Luego, otra generación se crea a partir de la generación anterior tras la aplicación de tres operadores básicos: selección, cruza y mutación. El proceso iterativo prosigue hasta que algún individuo presenta una aptitud suficientemente buena, se alcanza una cota máxima de evaluaciones o generaciones, toda la población está dominada por el mismo individuo o se cumple alguna condición específica del problema.

La selección es el proceso por el cual sobreviven los mejores individuos, asignando a cada individuo una cantidad de copias de sí mismo acorde a su aptitud de modo que un individuo con buena aptitud tendrá más probabilidades de participar de la cruza que uno poco apto. Algunos de los individuos seleccionados sobreviven directamente en la próxima generación y otros participan de la cruza contribuyendo en forma indirecta con el aporte de su material genético. La proporción de la población que es reemplazada en cada generación se denomina salto generacional.



La cruza toma dos individuos sobrevivientes, recombina algunas de sus características -material genético-

y produce dos nuevos individuos que pertenecen a la próxima generación.

Para garantizar que ninguna característica se pierda irremediablemente de la población se utiliza el operador de mutación. La mutación toma unos pocos individuos generados por la cruza y altera algunas características tomadas al azar.

El ciclo comienza con la generación aleatoria de una población inicial de estructuras. Luego se evalúa la aptitud de cada una aplicando la función f. Si alguna estructura, o la evolución misma, cumple con la condición de parada, el algoritmo termina y el resultado es la mejor estructura presente en la población. Sino, se seleccionan las estructuras más aptas asignando a éstas una cantidad de copias proporcional a su aptitud. Parte de esa población intermedia se cruza para generar nuevas estructuras que heredan el material genético de sus padres. Por último, algunas características de la población son elegidas para mutar, alterando así las estructuras que lo contienen. La población recién creada es evaluada y el ciclo vuelve a comenzar.

1.3 Sistemas Naturales y Sistemas Artificiales

En la naturaleza, las características de un organismo están determinadas por uno o más cromosomas. En los sistemas genéticos artificiales, cada individuo se representa con una única estructura.

Un cromosoma está compuesto por genes, cada uno de los cuales puede tomar un valor llamado alelo. La posición de un gen dentro de un cromosoma se denomina locus. En la genética artificial las estructuras están compuestas de características que pueden tomar diferentes valores.

El conjunto de genes de un individuo se llama genotipo. El fenotipo es el resultado de la interacción del genotipo con su entorno. Desde el punto de vista de la genética artificial, el fenotipo se corresponde con la aptitud del individuo.

1.4. Aplicaciones

Algoritmos Genéticos: características y operadores básicos.

Los Algoritmos Genéticos son atractivos por varias razones:

1. Pueden resolver en forma rápida y eficiente problemas complejos con características como alta cardinalidad del espacio de búsqueda, alta dimensionalidad de la función de aptitud, y funciones de aptitud no lineales.
2. Utilizan muy poca información específica del problema, dado que solo requieren la posibilidad de proponer una solución y asignar una evaluación.
3. Son extensibles: en los problemas reales es muy difícil anticipar todas las dificultades, sin embargo es muy fácil modificar o incorporar conocimiento en los operadores genéticos.
4. Pueden ser utilizados para realizar una primera búsqueda global para luego continuar con algún método de búsqueda local.

Las Algoritmos Genéticos tienen básicamente dos campos de aplicación:

1. Los problemas de optimización.

2. La simulación de ambientes donde el objetivo es maximizar los beneficios acumulados a través del tiempo.

2. Operadores básicos: Variantes

El propósito de cada uno de los tres operadores básicos está bien definido: elegir los individuos más aptos (selección), recombinar algunos de ellos para producir nuevos individuos (cruza), y alterar características de algunos para garantizar diversidad (mutación). Existe una gran cantidad de variantes para cada uno de los operadores y una elección adecuada puede influir decisivamente en la eficacia y eficiencia del proceso de búsqueda.

2.1 Métodos de Selección

Existen dos tipos de métodos usados comúnmente:

- Los proporcionales: eligen individuos teniendo en cuenta el peso de su aptitud respecto del resto de la población.
- Los basados en el orden: confeccionan una tabla ordenada de individuos en base a su aptitud, seleccionando cada uno de acuerdo a su ubicación es esta tabla de posiciones.

2.1.1 Selección por Ruleta

Este método consiste en construir una rulera particionada en ranuras de igual tamaño, las cuales se enumeran. A cada individuo de la población se le asigna una cantidad de ranuras proporcional a su aptitud. La ruleta se gira y con probabilidad uniforme se elige una ranura, seleccionándose aquel individuo dueño de la misma. Este método otorga mayor probabilidad de contribuir a la siguiente generación a los individuos con mayor aptitud. Como se trata de un método probabilístico, el número obtenido de copias para un individuo puede ser muy distante del esperado.

2.1.2. Selección con Control sobre el Número Esperado

Para reducir la distancia entre el número esperado de copias y la cantidad asignada en la selección por ruleta, en este método se define una cantidad de copias ci para un individuo i tal que $c_i = \frac{f_i}{\underline{f}}$donde fi es la aptitud del i-ésimo individuo y $\underline{f}$es la aptitud promedio de la población. Cada individuo recibe una cantidad de copias igual a la parte entera de ci i más una copia adicional con probabilidad igual a la parte fraccionaria de ci.

2.1.3. Selección Elitista

Los dos métodos anteriores no garantizan la preservación de los mejores individuos ya que estos pueden ser reemplazados por sus hijos durante la cruza. La selección elitista soluciona este problema común a la mayoría de los métodos de selección, preservando los mejores individuos de la generación actual, incluyéndolos directamente en la siguiente. Se busca mejorar la búsqueda local a expensas de la perspectiva global y provoca que la aptitud del mejor individuo de la población nunca decrezca. Siempre es utilizada en combinación con otras variantes de selección.

2.1.4. Selección por Ranking

Un problema habitual de los métodos proporcionales es la posibilidad de que súper-individuos dominen rápidamente la población antes de que se haya podido realizar una exploración

suficientemente amplia del espacio de búsqueda (convergencia prematura). En este método cada individuo recibe una cantidad de copias que sólo depende de su ubicación dentro de la tabla. Para esto, la población se ordena en forma descendente por la aptitud de cada individuo y solamente los primeros m individuos reciben copias. Dentro de esas m primeras posiciones de la tabla, el total de copias a asignar se distribuye en forma lineal. Sea n el tamaño de la población, la peor estructura recibe un número esperado de copias Rmin ∈ (0,1) y la mejor recibe un número esperado de 2-Rmin. Al individuo ubicado en la posición i de la tabla se le asigna la siguiente cantidad de copias: $R_{min} + 2 \,.\, \frac{(n-i)\,.(1-R_{min})}{n-1}$

2.2 Métodos de Cruza

Cada estructura participa de la cruza con probabilidad Pc. La cantidad de hijos generados depende de la implementación. La elección más aceptada consiste en generar dos hijos, siendo uno el complemento del otro. Sin embargo, también hay variantes donde la cruza genera un único hijo o más de dos.

2.2.1. Cruza Simple

La cruza simple o en un punto elige al azar uno de los l-1 posibles puntos de cruza. Luego de esta elección se intercambian los segmentos de cromosoma entre este punto y l.
Por ejemplo, sea una estructura de l=8 y eligiendo 3 como punto de cruza se tiene:
XYX | XYYYX → XYX | YYXXY
YYX | YYXXY → YYX | XYYYX

2.2.2 Cruza Multipunto

En este caso el cromosoma es considerado un anillo y se eligen n puntos de cruza en forma aleatoria. Si la cantidad de puntos de cruza es par, se intercambian las porciones de cromosomas definidas entre cada par de puntos consecutivos. Si es impar, se asume un punto de cruza adicional en la posición cero y se procede de igual modo.
Por ejemplo, sea la longitud l=8, con puntos de cruza en la posición 2, 4, 6 y 7 se tiene:
XY | XX | YY | Y | X → XY | XY | YY | X | X
YY | XY | YX | X | Y → YY | XX | YX | Y | Y

2.2.3. Cruza Binomial

Para generar un cromosoma hijo por cruza binomial se define la probabilidad P0 como la probabilidad de que el alelo de cualquier posición del descendiente se herede del padre y 1-P0 como la probabilidad de que lo herede de la madre. En este caso se puede construir un único hijo por cada aplicación del operador, o bien generar un segundo hijo como complemento del primero. Cuando existe igual probabilidad de heredar del padre como de la madre (P0=0,5), la cruza se denomina uniforme.
Por ejemplo, sea p=0,3 y m=0,7. Números aleatorios: 27, 14, 60, 33, 24, 88, 15, 46.
p: XYXXYYYX → XYXYYXYY
m: YYXYYXXY

2.3. Métodos de Mutación

La mutación permite mantener diversidad en la población, disminuyendo el riesgo de convergencia prematura, y cumple una función de exploración, ya que este operador brinda la posibilidad de generar cualquier estructura válida dentro del espacio de búsqueda. Cada

gen es mutado con una probabilidad p generalmente baja. La probabilidad puede ser constante durante toda la búsqueda genética, o bien, adaptativa, donde se utilizan frecuentemente estadísticas de la población para adaptar la velocidad de mutación.

2.3.1. Mutación Simple

Elige en forma aleatoria un gen, el cual se muta con cierta probabilidad, habitualmente muy baja, la cual se mantiene constante durante las sucesivas generaciones.

2.3.2. Mutación Adaptativa por Convergencia

La probabilidad de mutación varía en base a información proveniente de la búsqueda genética. Esta información está dada por el grado de convergencia de la población. Esta variante de mutación tiene la ventaja de aprovechar la información histórica para orientar la búsqueda, aumentando la mutación cuando la población se hace muy homogénea y disminuyéndola cuando hay demasiada diversidad.

2.3.3 Mutación Adaptativa por Temperatura

Este tipo de mutación no utiliza información genética de la población, por lo tanto, es independiente de las características de la misma. La probabilidad de mutación, Pm depende del tiempo o cantidad de generaciones: por lo tanto Pm = Pm(t). El rango de valores que puede tomar está acotado: Pmin(t) ≤ Pm(t) ≤ Pmax(t).

Partiendo de un valor inicial P(0), la actualización se realiza hasta alcanzar un valor final, de la siguiente manera: **Pm(t+1) = Pm(t) ∓ |ƛ|**. Dependiendo del signo de ƛ y de los valores inicial y final de Pm(t), hay dos variantes del método:

- Mutación Adaptativa por Temperatura Ascendente
  La probabilidad de mutación aumenta con las sucesivas generaciones hasta alcanzar el valor máximo, y a partir de allí, mantenerse constante. El objetivo de esta variante es mantener la diversidad de la población, que con el transcurso del tiempo tiende a hacerse homogénea. El aumento de mutación debe tener una cota máxima. De lo contrario, se hace imposible la supervivencia de buenos individuos a lo largo de la evolución y se corre el riesgo de que la población nunca converja.

- Mutación Adaptativa por Temperatura Descendente
  Se comienza con una mutación alta, la cual va descendiendo con el paso del tiempo. El objetivo es contribuir a una exploración más alta en las primeras generaciones, evitando el problema de convergencia prematura. A medida que el tiempo avanza, la probabilidad de mutación desciende hasta una cota mínima y se estabiliza. Esta cota siempre debe ser mayor a cero, puesto que sin mutación habrá probabilidad de perder para siempre estructuras interesantes de! espacio de búsqueda.

3. Consideraciones

Para que un AG tenga éxito es necesario considerar los siguientes aspectos:

- La búsqueda se debe prolongar en el tiempo sólo si se continúan realizando avances significativos.
- Durante la búsqueda se deben explotar posibilidades que mejoren la aptitud, de lo contrario, se desperdicia una considerable cantidad de esfuerzo.
- Siempre se puede estar no explotando posibilidades que contienen la clave para lograr la aptitud óptima.

Explique cómo se aplican los operadores de selección, cruce y mutación en los algoritmos genéticos para generar una población nueva.

Se determina de antemano el porcentaje de la población destinada a la cruza, selección y mutación. Luego se llevan a cabo dichos procesos, tomando para cada uno los individuos de la generación actual (no se encadenan los operadores):

- Se eligen los individuos más aptos mediante la selección (todos los individuos participan de este proceso). La cantidad de individuos seleccionados depende del porcentaje de la población asignado a la selección.
- Se determinan aleatoriamente los individuos que participarán de la cruza teniendo en cuenta el porcentaje asignado a este proceso. Los individuos seleccionados se combinan entre ellos para producir nuevos individuos.
- Se determinan aleatoriamente los individuos que participarán de la mutación teniendo en cuenta el porcentaje asignado a este proceso. A los individuos seleccionados se les alteran algunas características para garantizar diversidad.

Si se tiene una solución de AG que utiliza selección elitista 15%, cruza 70% y mutación 15% y no se llega a la solución esperada. ¿Es una buena alternativa cambiar los métodos de cruza y mutación por selección basada en orden? Justifique. En AG para resolver un problema se determina utilizar selección elitista 20%, cruza 75% y mutación por temperatura ascendente 5%. Como no se encontró la solución en varias corridas se decidió reemplazar los operadores de cruza y mutación por métodos de selección por control de copias esperadas y ranking respectivamente. ¿Considera adecuada esta decisión? Justifique su respuesta.

No, no es una decisión adecuada ya que solo quedarían métodos de selección, por lo que se seleccionarían una y otra vez los mismos individuos, sin que se produzcan cambios en estos. De esta manera, encontrar la solución solo dependería de los individuos generados en la población inicial. Si bien sería teóricamente posible llegar a una solución, se estaría reduciendo los algoritmos genéticos a una simple búsqueda aleatoria dependiente de la generación de la población inicial.

Determine un índice que permita medir la diversidad de una población y que sea distinto del propuesto en el material de estudio. Justifique.

Para medir la diversidad de la población se puede utilizar el criterio de la ortogonalidad: sea N el número de genes de los individuos, se asigna al gen X o 0 el valor -1 y al gen Y o 1 el valor +1. Luego la ortogonalidad está dada por la sumatoria de 1 a N del producto entre los genes en la misma posición. Otra forma sería contabilizar el número de individuos iguales.

Utilizando un método de selección basado en el orden:

- **El peor individuo no puede pasar a la generación siguiente**
- **El mejor individuo posee la máxima cantidad de copias asignadas**
- Se puede implementar selección por torneo
- Ninguna de las anteriores

¿Cómo se resuelve la Convergencia Prematura?

La convergencia prematura es el fenómeno por el cual individuos con aptitud muy alta dominan rápidamente la población sin que se haya podido realizar una exploración suficientemente amplia sobre el espacio de búsqueda. Para solucionar este problema se aplican operadores de mutación.

Diferencias entre Algoritmos Genéticos y de búsqueda Heurística.

Si bien ambos utilizan una función para determinar la deseabilidad de un estado, en la búsqueda heurística la solución está constituida por el camino entre el nodo inicial y el nodo objetivo, mientras que en algoritmos genéticos no interesa la secuencia de poblaciones que se generaron antes, solo importa el individuo solución en sí mismo. Además, en algoritmos genéticos no se parte de un estado inicial sino de posibles soluciones aleatorias y no hay operadores o transformaciones válidas entre ellas.

¿Cómo implementaría en AG, la mutación adaptativa por convergencia?

Se podría implementar, por ejemplo, haciendo que la probabilidad de mutación sea igual al porcentaje de individuos iguales existentes en la población. Así, si por ejemplo el 25% de los individuos son iguales, la probabilidad de mutación es de 0,25. De esta manera, cuanto más homogénea es la población, mayor es la probabilidad de mutación con el propósito de evitar la convergencia prematura.

Programación evolutiva

Los procesos de búsqueda evolutivos (donde se utiliza la idea de selección natural) se han usado con 2 grandes propósitos:

- En algoritmos genéticos, utilizados para buscar el valor óptimo de una función.
- Para evolucionar programas con el objetivo de resolver problemas específicos (por ejemplo, programas para controlar agentes reactivos). De esto trata este capítulo.

Los programas funcionales pueden expresarse como árboles con nodos etiquetados. Los nodos internos son funciones, acciones o predicados que toman uno o más argumentos. Los nodos hoja son constantes, acciones o funciones que no toman argumentos.

Veremos cómo usar la Programación Genética (PG) para evolucionar un programa que controla un robot que, desde cualquier posición inicial, busca una pared en una grilla bidimensional y, una vez encontrada, sigue por el borde de la misma indefinidamente. El movimiento del robot se produce por la ejecución reiterada del programa, que toma como entrada los datos actuales del sensor (posiciones n, ne, e, se, s, so, o y no con un 0 si la celda está libre para ser ocupada por el robot, y 1 en caso contrario) y ejecuta una sola acción. Las funciones a utilizar son las funciones booleanas IF, AND, OR, NOT y cuatro acciones de movimiento: norte, este, sur y oeste.

En programación genética se comienza con una población de programas aleatorios que constituyen la generación 0. El tamaño de la población de dicha generación es uno de los parámetros, para el ejemplo serán 5.000. Los programas en cada generación son evaluados y se produce una nueva generación hasta que se produzca uno que tenga un desempeño aceptable. Para el ejemplo se correrá cada programa 60 veces y se contará la cantidad de celdas pegadas a la pared que visita el robot en esos 60 pasos (hay 32 celdas, siendo este

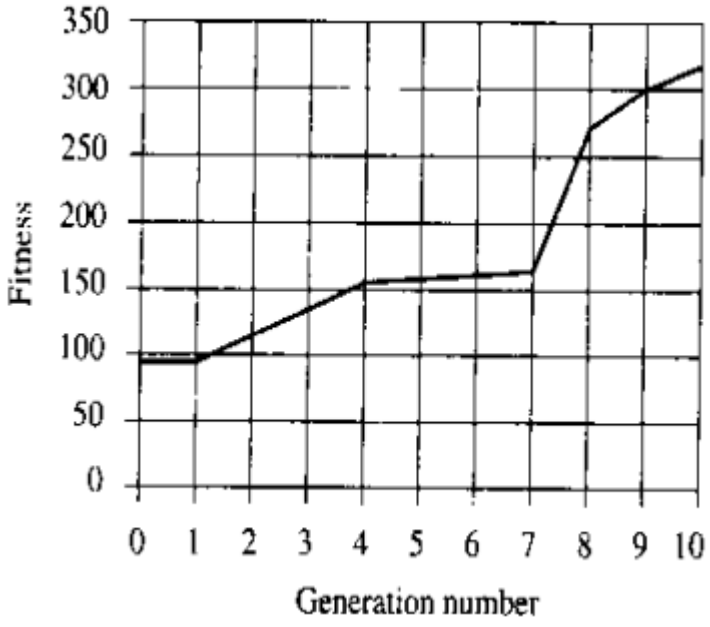
el máximo). Para cada programa, se ejecutarán 10 corridas con posiciones iniciales aleatorias; la cantidad total de celdas pegadas a la pared visitadas será el **fitness** del programa (siendo 320 el máximo valor posible).

A partir de la generación i se construye la generación i+1 de la siguiente manera:

1. El 10% de la generación i se copia directamente en la generación i+1, siguiendo la siguiente selección por torneo: se seleccionan 7 en forma aleatoria y se comparan, el mejor pasa a la siguiente generación.
2. El 90% restante son programas hijos: se selecciona un padre y una madre mediante selección por torneo. Luego, un subárbol del padre seleccionado aleatoriamente reemplaza un subárbol de la madre también seleccionado aleatoriamente. La posibilidad de ejecutar el programa hijo está garantizada por el requisito de que todas las funciones usadas en los programas sean ejecutables para cualquier valor de sus argumentos.
3. A veces se incluye un operador de mutación en la construcción de individuos para la siguiente generación (usado en un porcentaje bajo). Este selecciona un padre/madre por torneo y reemplaza un subárbol aleatorio por un nuevo subárbol (generado de la misma manera que la generación 0).

Son parámetros el porcentaje de individuos que se copia, el porcentaje producido por cruza, el porcentaje de mutación y el número usado en la selección por torneo.

En el ejemplo, el mejor fitness de la generación 0 es 82. De la generación 2 es 117. De la 6, 163. Finalmente, en la generación 10, un programa logra realizar correctamente el recorrido desde cualquier posición, logrando el fitness máximo (320).



Unidad 4: Aprendizaje

Explique aprendizaje supervisado, no supervisado y por refuerzo. Ventajas y desventajas de cada método.

Existen 3 tipos de aprendizaje:

- **Aprendizaje supervisado**: hay un supervisor externo y se tienen datos iniciales que están etiquetados con la clase correspondiente, los cuales se utilizan para el entrenamiento: el algoritmo aprende comparando el resultado obtenido con el deseado y modificando el modelo en consecuencia.
  Ejemplos: KNN, Perceptrón, MLP, SVM, Naive Bayes, árboles de decisión.
- **Aprendizaje no supervisado**: el algoritmo no cuenta con datos etiquetados para el entrenamiento, por lo que debe extraer las características relevantes y buscar correlaciones para generar automáticamente las clases sin saber qué resultados son satisfactorios.
  Ejemplos: Clustering, K-Means, SOM, Hopfield.
- **Aprendizaje por refuerzo**: el algoritmo interactúa con el medio, obteniendo retroalimentación sobre el efecto de sus acciones en dicho entorno (recompensas positivas o negativas) y usando dicha información para mejorar sus respuestas. Aprende a través de prueba y error descubriendo qué acciones brindan las mayores recompensas. Ejemplo: al aprender a andar en bicicleta no se sabe lo que hay que hacer, pero a partir de las caídas se aprende qué es lo que hay y no hay que hacer.

¿Cuál es la diferencia entre aprendizaje supervisado y aprendizaje por refuerzo?

En el aprendizaje supervisado se tienen datos etiquetados con la clase correspondiente y el algoritmo aprende comparando el resultado obtenido con el deseado y modificando el modelo en consecuencia. Mientras que en el aprendizaje por refuerzo el algoritmo interactúa con el medio y se basa en la monitorización de las respuestas a sus acciones para mejorar.

Los métodos de aprendizaje supervisado son:

- **Perceptrón simple**
- **Perceptrón multicapa**
- K-Means
- Ninguna de las anteriores

Algoritmos que tienen aprendizaje supervisado:

- **Perceptrón**
- **NB con distribución gaussiana**
- **NB con histogramas**
- Ninguna de las anteriores.

¿Qué es el aprendizaje por descubrimiento? Explicar los tipos de aprendizaje por descubrimiento.
¿Qué es el aprendizaje por analogía? Explicar los tipos de aprendizaje por analogía.

Tipos de aprendizaje en función de la adquisición de conocimiento:

1. Memorístico: consiste en la acumulación de conocimiento e información. Involucra:

    a. Almacenamiento organizado de la información: se almacenan con algún orden los valores calculados, de modo que se los pueda recuperar más rápido que repitiendo el cálculo.
    b. Capacidad de generalización: consiste en almacenar generalidades en lugar de todas las situaciones particulares.
2. A través de consejos: se dan consejos a alto nivel que luego deben traducirse en acciones de bajo nivel.
3. Por ajuste de parámetros: se aumenta el peso de aquellas características que resultan adecuadas para realizar predicciones exitosas y se disminuyen los de aquellas que no lo son. Si el ajuste se hace automáticamente, el sistema aprende de manera autónoma. Se debe considerar el momento del ajuste y su magnitud.
4. A través de ejemplos: se trata de un aprendizaje inductivo, de manera que se generaliza a partir de situaciones particulares.
5. Basado en explicaciones: se parte de una única explicación sobre por qué X es un ejemplo del concepto objetivo y luego se generalizan las explicaciones. El factor determinante es la teoría del dominio que sustenta y generaliza la explicación.
6. Por descubrimiento: consiste en la adquisición de conocimiento de forma no guiada, sin un “profesor”.
    a. Descubrimiento conducido por teorías: se realiza teniendo en cuenta alguna descripción del dominio de aplicación.
    b. Descubrimiento conducido por datos: se utilizan variables y a partir de la modificación de sus valores se infiere conocimiento.
    c. Agrupamiento/Clustering: es el mismo sistema el que debe descubrir similitudes entre los datos y a partir de ellas clasificarlos.
7. Por analogía: consiste en la resolución de nuevos problemas basándose en la comparación con algo similar ya conocido.
    a. Aprendizaje por analogía transformacional: consiste en la transformación de la solución de un problema previo en una solución para el problema actual.
    b. Aprendizaje por analogía derivacional: considera los pasos seguidos en la resolución de problemas anteriores para resolver el problema actual.

¿Cual es la razón de inicializar los valores optimista el proceso de aprendizaje por refuerzo? Justifique

Naive Bayes

1. Conceptos de probabilidad

Espacio muestral (S): es el conjunto de todos los resultados posibles para un experimento dado.
Ejemplo: dado un mazo de cartas, si se toma una carta el espacio muestral es de 52 elementos (cartas) posibles.

Evento: un evento A respecto a un espacio muestral S es simplemente un conjunto de resultados posibles (subconjunto de S) del experimento e.
Ejemplo: sale el 5 de corazones (conjunto unitario), sale una carta roja (26 elementos)

Probabilidad: es un número que se asocia a un evento determinado que refleja la posibilidad de que ocurra ese evento.

Probabilidad condicional: dados dos eventos A y B asociados a un experimento e, se define P(B|A) como la probabilidad de que ocurra B habiendo ya ocurrido A.
P(B|A) = P(A ^ B) / P(A)
P(A|B) = P(A ^ B) / P(B)
P(B|A) . P(A) = P(A|B) . P(B)

Si se cumplen las siguientes condiciones:

- El espacio muestral S está particionado en n particiones disjuntas B_i: $\forall\, i \neq j: B_i \cap B_j = \emptyset$
- El espacio muestral S es igual a la unión de todas las particiones: $S = \cup_{i=1}^{n} \quad B_i$
- La probabilidad de cada partición es mayor a 0: $\forall\, i: P(B_i) > 0$

La probabilidad de un evento A respecto a S se puede expresar como:

$$P(A) = P(A|B_1).P(B_1) + P(A|B_2).P(B_2) + \ldots + P(A|B_n).P(B_n)$$

Teniendo en cuenta la expresión anterior para la P(A) y que P(B|A).P(A)=P(A|B).P(B), tenemos el **Teorema de Bayes** o fórmula para la probabilidad de las causas:

$$P(B_i|A) = \frac{P(A|B_i).P(B_i)}{\sum_{j=1}^{n} \quad P(A|B_j).P(B_j)} \quad i = 1, 2, \ldots, n$$

Si X e Y son independientes, entonces la función de probabilidad conjunta de X e Y es igual al producto de sus marginales: p(x ^ y) = p(x) . p(y)

Independencia de 2 variables: sean X e Y dos conjuntos disjuntos del conjunto de variables aleatorias. Entonces X es independiente de Y si y sólo si: p(x|y) = p(x) para todo valor posible de x ∈ X y de y ∈ Y. Para tres variables tenemos: p(x,y | z) = p(x | z) . p(y | z)

Independencia de un conjunto de variables: sean las variables aleatorias $\{X_1, \ldots, X_m\}$se dice que son independientes si y sólo si: $p(x_1, \ldots, x_m) = \prod_{i=1}^{m} \quad p(x_1)$para todos los valores posibles $x_1, \ldots, x_m$ de $X_1, \ldots, X_m$.

2. Clasificador Naive Bayes

El clasificador Naive Bayes aplica a tareas de aprendizaje donde cada instancia x es descrita por un conjunto de valores de atributos y la función objetivo f(x) puede tomar cualquier valor de un conjunto finito C de clases posibles. Primero se proporciona un conjunto de ejemplos de entrenamiento de la función objetivo. Luego se presenta una nueva instancia, descrita por una tupla de valores de los atributos $(a_1, a_2, \ldots, a_m)$, y se desea predecir la clase c_i para esta nueva instancia.

Los Sistemas Basados en Probabilidades (SBP) manejan las relaciones entre las variables de un problema mediante la utilización de la probabilidad conjunta. Pero calcular los valores de probabilidad conjunta para todos los objetos que forman parte de un problema se vuelve intratable.

Para resolver esta "intratabilidad" se asume que los atributos son condicionalmente independientes (de ahí el término "naive" del método):

Extendiendo el teorema de Bayes para m atributos se tiene:

$$P(c_i \mid a_j, \ldots, a_m) = \frac{P(c_i) \,.\, P(a_1, \ldots, a_m \mid c_i)}{\sum_{j=1}^{n} \quad P(c_j) \,.\, P(a_1, \ldots, a_m \mid c_j)} \qquad i = 1, 2, \ldots, n$$

Y teniendo en cuenta la independencia condicional de los atributos: $P(a_1, \ldots, a_m \mid c_i) = P(a_1 \mid c_i) \,.\, P(a_2 \mid c_i) \ldots\ldots P(a_m \mid c_i)$ donde cada uno de estos términos se llama verosimilitud.

Dado que las probabilidades para cada c_i estarán divididas por el mismo valor, tenemos:

$$P(c_i \mid a_1, \ldots, a_m) \;\propto\; P(c_i) \,.\, \prod_{j=1}^{m} \quad P(a_j \mid c_i) \qquad i = 1, 2, \ldots, n$$

De esta manera, se asigna a la nueva instancia la clase con mayor proporción de probabilidad. Si se desea, se pueden normalizar las probabilidades dividiendo cada una por la sumatoria de todas ellas: sean p(c1)∝0,003, p(c2)∝0,015, p(c3)∝0,008 tenemos p(c1)=0,003/0,026=0,1154

$$c_{NB} = argmax \quad_{c_i \epsilon C} \; P(c_i) \prod_{j=1}^{m} \quad P(a_j \mid c_i)$$

Características:

- Es un método de clasificación global ya que utiliza todo el conjunto de datos
- Es un método perezoso (lazy) o basado en instancias: no entrena un modelo que luego solo se aplica sino que demora el procesamiento hasta que una nueva instancia deba ser clasificada
- Simpleza en los cálculos

Problemas:

- Clases con diferentes densidades: si hay una gran disparidad en la cantidad de elementos de las clases, las prioridades a priori serán muy desiguales, lo que inclinará la clasificación por la clase con mayor cantidad de elementos
- No realiza una buena clasificación cuando las clases están muy mezcladas
- Cuanto mayor es la cantidad de clases y atributos, mayor es la cantidad de cálculos que se deben realizar
- Maldición de la dimensionalidad: al aumentar la cantidad de atributos (dimensiones), disminuye la variación entre las instancias.

Naive Bayer con distribución Gaussiana. ¿Cómo se calcula? ¿Qué datos utiliza? ¿Cuáles son los elementos necesarios para poder aplicar Naives Bayes con distribución gaussiana?

3. Naive Bayes con Distribuciones continuas

Los casos de variables continuas pueden resolverse haciendo una aproximación de los valores de los atributos, suponiendo una distribución de probabilidades para calcular las verosimilitudes. Por ejemplo, se puede asumir que los datos tienen una distribución normal. Se calculan entonces los parámetros de la distribución (en este caso la media y el desvío estándar) de cada variable para cada clase, y se utilizan dichos valores en la función de distribución normal. Las probabilidades a priori de cada clase se calculan normalmente, haciendo $P(c_i) = \frac{\#c_i}{N}$. Finalmente, las verosimilitudes se utilizan en conjunto con la probabilidad a priori de la clase para determinar la probabilidad de dicha clase dados los valores de los atributos.

$$\underline{x} = \frac{1}{N}\sum_{i=1}^{N} x_i \qquad \sqrt{\sigma^2} = \sqrt{\frac{\sum_{i=1}^{N} (x_i-\mu)^2}{N}} \qquad f(x) = \frac{1}{\sigma\sqrt{2\pi}} . e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$

Ventajas y problemas al utilizar Naive Bayes con distribuciones de probabilidad. Explique y ejemplifique los problemas de NB con distribuciones continuas.

Ventajas:

- Si se conoce la distribución, no hay que tomar ninguna decisión (como por ej. establecer la cantidad o tamaño de bins), simplemente se deben aplicar fórmulas.

Problemas:

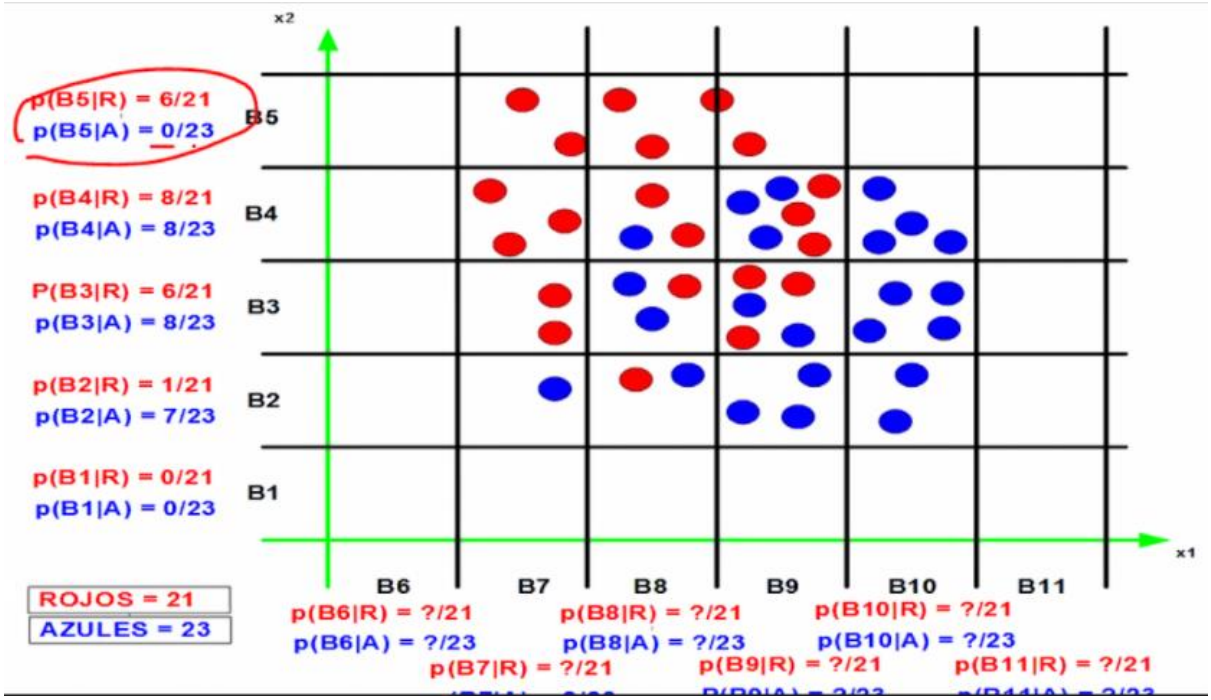
- En caso de que los datos no se ajusten a la distribución elegida, se pueden cometer grandes errores, haciendo que el método no tenga sentido. Para arreglar esto, se debería hacer un análisis de los datos y elegir una distribución que se adapte a ellos.
- Si los datos están en más de 3 dimensiones es imposible verlos para determinar su distribución.
- Añadir nuevas instancias implica que se tengan que recalcular los parámetros.

Explique el clasificador de Naive Bayes con Histogramas

4. Naive Bayes con Histogramas

El método de Naive Bayes con Histogramas consiste en realizar particiones del conjunto de valores posibles de los atributos, conformando intervalos o bins para cada atributo. Una vez determinados los intervalos, se contabilizan las ocurrencias de cada clase en cada intervalo de cada atributo. Luego, cada intervalo se considera como si fuera un valor discreto del atributo, pudiéndose calcular las verosimilitudes para los mismos. Para cada clase se calcula la probabilidad a priori $P(c_i) = \frac{\#c_i}{\sum_i^n \#c_i}$, las verosimilitudes para los valores dados de los atributos: $P(a_j|c_i) = \frac{\#c_i \; en \; Int(a_j)}{\#c_i}$ y la $P(c_i \mid a_1, \ldots, a_m) \propto P(c_i) \,.\, \prod_{j=1}^{m} P(a_j \mid c_i)$. Finalmente, se asigna la clase con la mayor probabilidad.

Al aumentar la cantidad de bins, se suele obtener una mayor precisión, pero se corre el riesgo de tener intervalos sin datos. Una opción para tratar esto consiste en hacer variable el tamaño de las particiones, siendo más pequeñas las más cercanas al centro del conjunto de datos.



Ventajas y problemas al utilizar Naive Bayes con histogramas.

Ventajas:

- Se puede utilizar cuando no se conoce una distribución de probabilidad que se ajuste a los datos
- Es más fácil calcular las verosimilitudes
- Permite introducir nuevas instancias fácilmente, ya que no hay que recalcular parámetros como la media o la varianza.
- Se puede reducir el error variando la cantidad y tamaño de los bins

Desventajas:

- Tener bins vacíos implica que una nueva instancia con valores que “caigan” allí no se podrá clasificar, por lo que se requiere de una estrategia para tratarlos
- No es tan sencillo determinar la cantidad de bins correctos.

¿Cuál es la cantidad de bins correcta?

La cantidad de bins correcta dependerá del problema en cuestión. En general, se debe determinar evaluando el error de las clasificaciones y ajustando dicha cantidad con el propósito de reducir este.

5. Redes de creencias bayesinas

Las redes de creencias bayesianas especifican, para un conjunto de variables, aquellas que se asumen condicionalmente independientes y otras descritas por conjunto de probabilidades condicionales. Es decir, en lugar de asumir que todas las variables son condicionalmente independientes, permite especificar subconjuntos que lo son. De esta manera, es un punto medio entre la restrictividad que implica asumir que todas son condicionalmente independientes, y la intratabilidad de asumir que ninguna lo es.

Definir cuales son las condiciones que deben darse para poder aplicar Naive Bayes.

El teorema de Bayes se define bajo tres condiciones:

- El espacio muestral S está particionado en n particiones disjuntas
- El espacio muestral S es igual a la unión de todas las particiones
- La probabilidad de cada partición es mayor a 0

Además, se debe poder suponer que los atributos o causas son condicionalmente independientes.

Diferencias entre Naive Bayes con Distribución Gaussiana (o continua) e Histogramas.

Naive Bayes con distribuciones continuas se utiliza cuando los valores de los atributos responden a alguna distribución de probabilidad, utilizando la misma para calcular las verosimilitudes. Por otro lado, en Naive Bayes con histogramas no es necesario conocer la distribución de probabilidad, sino que se divide a los posibles valores de los atributos en intervalos o bins y se tratan estos como si fueran valores discretos para calcular las verosimilitudes.

Explique y ejemplifique cómo interactúan la MT, el MI y la BC en un sistema basado en probabilidades que utiliza el teorema de Bayes para inferir.

La BC contiene el conjunto de clases con sus probabilidades a priori y el conjunto de atributos con sus respectivos parámetros (en caso de usar distribuciones continuas) o verosimilitudes (en casos discretos o con histogramas). La MT contiene los valores de los atributos del

problema particular. El MI es el encargado de utilizar los conocimientos y los datos para calcular las probabilidades a posteriori y clasificar las instancias.

En Naive Bayes:

- **Importa la cantidad de instancias**

¿La probabilidad conjunta puede ser expresada como $P(s_1, s_2, \ldots, s_n) = \prod \sum P(s_i/e_j) \,.\, P(e_j)$**? Justifique**

Demostración de p(s1,s2,...,sn) = productoria de la sumatoria p(si/ej).p(ej)

$P(s_i) = P(s_i) \,.\, 1$

$$P(s_i) = P(s_i) \,.\, [P(e_1) + P(e_2) + \ldots + P(e_m)]$$

$$P(s_i) = P(s_i) \,.\, P(e_1) + P(s_i) \,.\, P(e_2) + \ldots + P(s_i) \,.\, P(e_n)$$

$$P(s_i) = P(s_i \ y \ e_1) + P(s_i \ y \ e_2) + \ldots + P(s_i \ y \ e_n)$$

Sabiendo que $P(s_i|e_j) = P(s_i \ y \ e_j) \,/\, P(e_j)$ se tiene:

$$P(s_i) = P(s_i|e_1).P(e_1) + P(s_i|e_2).P(e_2) + \ldots + P(s_i|e_m).P(e_m) = \sum_{j=1}^{m} P(s_i/e_j) \,.\, P(e_j)$$

$$P(s_1, s_2, \ldots, s_n) = P(s_1) \,.\, P(s_2) \,.\, \ldots\ldots P(s_n) = \prod_{i=1}^{n} \quad P(s_i) = \prod_{i=1}^{n} \sum_{j=1}^{m} P(s_i/e_j) \,.\, P(e_j)$$

Redes Neuronales - Perceptrón - MLP

1. Redes neuronales biológicas

La teoría y modelado de Redes Neuronales Artificiales (RNA) está inspirada en la estructura y funcionamiento de los sistemas nerviosos, donde la neurona es el elemento fundamental. La idea es que las neuronas, a partir de un estímulo del entorno, actúan en consecuencia, enviando o no una señal hacia las siguientes neuronas conectadas.

Las dendritas y el cuerpo celular reciben señales de entrada; el cuerpo celular las combina e integra y emite señales de salida que se propagan a través del axón, el cual se conecta con las dendritas de otras neuronas. Se utilizan 2 tipos de señales: la señal que se transmite a lo largo del axón es eléctrica, mientras que la señal que se transmite entre los terminales axónicos y la dendrita de la siguiente neurona, es química.

Existen dos tipos de sinapsis:

- Las sinapsis excitadoras, cuyos neurotransmisores facilitan la generación de impulsos a mayor velocidad.
- Las sinapsis inhibidoras, cuyos neurotransmisores tienden a estabilizar el potencial de la membrana, dificultando la emisión de impulsos.

La idea básicamente es que si se tiene una entrada que supera el umbral de disparo, se emite una señal hacia adelante. El procesamiento que hace la neurona es si va a transmitir o no la señal hacia adelante en función a la “información” de entrada.

2. Abstracción del modelo biológico

Las redes neuronales artificiales son modelos que intentan reproducir el comportamiento del cerebro: las señales que llegan a la sinapsis son las entradas a la neurona; éstas son ponderadas (atenuadas o amplificadas) a través de un parámetro, denominado peso, asociado a la sinapsis correspondiente. Estas señales de entrada pueden excitar a la neurona (sinapsis con peso positivo) o inhibirla (peso negativo). El efecto es la suma de las entradas ponderadas. Si la suma es igual o mayor que el umbral de la neurona, entonces la neurona se activa (da salida). Esta es una situación todo o nada; cada neurona se activa o no se activa. La facilidad de transmisión de señales se altera mediante la actividad del sistema nervioso. Esta habilidad de ajustar señales es un mecanismo de aprendizaje. Las funciones de umbral integran la energía de las señales de entrada en el espacio y en el tiempo.

A partir de las neuronas se pueden generar representaciones específicas, de tal forma que un estado conjunto de ellas puede significar una letra, un número o cualquier otro objeto.

3. Elementos de una red neuronal artificial

Cada neurona está caracterizada en cualquier instante por un valor numérico denominado valor o **estado de activación a(t)**. Los valores de activación pueden ser continuos (valor entre [0,1] o en el intervalo [-1,1], generalmente siguiendo una función sigmoidal) o discretos (en una visión simplificada, podemos decir que hay dos posibles estados, reposo y excitado)

Asociada a cada neurona, existe una **función de salida o transferencia f**, que transforma el actual estado de activación a(t) en una señal de salida y(t): y(t)=f(a(t)). Existen cuatro funciones de transferencia típicas que determinan distintos tipos de neuronas:

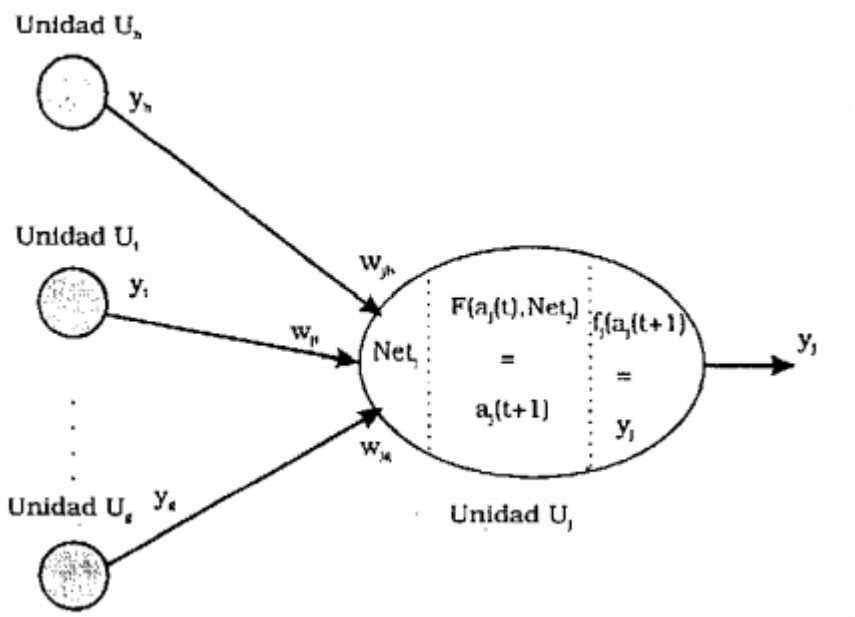
- Función escalón: se asocia a neuronas binarias en las cuales cuando la suma de las entradas es mayor o igual que el umbral de la neurona, la activación es 1; si es menor, la activación es 0 (o -1).
- Función lineal y mixta: la función lineal o identidad (f(x)=x) equivale a no aplicar función de salida y se usa muy poco. En las neuronas con función mixta, si la suma de las señales de entrada es menor que un límite inferior, la activación se define como 0 (o -1). Si dicha suma es mayor o igual que el límite superior, entonces la activación es 1. Si la suma de entrada está comprendida entre ambos límites, superior e inferior, entonces la activación se define como una función lineal de la suma de las señales de entrada.
- Función sigmoidal: para la mayoría de los valores del estímulo de entrada, el valor dado por la función es cercano a uno de los valores asintóticos. Esto hace que se puedan utilizar las reglas de aprendizaje definidas para las funciones escalón, con la ventaja de que la derivada está definida en todo el intervalo.
- Función gaussiana: los centros y anchura de estas funciones pueden ser adaptados, lo cual las hace más adaptativas que las funciones sigmoidales. Mapeos que suelen requerir dos niveles ocultos utilizando funciones sigmoidales algunas veces se pueden realizar con un solo nivel en redes con neuronas de función gaussiana.

La señal de salida es enviada a través de los canales de comunicación unidireccionales a otras unidades de la red; en estos canales la señal se modifica de acuerdo con el **peso** w asociado a cada uno de ellos según una determinada regla. Estos valores que aparecen en las conexiones indican la fuerza de las mismas. El proceso de aprendizaje se da en la modificación de los pesos en esas conexiones para resolver un problema particular. Estos pesos son los que tienen el conocimiento del dominio. Las señales moduladas que han llegado a la unidad j-ésima se combinan entre ellas, generando así la entrada total Net. $Net_j = \sum_i \quad y_i w_{ji}$ Esta regla para combinar los valores de entrada a una unidad con los pesos de las conexiones que llegan a esa unidad es conocida como **regla de propagación**.

Así como es necesario una regla que combine las entradas a una neurona con los pesos de las conexiones, también se requiere una regla que combine las entradas con el estado actual de la neurona para producir un nuevo estado de activación. Una **función de activación F** determina el nuevo estado de activación a(t+1) de la neurona teniendo en cuenta la entrada total calculada Net y el anterior estado de activación a(t): $a_i(t+1) = F(a_i(t), Net_i)$
En la mayoría de los casos F es la función identidad, por lo que el estado de activación de una neurona en t+1 coincidirá con el Net de la misma en t: $y_i(t+1) = f(Net_i)$ Además, normalmente la función de activación no está centrada en el origen del eje que representa el valor de la entrada neta sino que se debe considerar el umbral de activación θ: $y_i(t+1) = f(Net_i - \theta_i)$. Para simplificar la expresión de la salida de una neurona i es habitual considerar la existencia de una neurona ficticia asociada a la entrada de cada neurona i con valor de salida unidad mediante una conexión con peso de valor -θ.

La dinámica que rige la actualización de los estados de las unidades (evolución de la red neuronal) puede ser de tipo asíncrono (las neuronas evalúan su estado continuamente. según les va llegando información, y lo hacen de forma independiente) o síncrono (la información

también llega de forma continua, pero los cambios se realizan simultáneamente, como si existiera un reloj interno que decidiera cuándo deben cambiar su estado).



Generalmente, se pueden encontrar tres tipos de neuronas:

- **Neuronas de entrada**: reciben a partir de estímulos externos la información proveniente del exterior de la red.
- **Neuronas ocultas**: la información de entrada se transmite a ciertos elementos internos que se ocupan de su procesamiento. Es en las sinapsis y neuronas correspondientes a este segundo nivel donde se genera cualquier tipo de representación interna de la información. Se llaman ocultas porque no tienen relación directa con la información de entrada ni con la de salida.
- **Neuronas de salida**: una vez ha finalizado el procesamiento, la información llega a las unidades de salida, cuya misión es dar la respuesta del sistema

Se conoce como **capa o nivel** a un conjunto de neuronas cuyas entradas provienen de la misma fuente (que puede ser otra capa de neuronas) y cuyas salidas se dirigen al mismo destino (que puede ser otra capa de neuronas). Se dice que una red es **totalmente conectada** si todas las salidas desde un nivel llegan a todos y cada uno de los nodos del nivel siguiente.

La conectividad está relacionada con la forma en que las salidas de las neuronas están canalizadas para convertirse en entradas de otras neuronas (pudiendo ser una entrada de sí misma -conexión autorrecurrente-):

- Cuando ninguna salida de las neuronas es entrada de neuronas del mismo nivel (o uno anterior), la red se conoce como **red de propagación hacia adelante**.
- Cuando las salidas pueden ser conectadas como entradas a otras de niveles anteriores (o del mismo nivel), la red se conoce como **red de propagación hacia atrás**. Las redes de propagación hacia atrás que tienen lazos cerrados son sistemas recurrentes.

4. RNA - Características

4.1 Topología

La topología o arquitectura de las redes neuronales consiste en la organización y disposición de las neuronas en la red formando capas. Los parámetros fundamentales son:

- Número de capas
- Número de neuronas por capa
- Grado de conectividad
- Tipos de conexiones entre neuronas

Al clasificar las redes en términos topológicos se suele distinguir entre:

- **Redes monocapa**: se establecen conexiones laterales entre las neuronas que pertenecen a la única capa que constituye la red. También pueden existir conexiones autorrecurrentes (salida de una neurona conectada a su propia entrada). Se utilizan típicamente en tareas relacionadas con lo que se conoce como autoasociación; por ejemplo, para regenerar informaciones de entrada que se presentan a la red incompletas o distorsionadas.
- **Redes multicapa**: normalmente todas las neuronas de una capa reciben señales de entrada de otra capa anterior, más cercana a las entrada de la red, y envían las señales de salida a una capa posterior, más cercana a la salida de la red. A estas conexiones se les denomina **conexiones hacia adelante o feedforward**: no existen conexiones hacia atrás y normalmente tampoco autorrecurrentes ni laterales. Son especialmente útiles en aplicaciones de reconocimiento o clasificación de patrones. Sin embargo, en un gran número de redes multicapa existe la posibilidad de conectar las salidas de las neuronas de capas posteriores a las entradas de las capas anteriores, á estas conexiones se les denomina **conexiones hacia atrás o feedback**: en este tipo de redes circula información tanto hacia adelante como hacia atrás, existiendo por tanto dos conjuntos de pesos. Suelen ser bicapa, existen algunas que tienen conexiones laterales, y algunas redes de este tipo tienen un funcionamiento basado en resonancia, de tal forma que las informaciones en la primera y segunda capas interactúan entre sí hasta que alcanzan un estado estable.

4.2 Mecanismo de aprendizaje

El aprendizaje es el proceso por el cual una red neuronal modifica sus pesos en respuesta a una información de entrada. Los cambios que se producen durante el proceso de aprendizaje se reducen a la destrucción, modificación y creación de conexiones entre las neuronas: la creación de una nueva conexión implica que el peso de la misma pasa a tener un valor distinto de cero, y. una conexión se destruye cuando su peso pasa a ser cero.

Los criterios que se siguen para cambiar el valor asignado a las conexiones determinan lo que se conoce como la **regla de aprendizaje de la red**.

De acuerdo a si la red puede aprender durante su funcionamiento habitual o si el aprendizaje supone la desconexión de la red; es decir su inhabilitación hasta que el proceso termine, se trata de un aprendizaje ON LINE (los pesos varían dinámicamente siempre que se presente una nueva información) o un aprendizaje OFF LINE (los pesos de las conexiones permanecen fijos después de que termina la etapa de entrenamiento de la red) respectivamente.

Por otro lado, de acuerdo a la existencia o no de un agente externo (supervisor) que controle el proceso de aprendizaje, se pueden clasificar en:

- **Redes neuronales con aprendizaje supervisado**: el proceso de aprendizaje se realiza con un supervisor que comprueba la salida de la red y, en el caso de que ésta no coincida con la deseada, se procede a modificar los pesos de las conexiones para que la salida obtenida se aproxime a la deseada.
    - **Aprendizaje por corrección de error**: consiste en ajustar los pesos de las conexiones de la red en función del error cometido en la salida. Una regla simple puede ser: $\Delta w_{ji} = \alpha\, y_i (d_j - y_j)$
      Siendo Δw_{ji}la variación del peso en la conexión entre i y j, □ el factor de aprendizaje entre 0 y 1 que regula la velocidad del aprendizaje, y_i el valor de salida de la neurona i, d_j el valor de salida deseado de la neurona j, $y_j$el valor de salida obtenido de la neurona j.
      Por otro lado tenemos la regla delta o regla del mínimo error cuadrado, que en lugar de considerar los errores individuales, permite cuantificar el error global cometido en cualquier momento: $Error_{global} = \frac{1}{2P}\sum_{k=1}^{P} \sum_{j=1}^{N} (y_j{}^{(k)} - d_j{}^{(k)})^2$
      Siendo N el número de neuronas de salida, P el número de informaciones que debe aprender la red y $\sum_{j=1}^{N} (y_j{}^{(k)} - d_j{}^{(k)})^2$ el error cometido en el aprendizaje de la información k-ésima. Luego el ajuste de los pesos de las conexiones se puede hacer con $\Delta w_{ji} = k \frac{\partial Error_{global}}{\partial w_{ji}}$
    - **Aprendizaje por refuerzo**: se basa en la idea de no disponer de un ejemplo completo del comportamiento deseado, es decir, de no indicar durante el entrenamiento exactamente la salida que se desea que proporcione la red ame una determinada entrada. La función del supervisor se reduce a indicar mediante una señal de refuerzo si la salida obtenida en la red se ajusta a la deseada (éxito=+1 o fracaso=0), y en función de ello se ajustan los pesos basándose en un mecanismo de probabilidades.
    - **Aprendizaje estocástico**: consiste en realizar cambios aleatorios en los valores de los pesos de las conexiones de la red y evaluar su efecto a partir del objetivo deseado y de distribuciones de probabilidad: si el comportamiento de la red se acerca al deseado, se acepta el cambio, sino, se acepta el cambio o no en función de una determinada y preestablecida distribución de probabilidades.
- **Redes neuronales con aprendizaje no supervisado**: la red no recibe ninguna información por parte del entorno que le indique si la salida generada en respuesta a una determinada entrada es o no correcta; por ello. suele decirse que estas redes son capaces de autoorganizarse. En algunos casos, la salida representa el grado de familiaridad o similitud entre la información que se le está presentando en la entrada y las informaciones que se le han mostrado hasta entonces (en el pasado). En otro caso, podría realizar un clustering o establecimiento de categorías, indicando a qué categoría pertenece la información presentada a la entrada. También el aprendizaje sin supervisión permite realizar una codificación, generando a la salida una versión codificada de la entrada, o un mapeo de características obteniéndose en la neuronas

de salida una disposición geométrica que representa un mapa topográfico de las características de los datos de entrada.

- **Aprendizaje hebbiano**: consiste en el ajuste de los pesos de las conexiones de acuerdo con la correlación (multiplicación en el caso de valores binarios +1 y -1) de los valores de activación (salidas) de las dos neuronas conectadas, de manera que si las dos unidades son activas (positivas), se produce un reforzamiento de la conexión. Por el contrario, cuando una es activa y la otra pasiva (negativa), se produce un debilitamiento de la conexión: $\Delta w_{ij} = y_i \, . y_j$
- **Aprendizaje competitivo y cooperativo**: las neuronas compiten unas con otras con el fin de llevar a cabo una tarea dada; se pretende que cuando se presente a la red cierta información de entrada, sólo una de las neuronas de salida de la red (o una por cierto grupo de neuronas) se active (alcance su valor de respuesta máximo). Por tanto. las neuronas compiten por activarse, quedando finalmente una (o una por grupo) como neurona vencedora, quedando anuladas el resto, que son forzadas a sus valores de respuesta mínimos. El objetivo de este aprendizaje es categorizar los datos que se introducen en la red. En este tipo de redes cada neurona tiene asignado un peso total, suma de todos los pesos de las conexiones que tiene a su entrada. El aprendizaje afecta sólo a las neuronas ganadoras (activas), redistribuyendo este peso total entre sus conexiones, sustrayendo una porción a los pesos de todas las conexiones que llegan a la neurona vencedora y repartiendo esta cantidad por igual entre todas las conexiones procedentes de unidades activas. Por tanto, la variación del peso de una conexión entre una unidad i y otra j será nula si la neurona j no recibe excitación por parte de la neurona i (no vence en presencia de un estímulo por parte de i), y se modificará (se reforzará) si es excitada por dicha neurona i.

4.3 Tipo de asociación entre las informaciones de entrada y salida

Las redes neuronales se comportan como lo que habitualmente se conoce por memoria asociativa: cuando se aplica un estímulo (dato de entrada) la red responde con una salida asociada a dicha información de entrada. Existen dos formas primarias de realizar esta asociación entre entrada/salida:

- **Redes heteroasociativas**: la red aprende parejas de datos de tal forma que cuando se presente cierta información de entrada A, deberá responder generando la correspondiente salida asociada B. Precisan al menos de 2 capas, una para captar y retener la información de entrada y otra para mantener la salida con la información asociada. Existen redes de este tipo con conexiones hacia adelante, hacia atrás y laterales, con aprendizaje supervisado y no supervisado.
- **Redes autoasociativas**: la red aprende ciertas informaciones de tal forma que cuando se le presenta una información de entrada realizará una autocorrelación, respondiendo con uno de los datos almacenados, el más parecido al de entrada. Su principal misión es reconstruir una determinada información de entrada que se presenta incompleta o distorsionada (le asocia el dato almacenado más parecido). Pueden implementarse con una sola capa de neuronas: esta capa comenzará reteniendo la información inicial presentada a la entrada, y terminará representando la información autoasociada. Presentan conexiones laterales y, en algunos casos,

autorrecurrentes; habitualmente el aprendizaje es no supervisado pero existen algunas con aprendizaje supervisado.

Una red heteroasociativa puede siempre ser reducida a una autoasociativa mediante la concatenación de una información de entrada y su salida (respuesta) asociada, para obtener la información de entrada de la red autoasociativa equivalente. También puede conseguirse que una red autoasociativa se comporte como una heteroasociativa, simplemente presentando, como entrada parcial de la autoasociativa, la información de entrada para la heteroasociativa y haciendo que la red complete la información para producir lo que sería la salida de la red heteroasociativa equivalente.

4.4 Representación de la información de entrada y salida

- **Redes continuas**: tanto los datos de entrada como de salida son de naturaleza analógica, es decir, son valores reales continuos. Las funciones de activación de las neuronas son también continuas, del tipo lineal o sigmoidal.
- **Redes híbridas**: las informaciones de entrada pueden ser valores continuos, aunque las salidas de la red son discretas.
- **Redes discretas**: admiten y producen sólo valores discretos o binarios. Las funciones de activación de las neuronas son del tipo escalón.

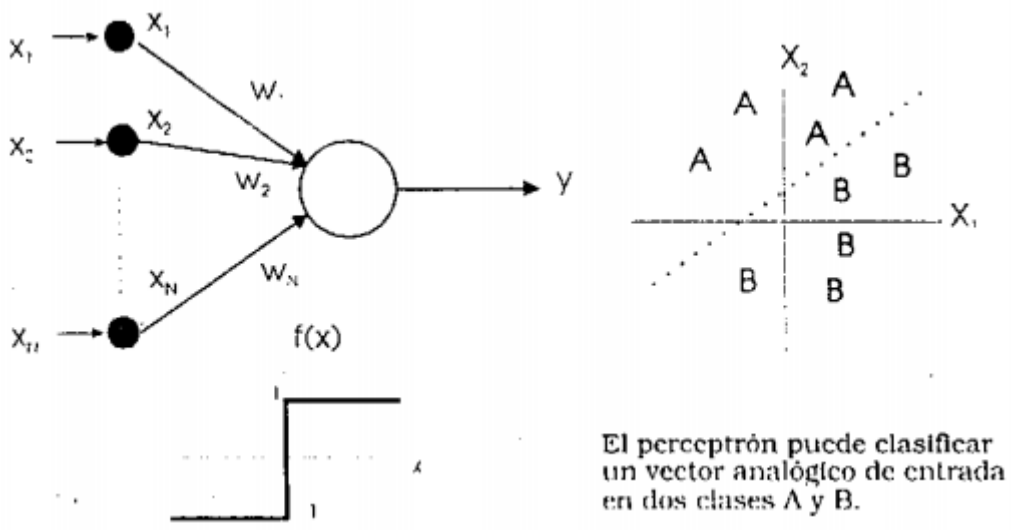
5. Perceptrón simple

Perceptrón: funcionamiento y limitaciones. ¿Cual es la condición para que pueda hacer reconocimientos de forma correcta?

Fue el primer modelo de red neuronal artificial desarrollado en 1958. Cuenta con N entradas, N conexiones, N pesos, 1 entrada ficticia que vale siempre 1 y tiene peso -θ (que es el umbral) y 1 neurona de salida. Cuándo una entrada se presenta a la red, es capaz de decidir a cuál de las dos clases que puede reconocer, pertenece dicha entrada.

La única neurona del Perceptrón realiza la suma ponderada de las entradas, resta el umbral y pasa el resultado a una función de transferencia de tipo escalón. La regla de decisión es responder +1 si el patrón presentado pertenece a la clase A, o -1 si el patrón pertenece a la clase B. La salida dependerá de la entrada neta y del valor del umbral θ.

Es un clasificador lineal, que distingue solamente 2 clases. Solo es capaz de clasificar correctamente patrones muy sencillos, linealmente separables, es decir, cuyas clases pueden separarse por un hiperplano (en será 2D será una recta, en 3D un plano)



El perceptrón puede clasificar un vector analógico de entrada en dos clases A y B.

5.1 Regla de aprendizaje del Perceptrón

Explique el algoritmo de aprendizaje del Perceptrón.

El algoritmo de aprendizaje del Perceptrón es de tipo supervisado (aprendizaje por corrección de error):

1. Inicialización de los pesos y del umbral: se asignan valores aleatorios a cada uno de los pesos (w_i) de las conexiones y al umbral ($-w_0 = \theta$).
2. Presentación de un nuevo par (Entrada, Salida deseada): se presenta un nuevo patrón de entrada $X_p = (x_1, x_2, \ldots, x_n)$junto con la salida esperada d(t).
3. Cálculo de la salida actual: $y(t) = f\ [\sum_i \quad w_i(t)\, x_i(t) - \theta]$ siendo f(x) la función de transferencia escalón.
4. Adaptación de los pesos: $w_i(t+1) = w_i(t) + \alpha[d(t) - y(t)]\, x_i(t)$ donde d(t) representa la salida deseada, y será 1 si el patrón pertenece a la clase A, y -1 si es de la clase B. □ es el factor de aprendizaje (rango (0, 1]) que debe ser ajustado de forma que satisfaga tanto los requerimientos de aprendizaje rápido como la estabilidad de las estimaciones de los pesos (es el porcentaje de error que se suma o resta a los pesos en la actualización).
5. Volver al paso 2

Este proceso se repite hasta que el error que se produce para cada uno de los patrones (diferencia entre el valor de salida deseado y obtenido) es cero o bien menor que un valor preestablecido, es decir, cuando se produce una época sin errores (una época es la presentación de todos los patrones de entrada-salida). Obsérvese que los pesos no se cambian si la red ha tomado la decisión correcta.

Resolución gráfica/analítica

Explique cómo se determinan analíticamente los pesos en Perceptrón.

Dada una recta cualquiera que separa correctamente las clases, se toman los 2 puntos de ella que cortan los ejes de abscisas y ordenadas (por ejemplo, $P1: x_1 = 1, x_2 = 0$; $P2: x_1 = 0, x_2 = 1$), y se reemplazan en la siguiente ecuación (ecuación de la recta) los valores de x1 y x2 de un punto para obtener w1, y luego los valores del otro punto junto con w1 para obtener w2. Luego se debe verificar que se clasifica correctamente todos los patrones. Si para todos los patrones produce error, se debe cambiar el signo de θ.

$x_0 w_0 + x_1 w_1 + x_2 w_2 = 0 \quad \rightarrow \quad 1.(-\theta) + x_1 w_1 + x_2 w_2 = 0 \quad \rightarrow \quad x_1 w_1 + x_2 w_2 = \theta \quad \rightarrow \quad x_2 = -\frac{w_1}{w_2} x_1 + \frac{\theta}{w_2}$

Un Perceptrón con función escalón [0,1] y un umbral θ = -3. Se tiene una entrada ficticia con –w0=θ. ¿Es correcta la Fnet?

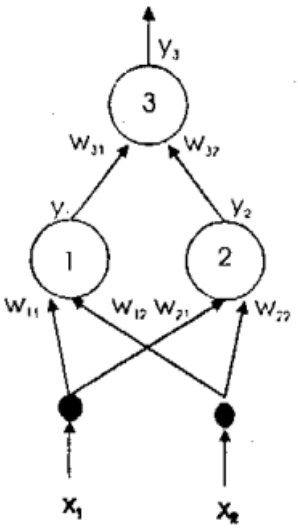
$$Fnet \begin{cases} 1 & \text{si net} > -3 \\ 0 & \text{en otro caso} \end{cases}$$

No, no es correcta, ya que si se tiene la entrada ficticia, la comparación de net se debe hacer con cero porque el umbral ya está incluido en el cálculo de net.

5.2 Solución al problema de la separabilidad lineal

Para la función XOR el perceptrón simple no puede realizar la correcta clasificación debido a que no existe ninguna recta que separe los patrones de una clase de los de la otra.

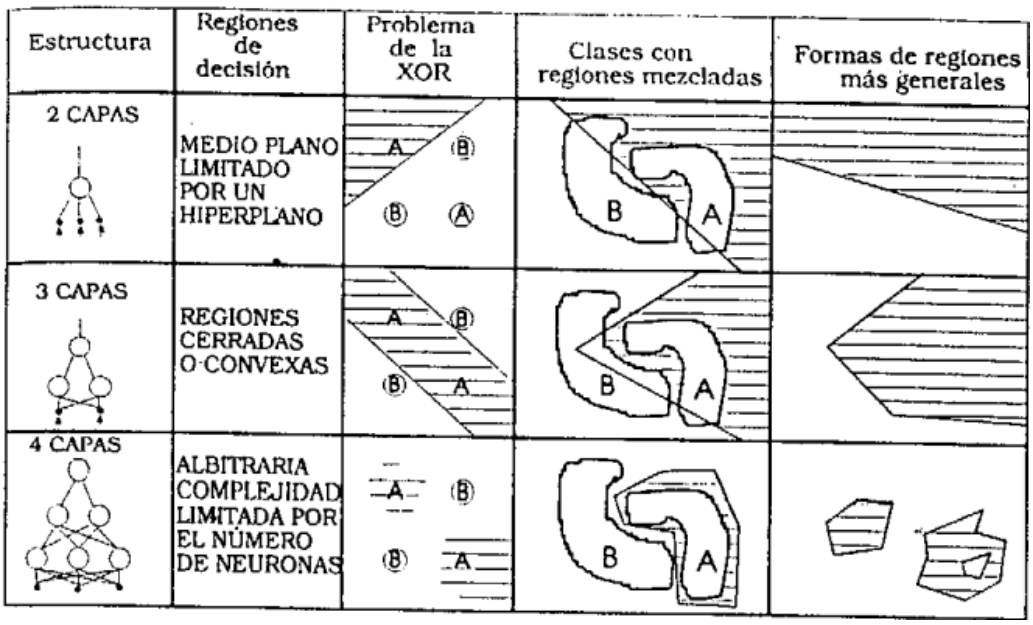
La solución podría darse si descompusiéramos el espacio en tres regiones: una región pertenecería a una de las clases de salida y las otras dos pertenecerían a la segunda clase. Si en lugar de utilizar únicamente una neurona de salida se utilizaran dos, se obtendrían dos rectas, por lo que podrían delimitarse tres zonas. Para poder elegir entre una zona u otra de las tres, es necesario utilizar otra capa con una neurona cuyas entradas serán las salidas de las neuronas anteriores. Por lo tanto, se ha de utilizar una red de tres neuronas, distribuidas en dos capas, para solucionar el problema de la función XOR.



Se tienen que ajustar entonces seis pesos. En el caso de los pesos de las conexiones de la capa de salida, el ajuste de los pesos se realiza de forma idéntica a la estudiada anteriormente, ya que conocemos la salida deseada. Sin embargo, no se tiene por qué conocer la salida deseada de las neuronas de la capa oculta, por lo que el método no es aplicable.

6. Perceptrón multicapa

Es una red de tipo feedforward compuesta de varias capas de neuronas entre la entrada y la salida de la misma. Esta red permite establecer regiones de decisión mucho más complejas que las de dos semiplanos, como hacía el Perceptrón.



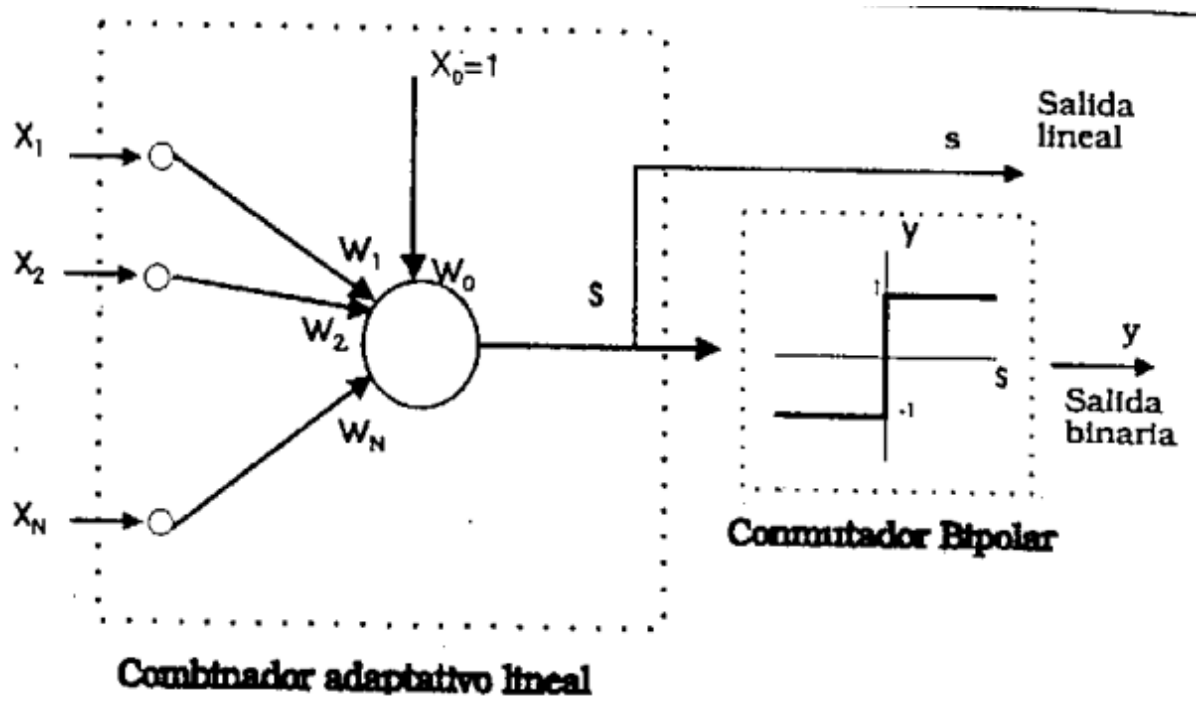
No se requieren más de cuatro capas en una red de tipo Perceptrón: una red con cuatro niveles, puede generar regiones de decisión arbitrariamente complejas.

Un problema que se debe resolver es la selección del número de neuronas ocultas: el número de nodos de la 3º capa debe ser >1 cuando las regiones de decisión están desconectadas o endentadas y no se pueden formar con una región convexa. Este número, en el peor de los casos, es igual al número de regiones desconectadas en las distribuciones de entrada. El número de neuronas en la 2º capa normalmente debe ser suficiente para proveer tres o más ángulos por cada área convexa generada por cada neurona de la 3º capa. Así, deberá de haber más de tres veces el número de neuronas de la 3* capa (N2>3N3). En la práctica, un número de neuronas excesivo en cualquier capa puede generar ruido. Por otro lado, si existe un número de neuronas redundantes se obtiene mayor tolerancia a fallos.

Diferencias entre ADALINE y Perceptrón.

La red ADALINE (ADAptive LINear Element) tiene una arquitectura esencialmente igual a la del perceptrón: está limitada a una única neurona de salida, mientras que MADALINE puede tener varias. ADALINE tiene un combinador adaptativo lineal (ALC), que obtiene una salida lineal (s). Esta salida lineal luego puede ser transformada por un conmutador bipolar para obtener una salida binaria (también se puede utilizar un conmutador sigmoidal).

La diferencia fundamental respecto al Perceptrón se refiere al mecanismo de aprendizaje: utilizan un aprendizaje OFF LINE con supervisión, la denominada regla Delta o regla del mínimo error cuadrado medio (LMS), basada en la búsqueda del mínimo de una expresión del error entre la salida deseada y la salida lineal obtenida antes de aplicarle la función escalón.



6.1 Aprendizaje de la red ADALINE

LMS (Adaline). Explicación y fórmulas.

La regla de aprendizaje LMS minimiza el error cuadrado medio definido como:

$$< {\varepsilon_k}^2 > = \frac{1}{2L} \sum_{k=1}^{L} {\varepsilon_k}^2$$

donde L es el número de vectores de entrada (patrones) que forman el conjunto de entrenamiento, y ε la diferencia entre la salida deseada y la obtenida cuando se introduce el patrón k-ésimo, que, en el caso de la red ADALINE se expresa como $\varepsilon = d_k - s_k$ siendo sk la salida del ALC.

La función de error es una superficie que tendrá muchos mínimos locales, y la regla de aprendizaje va a buscar el punto en el espacio de pesos donde se encuentra el mínimo global de esta superficie. Aunque la superficie de error es desconocida, el método de gradiente decreciente (máxima pendiente negativa) consigue obtener información local de dicha superficie. Con esa información se decide qué dirección tomar para llegar hasta el mínimo global de la superficie de error.

De esta manera, los pesos se modifican de tal manera que se encuentra un nuevo punto en el espacio de pesos más próximo al mínimo global de la superficie de error.

Se deriva entonces la función error con respecto a los pesos:

$$\Delta w_j = -\alpha \frac{\partial < \varepsilon_k^2 >}{\partial w_j} = -\alpha \frac{\partial < \varepsilon_k^2 >}{\partial s_k} \frac{\partial s_k}{\partial w_j}$$

$$= -\alpha \frac{\partial [\frac{1}{2}(d_k - s_k)^2]}{\partial s_k} \cdot \frac{\partial (\sum_{i=0}^{N} (w_i\, x_{ki}))}{\partial w_j} = (-\alpha)(-\varepsilon_k)\, x_{kj}$$

$$w_j(t+1) = w_j(t) + \alpha\, \varepsilon_k\, x_{kj} = w(t) + \alpha\, (d_k - s_k)\, x_{kj}$$

La única diferencia entre esta expresión y la del Perceptrón está el en valor del error ε_k, que en el caso del Perceptrón se refería a la diferencia entre el valor deseado y la salida binaria y, no la salida lineal s_k de la red ADALINE.

Explicar el proceso de aprendizaje en ADALINE.

El algoritmo de aprendizaje de una red ADALINE consta de los siguientes pasos:

1. Se presenta un patrón de entrada X_k en las entradas de ADALINE.
2. Se obtiene la salida lineal $s_k = \sum_{j=0}^{N} w_j\, x_{kj}$ y se calcula la diferencia con respecto a la deseada $\varepsilon_k = d_k - s_k$
3. Se actualizan los pesos: $w_j(t+1) = w_j(t) + \alpha\, \varepsilon_k\, x_{kj}$
4. Se repiten los pasos 1 a 3 con todos los vectores de entrada (L).
5. Si el error cuadrado medio: $< \varepsilon_k{}^2 > = \frac{1}{2L} \sum_{k=1}^{L} \varepsilon_k{}^2$ es un valor reducido aceptable, termina el proceso de aprendizaje. Sino, se repite otra vez desde el paso 1 con todos los patrones.

Para resolver un problema concreto, es necesario determinar una serie de aspectos como el número de vectores de entrenamiento necesarios, hallar la forma de generar la salida deseada para cada vector de entrenamiento, la dimensión óptima del vector de pesos, cuáles deberían ser los valores iniciales de los pesos, así como si es necesario o no un valor umbral, cuál debe ser el valor de □, o cuándo se debe finalizar el entrenamiento. En general, la solución de estas cuestiones depende del problema concreto que se pretenda resolver.

La dimensión del vector de pesos tiene una influencia directa en el tiempo necesario de entrenamiento, por lo que generalmente se debe tomar un compromiso entre este aspecto y la aceptabilidad de la solución (normalmente, se reduce el error aumentando el número de pesos). El valor del parámetro □ tiene una gran influencia sobre el entrenamiento: si □ es demasiado grande, la convergencia es posible que no se produzca, debido a que se darán saltos en torno al mínimo sin alcanzarlo; si □ es demasiado pequeño, alcanzaremos la convergencia, pero a costa de una etapa de aprendizaje más larga.

6.3 La red Backpropagation (BPN)

Explicar BPN

La red backpropagation (propagación del error hacia atrás), está basada en la regla delta generalizada, que es una extensión de la regla delta aplicable en redes con más de dos capas.

Emplea un ciclo propagación-adaptación de **dos fases**: primero se aplica un patrón de entrada como estímulo para la primera capa de neuronas de la red, se va propagando a través de todas las capas hasta generar una salida, se compara el resultado obtenido en las neuronas de salida con la salida que se desea obtener y se calcula un valor del error para cada neurona de salida. A continuación, estos errores se transmiten hacia atrás, partiendo de la capa de salida, hacia todas las neuronas de la capa intermedia que contribuyan directamente a la salida, recibiendo el porcentaje de error aproximado a la participación de la neurona intermedia en la salida original. Este proceso se repite, capa por capa, hasta que todas las neuronas de la red hayan recibido un error que describa su aporte relativo al error total. Basándose en el valor del error recibido, se reajustan los pesos de las conexiones de cada neurona, de manera que en la próxima vez que se presente el mismo patrón, la salida sea más cercana a la deseada.

Una característica importante que se exige a los sistemas de aprendizaje, es la capacidad de generalización, entendida como la facilidad de dar salidas satisfactorias a entradas que el sistema no ha visto nunca en su fase de entrenamiento. La red debe encontrar una representación interna que le permita generar las salidas deseadas cuando se le dan las entradas de entrenamiento, y que pueda aplicar, además, a entradas no presentadas durante la etapa de aprendizaje para clasificarlas según las características que compartan con los ejemplos de entrenamiento.

6.3.1 La regla delta generalizada

La regla delta ha sido extendida a redes con capas intermedias con conexiones hacia adelante (feedforward) y cuyas neuronas tienen funciones de activación continuas (lineales o sigmoidales), dando lugar al algoritmo de backpropagation. Estas funciones continuas son no decrecientes y derivables.

Explique la expresión de actualización de pesos entre 2 neuronas en BPN y describa cada componente de la misma considerando un término de momento. Describa cómo se actualizan los pesos en BPN desde i a j con momento. Explique cada término.

La modificación de los pesos es proporcional al gradiente decreciente de la función de error cuadrático medio y está dada por: $\Delta w_{ji} = \alpha\ \delta_{pj}\ y_{pi}$

Cuanto mayor es la tasa de aprendizaje α, mayor es la modificación de los pesos en cada iteración, con lo que el aprendizaje será más rápido, pero, por otro lado, puede dar lugar a oscilaciones. Para reducir las oscilaciones se puede añadir en la expresión del incremento de los pesos un término de **momento**, obteniéndose:

$$w_{ji}(t+1) = w_{ji}(t) + \alpha\ \delta_{pj}\ y_{pi} + \beta(w_{ji}(t) - w_{ji}(t-1))$$

β es una constante que acelera la convergencia de la red: si en t el incremento de un peso era positivo y en t+1 también, entonces el descenso por la superficie de error en t+1 es mayor. Sin embargo, si en t el incremento era positivo y en t+1 es negativo, el paso que se da en t+1 es más pequeño, ya que se ha pasado por un mínimo y los pasos deben ser menores para poder alcanzarlo.

δ_{pj} describe el aporte relativo de la neurona al error total, permitiendo la propagación del error hacia atrás, y se define como $\delta_{pj} = f'(net_{pj}) \cdot (d_{pj} - y_{pj})$ donde d_{pj} es la salida deseada de la neurona j para el patrón p y net_j es la entrada neta que recibe la neurona j. Si j no es una neurona de salida: $\delta_{pj} = f'(net_{pj}) \cdot (\sum_k \delta_{pk} w_{kj})$.

6.3.2 Estructura y aprendizaje de la red Backpropagation

En una red backpropagation existe una capa de entrada con n neuronas, una capa de salida con m neuronas y al menos una capa oculta de neuronas internas. Cada neurona de una capa (excepto las de entrada) recibe entradas de todas las neuronas de la capa anterior y envía su salida a todas las neuronas de la capa posterior (excepto las de salida), es decir, es una red completamente conectada. No hay conexiones hacia atrás ni laterales entre neuronas de la misma capa. Generalmente la función de activación utilizada es de tipo sigmoidal.

Aprendizaje en MLP. El algoritmo de entrenamiento de una red MLP.

El algoritmo de aprendizaje consta de los siguientes pasos:

1. Inicializar los pesos de la red con valores pequeños aleatorios.
2. Presentar un patrón de entrada X_p: $x_{p1}, x_{p2} \ldots, x_{pN}$ y especificar la salida deseada que debe generar la red: $d_1, d_2, \ldots, d_M$
3. Calcular la salida actual de la red: para ello se calcula la salida que presenta cada capa hasta llegar a la capa de salida; ésta será al salida de la red $y_1, y_2, \ldots, y_M$:
   a. Para cada neurona se calculan las entradas netas procedentes de la capa anterior: $net_{pj} = \sum_i w_{ji} x_{pi}$
   b. Se calculan las salidas de las neuronas: $y_{pj} = f_j(net_{pj}) = \frac{1}{1+e^{-net_{pj}}}$ si es una función sigmoidal, o $y_{pj} = net_{pj}$ si es lineal.
4. Calcular el error para todas las neuronas partiendo desde las de salida hacia atrás.
   a. Si la neurona j es de la capa de salida: $\delta_{pj} = f_j'(net_{pj}) \cdot (d_{pj} - y_{pj}) = y_{pj}(1 - y_{pj})(d_{pj} - y_{pj})$ si f es sigmoidal, o $\delta_{pj} = (d_{pj} - y_{pj})$ si es lineal.
   b. Si la neurona j no es de salida: $\delta_{pj} = f_j'(net_{pj}) \sum_k \delta_{pk} w_{kj} = y_{pj}(1 - y_{pj}) \sum_k \delta_{pk} w_{kj}$ si f es sigmoidal, o $\delta_{pj} = \sum_k \delta_{pk} w_{kj}$ si es lineal.
5. Actualizar los pesos: $w_{kj}(t+1) = w_{kj}(t) + \alpha\, \delta_{pk}\, y_{pj}$
6. El proceso se repite hasta que el término de error resulta aceptablemente pequeño. Se puede utilizar el error de patrón $E_p = \frac{1}{2}\sum_{k-1}^{M} (d_{pk} - y_{pk})^2$ debiendo ser el error de cada patrón menor al mismo. O utilizar el error global $E_g = \frac{1}{L}\sum_{p=1}^{L} E_p$ que promedia el error de todos los patrones.

6.3.3 Consideraciones sobre el algoritmo de aprendizaje

Normalmente α debe ser un número pequeño (del orden de 0,05 a 0,25) para asegurar que la red llegue a asentarse en una solución.

Con el gradiente descendente, siempre que se realiza un cambio en todos los pesos de la red, se asegura el descenso por la superficie del error hasta encontrar el valle más cercano, lo que puede hacer que el proceso de aprendizaje se detenga en un mínimo local de error.

No se asegura en ningún momento que el mínimo que se encuentre sea global. En todo caso, si la solución es admisible desde el punto de vista del error, no importa si el mínimo es local o global o si se ha detenido en algún momento previo a alcanzar un verdadero mínimo.

En la práctica, si una red deja de aprender antes de llegar a una solución aceptable, se realiza un cambio en el número de neuronas ocultas o en los parámetros de aprendizaje, o simplemente se vuelve a empezar con un conjunto distinto de pesos originales.

Respecto al número de capas de la red, en general tres capas son suficientes (entrada-ocultas-salida). Sin embargo, hay veces en que un problema es más fácil de resolver (la red aprende más deprisa) con más de una capa oculta. El tamaño de las capas, tanto de entrada como de salida, suele venir determinado por la naturaleza de la aplicación.

En cambio, decidir cuántas neuronas debe tener la capa oculta no suele ser tan evidente. No hay ninguna regla que indique el número óptimo. En cada problema se debe ensayar con distintos números de neuronas para organizar la representación interna y escoger el mejor. La idea más utilizada consiste en tener el menor número posible de neuronas en la capa oculta porque cada una de ellas supone una mayor carga de procesamiento.

Es posible eliminar neuronas ocultas si la red converge sin problemas. Si la red no converge, es posible que sea necesario aumentar este número. Por otro lado, examinando los valores de los pesos de las neuronas ocultas periódicamente en la fase de aprendizaje, se pueden detectar aquellas cuyos pesos cambian muy poco durante el aprendizaje respecto a sus valores iniciales, y reducir por lo tanto el número de neuronas que apenas participan en el proceso de aprendizaje.

La modificación de los pesos puede realizarse cada vez que un patrón ha sido presentado. o bien después de haber acumulado los cambios de los pesos en un número de iteraciones. El momento adecuado para cambiar los pesos depende de cada problema concreto.

6.4 Errores

Explicar el error del conjunto de entrenamiento, validación y test. Diferencia entre error de validación y error de entrenamiento. Error de validación y error de entrenamiento. Explique cómo se calculan teniendo en cuenta 100 patrones.

El conjunto de datos se divide en al menos dos partes: el conjunto de entrenamiento (que se utiliza para ajustar los pesos de la red) y el conjunto de validación (que se utiliza para evaluar la capacidad de generalización de la red). Además, se puede añadir un tercer conjunto, el de test. Se puede asignar, por ejemplo, 70% a entrenamiento, 20% a validación y 10% a test.

Durante el entrenamiento de la red, en cada iteración se presentan todos los patrones, tanto del conjunto de entrenamiento como del de validación, y se calcula para cada uno el error global (con el conjunto de validación solo se hace la propagación hacia adelante, sin actualizar los pesos).

El error mínimo aceptable de entrenamiento determina la finalización del proceso, significando que la red aprendió los patrones del conjunto de entrenamiento. Sin embargo, un valor muy pequeño puede conducir a un sobreajuste, es decir, que la red funcione bien con los datos

que se conocen pero se obtenga un mal resultado cuando la red se ponga en funcionamiento, con datos nunca vistos anteriormente.

Para evitar esto, se utiliza el conjunto de validación: si en una iteración determinada, comienza a aumentar el error de validación, se debe terminar con el proceso de entrenamiento (dicho momento se conoce como early-stop) ya que sino la red perderá capacidad de generalización.

Finalmente, el error de test es similar al de validación (no se utiliza para ajustar los pesos) pero no corta el entrenamiento sino que se utiliza solo para evaluar la diferencia de precisión entre datos conocidos y datos nunca vistos.

MLP:

- **Separa clases linealmente**
- **Se puede usar distintos tipos de función de transferencia en las capas**
- **Es aprendizaje offline**
- Ninguna de las anteriores.

Determine la arquitectura de una red BPN que aproxime la salida de la función sen ((x+y) / z). Detallar todas las consideraciones de diseño. Diseñar una red MLP que se aproxime a la función y= 2 *Sen ((x+y)/z)

Utilizaría 3 neuronas de entrada (x,y,z) y una neurona de salida con función de transferencia tangente hiperbólica debido a la imagen [-1,1] de la función seno, pre multiplicada por 2. La cantidad de capas y neuronas ocultas se determinarían mediante prueba y error. El valor alfa=0,15 (por ser el valor recomendado entre 0,05 y 0,25), con término de momento y β=0,1 para acelerar la convergencia.

Patrones de entrenamiento en el rango de x=[0, 32□], y=[0, 32□] z=(0, 100]

1024 valores diferentes de x (8 vueltas a la circunferencia con 128 divisiones cada vuelta)

1024 valores diferentes de y (8 vueltas a la circunferencia con 128 divisiones cada vuelta)

1000 valores diferentes de z (0,1 a 100,0)

Utilizaría el error global ya que interesa que tenga un funcionamiento general relativamente bueno, sin haber un riesgo de “seguridad” si un patrón no cumple el error mínimo.

Proponga una solución para una red MLP que finaliza el entrenamiento porque el error de validación crece antes de alcanzar el mínimo aceptable.

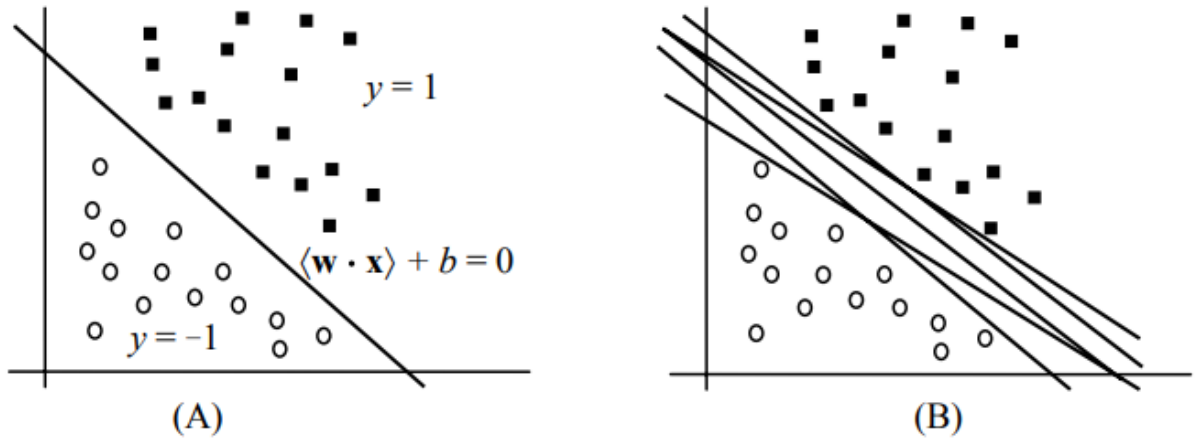
Máquinas de vectores de soporte

Las máquinas de vectores de soporte son otro tipo de mecanismo de aprendizaje que tiene muchas cualidades deseables que lo convierten en uno de los algoritmos más populares. No solo tiene una base teórica sólida, sino que también realiza la clasificación con mayor precisión que la mayoría de los otros algoritmos en muchas aplicaciones, especialmente aquellas aplicaciones que involucran datos de muy alta dimensión.

En general, SVM es un sistema de aprendizaje que construye clasificadores lineales de dos clases. Sea D el conjunto de ejemplos de entrenamiento $\{(\underline{x}_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$donde $\underline{x_i} = (x_{i1}, x_{i2}, \ldots, x_{ir})$es un vector de entrada r-dimensional en un espacio real X y $y_i$es una etiqueta de clase (valor de salida) ∈ {1,-1}.

Para construir un clasificador, SVM encuentra una función lineal de la forma $f(\underline{x}) = < \underline{w}.\underline{x} > +b$ de manera que el vector de entrada $\underline{x_i}$es asignado a una clase positiva si $f(\underline{x_i}) \geq 0$, y a la negativa en otro caso. El hiperplano que separa los ejemplos de entrenamiento positivos y negativos se llama límite o superficie de decisión y está dado por $< \underline{w}.\underline{x} > +b = 0$.

Entonces, $f(\underline{x})$ es una función real $f: X \subseteq R^r \rightarrow R$, $\underline{w} = (w_1, w_2, \ldots, w_r) \in R^r$es el vector de pesos, b ∈ R es la parcialidad (bias), $< \underline{w}.\underline{x} >$ es el producto escalar de $\underline{w}$ y $\underline{x}$, y la ecuación anterior, sin usar notación vectorial se puede expresar como:$f(x_1, x_2, \ldots, x_r) = w_1x_1 + w_2x_2 + \ldots + w_rx_r + b$.



Esta imagen da lugar a 2 preguntas:

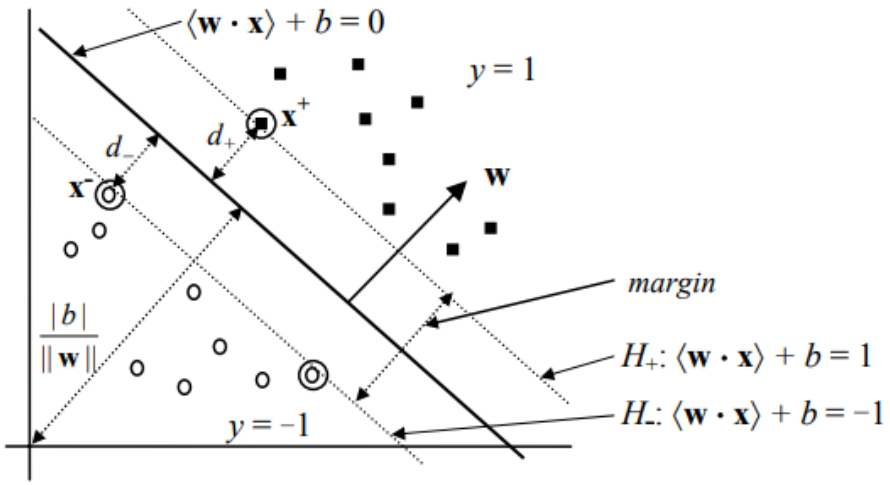
1. Hay un número infinito de líneas que pueden separar los puntos de datos positivos y negativos como se ilustra. ¿Qué línea deberíamos elegir?
2. Un clasificador de hiperplano solo es aplicable si los datos positivos y negativos pueden separarse linealmente. ¿Cómo podemos lidiar con separaciones no lineales o conjuntos de datos que requieren límites de decisión no lineales?

Con respecto a la pregunta 1, SVM elige el hiperplano que maximiza la separación entre los puntos positivos y negativos (hiperplano de máximo margen), minimizando así el riesgo de malas clasificaciones. Con respecto a la 2, SVM usa para ello funciones kernel (no se ven). Se debe mencionar además, que SVM requiere datos numéricos y solo construye clasificadores de dos clases.

1. SVM casos linealmente separables

Se asume que los puntos positivos y negativos son linealmente separables.

Dado $< \underline{w}.\underline{x} > +b = 0$, $\underline{w}$ define una perpendicular al hiperplano y se llama vector normal. Sin cambiar el vector normal $\underline{w}$, variando b se mueve el hiperplano de forma paralela.



SVM maximiza el margen de separación. Sea d_+ la distancia más corta desde el hiperplano de separación hasta el punto positivo más cercano (y d_- al negativo), el margen es $d_+ + d_-$.

Se definen dos hiperplanos $H_+ \, y \, H_-$ tales que $< \underline{w}.\underline{x_i} > +b \geq 1 \; si \; y_i = 1$y $< \underline{w}.\underline{x_i} > +b \leq -1 \; si \; y_i = -1$,de manera que ningún dato de entrenamiento cae entre dichos hiperplanos. Su distancia al hiperplano de separación $(d_+ \, o \, d_-)$ está dada por $\frac{|<\underline{w}.\underline{x_i}>+b|}{||\underline{w}||}$ donde $\left|\left|\underline{w}\right|\right| = \sqrt{{w_1}^2 + {w_2}^2 + \ldots + {w_r}^2}$.

Para calcular d_+, en lugar de calcular la distancia desde un punto sobre H+ $\underline{x^+}$ al hiperplano de separación $< \underline{w}.\underline{x} > +b = 0$, calculamos la distancia desde un punto x_s de ese hiperplano a H_+ $(< \underline{w}.\underline{x^+} > +b = 1)$. Teniendo en cuenta que $< \underline{w}.\underline{x_s} > +b = 0$:

$$d_+ = \frac{\left|<\underline{w}.\underline{x_s}>+b-1\right|}{\left|\left|\underline{w}\right|\right|} = \frac{1}{\left|\left|\underline{w}\right|\right|}$$

Por lo tanto: $margen = d_+ + d_- = \frac{2}{\left|\left|\underline{w}\right|\right|}$

Teniendo en cuenta que maximizar el margen es equivalente a minimizar $\frac{\left|\left|\underline{w}\right|\right|^2}{2}$tenemos la siguiente formulación de SVM linealmente separable:

Algoritmo de aprendizaje de SVM

Dado un conjunto de ejemplos de entrenamiento linealmente separables $D = \{(\underline{x_1}, y_1), (\underline{x_2}, y_2), \ldots, (\underline{x_n}, y_n)\}$el aprendizaje consiste en

Minimizar: $\frac{<\underline{w}.\underline{w}>}{2}$

Sujeto a: $y_i(<\underline{w}.\underline{x_i}>+b) \geq 1, i = 1,2,\ldots,n$ que incluye tanto a casos + como -

Para resolverlo se utiliza el método de los multiplicadores de Lagrange. El lagrangiano se forma sustrayendo de la función objetivo el multiplicador de lagrange multiplicado por la restricción:

$$L_P = \frac{1}{2} <\underline{w}.\underline{w}> - \sum_{i=1}^{n} \alpha_i [y_i(<\underline{w}.\underline{x_i}>+b) - 1]$$

Una solución óptima al problema debe satisfacer las condiciones de Kuhn-Tucker:

1) $\frac{\partial L_P}{\partial w_j} = w_j - \sum_{i=1}^{n} y_i \alpha_i x_{ij} = 0 \qquad j = 1,2,\ldots,r$
2) $\frac{\partial L_P}{\partial b} = -\sum_{i=1}^{n} y_i \alpha_i = 0$
3) $y_i(<\underline{w}.\underline{x_i}>+b) - 1 \geq 0 \qquad i = 1,2,\ldots,n$
4) $\alpha_i \geq 0, i = 1,2,\ldots,n$
5) $\alpha_i(y_i(<\underline{w}.\underline{x_i}>+b) - 1) = 0 \quad i = 1,2,\ldots,n$

La condición complementaria (5) indica que solo aquellos puntos que se encuentran sobre los hiperplanos H_+ y H_- tienen $\alpha_i > 0$. Estos puntos se conocen como **vectores de soporte** y son los que le dan el nombre al método.

En general, las condiciones de Kuhn-Tucker son necesarias pero no suficientes. Sin embargo, para nuestro problema, son suficientes para encontrar una solución óptima.

SVM: cómo se obtiene el dual. SVM fórmula dual y explicación de cada término

La formulación dual del problema es más fácil de resolver que el problema original (Lagrangiano primal). Para transformarlo se debe derivar parcialmente el primal con respecto a $\underline{w}$ y b, igualar dichas derivadas a 0 y utilizar esas relaciones para sustituir las variables $\underline{w}$ y b. Esto nos da la formulación dual del problema:

Maximizar: $L_D = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} y_i\, y_j\, \alpha_i\, \alpha_j\, <\underline{x_i}^T.\underline{x_j}>$
Sujeto a: $\sum_{i=1}^{n} y_i\, \alpha_i = 0 \qquad \alpha_i \geq 0, i = 1,2,\ldots,n$

Al resolverla, se obtienen los valores para α_i que se usan para calcular el vector $\underline{w}$ y el bias b usando las ecuaciones de las restricciones 1 y 5, respectivamente. En la práctica, en lugar de depender de un vector de soporte, para calcular b, se utilizan todos los vectores de soporte y luego se toma su promedio como el valor final.

Finalmente, el límite de decisión (hiperplano de máximo margen) es:
$<\underline{w}.\underline{x}>+b = \sum_{i\,\in\,sv} y_i\, \alpha_i <\underline{x_i}.\underline{x}>+b = 0$ donde sv es el conjunto de índices de los vectores de soporte en los datos de entrenamiento. Para clasificar una nueva instancia $\underline{z}$ se utiliza $signo(<\underline{w}^T.\underline{z}>+b)$.

2. SVM casos no linealmente separables

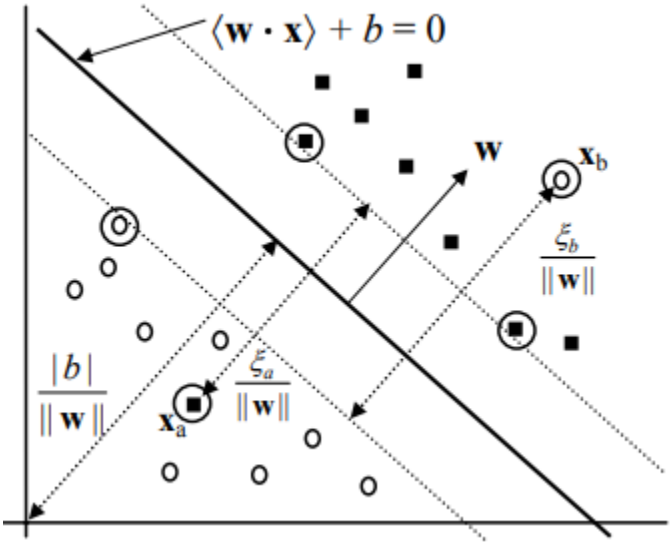
SVM caso no linealmente separable

Los datos de entrenamiento suelen contener ruido, por lo que SVM linealmente separable no podría encontrar una solución porque no se cumplirían las restricciones. Para permitir errores,

se pueden relajar dichas restricciones mediante variables de holgura $\xi_i \geq 0$. Además, necesitamos penalizar los errores en la función objetivo. Esto se puede hacer asignando un costo extra C por los errores.

Minimizar: $\frac{<\underline{w}.\underline{w}>}{2} + C\ (\sum_{i=1}^{n} \quad \xi_i)$ donde $C \geq 0$ es un parámetro especificado por el usuario

Sujeto a: $y_i(<\underline{w}.\underline{x_i}> + b) \geq 1 - \xi_i,\ \ y \quad \xi_i \geq 0 \qquad\qquad i = 1,2,\ldots,n,$



El problema dual está dado por:

Maximizar: $L_D(\alpha) = \sum_{i=1}^{n} \quad \alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} \quad y_i\ y_j\ \alpha_i\ \alpha_j\ \ <\underline{x_i}^T.\underline{x_j}>$

Sujeto a: $\sum_{i=1}^{n} \quad y_i\ \alpha_i = 0 \qquad\qquad 0 \leq \alpha_i \leq C, i = 1,2,\ldots,n$

La función objetivo es idéntica al caso linealmente separable y la única diferencia es la restricción $\alpha_i \leq C$.

Para calcular b se usa:

$b = \frac{1}{y_i} - \sum_{i=1}^{n} \quad y_i\ \alpha_i <\underline{x_i}^T.\underline{x_j}>$ siendo j el índice de algún vector de soporte

Finalmente, el límite de decisión es:

$$<\underline{w}.\underline{x}> + b = \sum_{i=1}^{n} \quad y_i\ \alpha_i <\underline{x_i}.\underline{x_j}> + b = 0$$

Se puede notar que no es necesario calcular explícitamente $\underline{w}$.

SVM: determinación de H+ y H-
¿Qué son y cómo se calculan H+ y H- en SVM?
Vectores de soporte SVM (H+ y H-). Cálculo y representación.
SVM: explicar cómo se obtienen los vectores de soporte una vez obtenida la expresión dual.
¿Para qué se utilizan los hiperplanos h+ y h- en máquinas de vectores de soporte? Explicación y gráfico

H+ y H- son los hiperplanos que pasan por los vectores de soporte (puntos de una clase más cercanos a los de la otra clase), limitando las clases del conjunto de entrenamiento, de

manera que entre ellos no hay ninguna instancia. Se utilizan para calcular la máxima distancia entre ellos y obtener así el hiperplano de separación que pasa por el medio de ambos.

$H+ = < \underline{w}.\underline{x} > +b = 1 \qquad H- = < \underline{w}.\underline{x} > +b = -1$

Para obtenerlos se desarrolla la sumatoria del dual a maximizar: $L_D(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} y_i\, y_j\, \alpha_i\, \alpha_j \; < \underline{x_i}^T . \underline{x_j} >$

Luego se hace el producto escalar de los $\underline{x_i}^T . \underline{x_j}$ y se obtiene un conjunto de términos donde las únicas variables son los multiplicadores de lagrange (α).

Se maximiza la expresión resultante sujeta a $\sum_{i=1}^{n} y_i\, \alpha_i = 0$, la cual también se desarrolla quedando en función a los α, y $\alpha_i \geq 0$. Se obtienen así los valores de los α que se utilizan para calcular b y w.

Cómo se obtienen b y w en SVM luego de encontrar los vectores de soporte.

b y w se obtienen despejándolas de las restricciones 1 y 5 del problema dual:

$$\frac{\partial L_p}{\partial \underline{w}} = \underline{w} - \sum_{i=1}^{N} y_i.\alpha_i . \underline{x_i} = 0 \Rightarrow \underline{w} = \sum_{i=1}^{N} y_i.\alpha_i . \underline{x_i}$$

$$y_i(\underline{w}^T.\ \underline{x_i} + b) - 1 = 0 \Rightarrow b = \frac{1}{y_i} - \underline{w}^T.\underline{x_i}$$

Luego se reemplazan en estas expresiones los valores de los multiplicadores de Lagrange obtenidos al maximizar el dual, los vectores de entrada x y sus etiquetas y.

El valor de b se calcula para todos los vectores y se utiliza el promedio de los resultados obtenidos.

Finalmente, con los valores de b y w, se obtiene hiperplano de separación: $< \underline{w}.\underline{x} > +b = 0$

SVM:

- **Los vectores de soporte tienen un valor de alfa > 0**
- **Ante cada instancia de entrenamiento nueva, hay que volver a recalcular todo**
- **Entre H+ y H- no hay datos de entrenamiento**

SVM:

- **Los vectores de soporte son aquellos cuyo alfa es distinto de 0**
- **Se recalcula el método cuando ingresa una nueva instancia de entrenamiento**
- **Entre H+ y H- no existen instancias de entrenamiento**
- Ninguna de las anteriores.

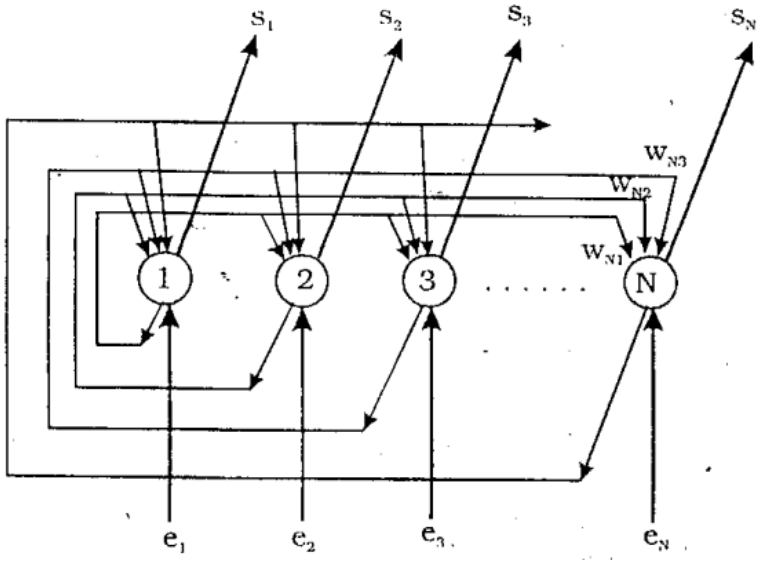
SVM:

- **Los vectores de soporte son aquellos cuyos multiplicadores de lagrange son mayores a 0**
- **Se recalcula el método cuando ingresa una nueva instancia de entrenamiento**
- **Se debe conocer la clase de la instancia antes de comenzar**
- Ninguna de las anteriores.

Modelo Hopfield

1. Arquitectura

El modelo de Hopfield consiste en una red **monocapa** con N neuronas cuyos valores de salida son binarios (-1/+1). Cada neurona de la red se encuentra conectada a todas las demás (conexiones laterales) pero no consigo misma (no existen conexiones autorrecurrentes). Además, los pesos asociados a las conexiones entre pares de neuronas son simétricos ($w_{ij} = w_{ji}$).



$$f(x) = \begin{cases} +1 & x>0, \\ -1 & x<0, \end{cases}$$

Cuando el valor de x coincide exactamente con θ, la salida de la neurona i permanece con su valor anterior. θ es el umbral de disparo de la neurona i que representa el desplazamiento de la función de transferencia a lo largo del eje de ordenadas. En el modelo de Hopfield discreto suele adoptarse un valor proporcional a la suma de los pesos de las conexiones de cada neurona con el resto:

$$\theta_i = k \sum_{j=1}^{N} w_{ji}$$

Si se trabaja con los valores binarios -1 y +1, suele considerarse el valor nulo para θ. Si los valores binarios son 0 y 1 se toma un valor de 1/2 para k.

En el caso de las redes de Hopfield continuas, se trabaja con valores reales en los rangos [-1,+1] (función tangente hiperbólica) o [0,1] (función sigmoidal).

2. Funcionamiento

Una de las características del modelo de Hopfield es que se trata de una **red autoasociativa**. Así, varias informaciones (patrones) diferentes pueden ser almacenadas en la red, como si de una memoria se tratase, durante la etapa de aprendizaje. Posteriormente, si se presenta

a la entrada alguna de las informaciones almacenadas, la red evoluciona hasta estabilizarse, ofreciendo entonces en la salida la información almacenada, que coincide con la presentada en la entrada. Si, por el contrario, la información de entrada no coincide con ninguna de las almacenadas. por estar distorsionada o incompleta, la red evoluciona generando como salida la más parecida.

La información que recibe esta red debe haber sido previamente codificada y representada en forma de vector (como una configuración binaria si la red es discreta, y como conjunto de valores reales si es continua) con tantas componentes como neuronas (N) tenga la red.

Esa información es aplicada directamente a la única capa de la que consta la red, siendo recibida por las neuronas de dicha capa (cada neurona recibe un elemento del vector que representa dicha información). Si consideramos en principio el caso de una neurona concreta de la red, esta neurona recibiría como entrada las salidas de cada una de las otras neuronas, valores que inicialmente coincidirán con los de entrada, multiplicadas por los pesos de las conexiones correspondientes. La suma de todos estos valores constituirá el valor de entrada neta de la neurona, al que le será aplicada la función de transferencia, obteniéndose el valor de salida correspondiente, 0/1 o -1/+1 si la red es discreta, y un número real en el rango [0,1] ó [-1,+1] si es continua.

La descripción anterior correspondería a un primer paso en el procesamiento realizado por la red. Este proceso continúa hasta que las salidas de las neuronas se estabilizan, lo cual ocurrirá cuando dejen de cambiar de valor. Entonces, el conjunto de estos (N) valores de salida de todas las neuronas constituye la información de salida que ha generado la red, que se corresponderá con alguna de las informaciones que durante la etapa de aprendizaje fueron almacenadas en la misma.

Este funcionamiento puede expresarse de la siguiente forma:

1. En el instante inicial (t=0) se aplica la información de entrada (valores $e_1, e_2, \ldots, e_N$). Inicialmente la salida de las neuronas coincide con la información aplicada a la entrada: $s_i(t=0) = e_i \quad i = 1..N$
2. La red realiza iteraciones hasta alcanzar la convergencia (hasta que $s_i(t+1) = s_i(t)$): $s_i(t+1) = f(\sum_{j=1}^{N} \quad w_{ij}\, s_j(t) - \theta_i) \quad i = 1..N$ donde f es la función de transferencia de las neuronas de la red.

Una vez que se ha estabilizado la red, la salida $(s_1, s_2, \ldots, s_n)$ representa la información almacenada por la red que más se parece a la información presentada en la entrada $(e_1, e_2, \ldots, e_n)$.

Si esta actualización se realiza de forma simultánea en todas las neuronas, se trata de una red Hopfield con funcionamiento paralelo o síncrono, ya que supone que todas las neuronas son capaces de operar sincronizadas y que, por tanto, la salida es generada al mismo tiempo por todas ellas en cada iteración, de tal forma que en la próxima iteración (t + 1) todas van a utilizar como entradas las salidas generadas por las otras en el instante anterior (t). Si, por el contrario, las neuronas trabajan de forma secuencial, actualizándose sólo la salida de una neurona en cada iteración, se tratará de una red Hopfield con funcionamiento secuencial o asíncrono. En este caso, ocurre que la salida a la que converge la red puede ser diferente en función del orden de la secuencia de activación de las neuronas.

También existe la posibilidad, sólo utilizada en las redes Hopfield continuas que trabajan con valores no binarios, de que la generación de la salida de todas las neuronas se realice de forma simultánea (síncrona) y continuada en el tiempo. En este caso. el funcionamiento de la red tendría que ser representado en forma de una ecuación diferencial:

$\tau_i \frac{ds_t}{dt} = f(\sum_{j=1}^{N} \quad w_{ij}\, s_j - \theta_i) - s_i$ donde τ_i es la tasa de retardo

3. Aprendizaje

La red Hopfield tiene un mecanismo de aprendizaje **offline**, por lo que existe una etapa de aprendizaje y otra de funcionamiento de la red. En la etapa de aprendizaje se fijan los valores de los pesos en función de las informaciones que se pretende que memorice o almacene la red. Una vez establecidos los pesos, la red entra en funcionamiento devolviendo la información almacenada más similar a la presentada en la entrada.

Modelo de Hopfield: explicar el algoritmo de aprendizaje

Esta red utiliza un aprendizaje **no supervisado** de **tipo hebbiano**, de tal forma que el peso de una conexión entre una neurona i y otra j se obtiene mediante el producto de las componentes i-ésima y j-ésima del vector que representa la información o patrón que debe almacenar. Si el número de patrones a aprender es M, el valor definitivo de cada uno de los pesos se obtiene mediante la suma de los M productos obtenidos por el procedimiento anterior, un producto por información a almacenar.

En el caso de la red Hopfield discreta, que trabaja con valores -1/+1 este algoritmo de aprendizaje puede expresarse de la siguiente forma:

$$w_{ij} = \begin{cases} \sum_{k=1}^{M} e_i^{(k)} e_j^{(k)} & 1 \le i,j \le N \;;\; i \neq j \\ 0 & 1 \le i,j \le N \;;\; i = j \end{cases}$$

Siendo:

- w_{ij}: peso asociado a la conexión entre la neurona j y la neurona i, que coincide con w_{ji}
- $e_i^{(k)}$: valor de la componente i-ésima del vector correspondiente a la información k-ésima que debe aprender la red.
- N: Número de neuronas de la red, y por tanto, tamaño de los vectores de aprendizaje.
- M: Número de informaciones que debe aprender la red.

Si la red trabajase con valores discretos 0/1, en lugar de -1/+1, entonces los pesos se calculan según la expresión:

$$w_{ij} = \begin{cases} \sum_{k=1}^{M} (2e_i^{(k)} - 1)(2e_j^{(k)} - 1) & 1 \le i,j \le N \;;\; i \neq j \\ 0 & 1 \le i,j \le N \;;\; i = j \end{cases}$$

El algoritmo de aprendizaje también se suele expresar utilizando una notación matricial. En tal caso se podría considerar una matriz W de dimensiones NxN que representase todos los pesos de la red:

$$W = \begin{bmatrix} w_{11} & w_{21} & w_{31} & \cdots & w_{N1} \\ w_{12} & w_{22} & w_{32} & \cdots & w_{N2} \\ & & \cdots & & \\ w_{1N} & w_{2N} & w_{3N} & \cdots & w_{NN} \end{bmatrix}$$

Esta matriz es simétrica, al cumplirse que $w_{ij} = w_{ji}$ y tiene una diagonal principal con valores nulos debido a la no existencia de conexiones auto-recurrentes ($w_{ii} = 0$).

También se tendría el conjunto de los M vectores que representan las informaciones que ha de aprender la red:

$$E_1 = [e_1^{(1)}, e_2^{(1)}, \ldots, e_N^{(1)}]$$
$$E_2 = [e_1^{(2)}, e_2^{(2)}, \ldots, e_N^{(2)}]$$
$$E_M = [e_1^{(M)}, e_2^{(M)}, \ldots, e_N^{(M)}]$$

Utilizando esta notación, el aprendizaje consistiría en la creación de la matriz de pesos W a partir de los M vectores o informaciones de entrada ($E_1, \ldots, E_M$) que se enseñan a la red:

$$W = \sum_{k=1}^{M} \quad [E_k^T \, E_k \, - \, I]$$

Luego, dada una entrada E, para obtener la salida correspondiente se hace E.W hasta que S(t+1) sea igual a S(t).

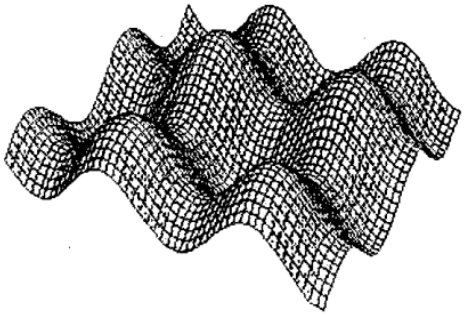
4. La función energía

La elección de la regla de aprendizaje hebbiano fue, entre otras razones, debido a que asegura la estabilidad de la red, es decir, la convergencia hacia una respuesta estable cuando se presenta una información de entrada.

Muchas de las investigaciones acerca de la estabilidad de las redes se basan en el establecimiento de una función, denominada función energía, para representar los posibles estados (puntos de equilibrio) de la red. La función energía de una red Hopfield discreta tiene la siguiente forma:

$$E = \frac{1}{2} \sum_{i=1}^{N} \quad \sum_{j=1}^{N} \quad w_{ij} \, s_i \, s_j + \sum_{i=1}^{N} \quad \theta_i \, s_i$$

La función energía puede imaginarse como una superficie que presenta determinados valores mínimos, algo semejante a un paisaje montañoso donde los mínimos serían los valles. Cuando en la red se han almacenado M informaciones o patrones, los posibles estados estables de la red serán también M. Estos M estados corresponden precisamente a los mínimos de la función energía.



Cuando la red Hopfield se utiliza como memoria asociativa, el objetivo es conseguir que los patrones que debe memorizar se sitúen en los mínimos de la función y, consecuentemente, sean los estados estacionarios (estables) de la red. Puede demostrarse que esto se consigue si se verifica que los pesos de las conexiones autorrecurrentes son nulos y si el resto cumple la regla de Hebb.

5. Limitaciones del modelo

Mencione todos los problemas de reconocimiento que puede tener una red neuronal de Hopfield. Explicar cómo se calculan, sus fórmulas. Hopfield: determinación de la cantidad de información a almacenar para una recuperación bastante buena y para una recuperación perfecta. Ortogonalidad.

En una red de Hopfield, el **número de informaciones que puede ser aprendido** (almacenado) está severamente limitado. Si se almacenan demasiadas informaciones, durante su funcionamiento la red puede converger a valores de salida diferentes de los aprendidos durante la etapa de entrenamiento, con lo que la tarea de asociación entre la información presentada y alguna de las almacenadas se realiza incorrectamente, pues se está asociando dicha información de entrada a otra desconocida.

Esta situación no se produce nunca si el número de informaciones almacenadas es menor o igual que N/(4 In N), siendo N el número de neuronas de la red. Sin embargo, este límite puede suavizarse si se permite la posibilidad de un mínimo error en la recuperación de las informaciones almacenadas, suficientemente pequeño para poder identificar dicha información sin ningún problema; en tal caso, el número de informaciones que se pueden almacenar debe ser menor que el 13,8% del número de neuronas de la red (0,138N).

$$Capacidad = \begin{cases} 0.138 \cdot N & \text{para una recuperación suficientemente buena} \\ \dfrac{N}{4\ln(N)} & \text{para una recuperación perfecta} \end{cases}$$

Una segunda limitación del modelo, es que, a pesar de cumplirse el requisito anterior, no siempre se puede garantizar que la red realice una asociación correcta entre una entrada y una de las informaciones almacenadas. Si estas últimas no son **suficientemente diferentes** entre sí, es decir, si no son ortogonales, puede ocurrir que se generen salidas diferentes a todas ellas o que, ante una entrada que coincida con una de las informaciones aprendidas, la red converja hacia una salida correspondiente a otra información almacenada que fuese muy parecida.

En definitiva, lo que se debe pretender siempre es que estas informaciones sean ortogonales, lo cual ocurre si se cumple que cada par de patrones de entrada difiere en, al menos, N/2 componentes siendo N el número total de componentes por patrón. Esta condición puede expresarse como: $\sum_{i=1}^{N} \quad e_i^{(k)} \, e_i^{(m)} \leq 0 \;\; \forall \, k \neq m$ donde $e_i^{(k)} \, e_i^{(m)}$ son los valores binarios (+1 o -1) de los componentes (el producto da +1 cuando son iguales y -1 cuando son distintos). Esta condición puede ser relajada, estableciendo una distancia mínima del 30% del total para que sean casi ortogonales, garantizándose todavía un funcionamiento aceptable.

Hopfield: ¿Qué significa ek * em>0?

Significa que los patrones aprendidos no son ortogonales, es decir, que no difieren en al menos el 50% de sus componentes unos de otros. Al hacer el producto de las componentes binarias de la misma posición se obtiene +1 cuando son iguales y -1 cuando son distintas, por lo que un valor mayor a 0 implica que más de la mitad de ellas son iguales.

En el modelo de Hopfield, el concepto de ortogonalidad está asociado a:

- La cantidad de información que puede almacenar la red
- **Al grado de diferencia que hay entre los patrones de entrenamiento**
- A la complejidad de los cálculos en el proceso de entrenamiento
- Ninguna de las anteriores

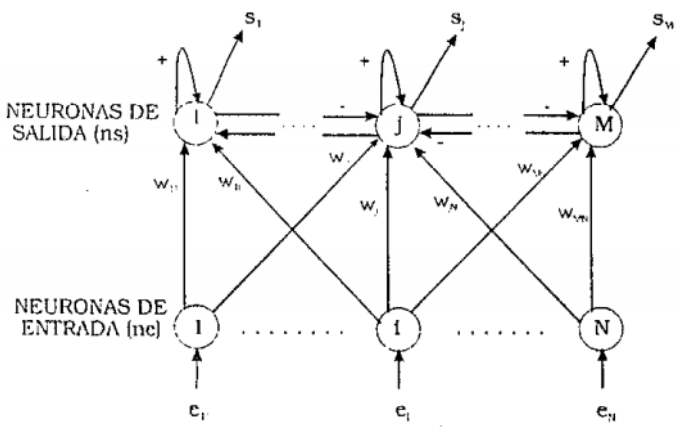
Self-Organizing Map (SOM)

T. Kohonen presentó en 1.982 un modelo de red neuronal con capacidad para formar mapas de características de manera similar a como ocurre en el cerebro. El objetivo de Kohonen era demostrar que un estímulo externo (información de entrada) por sí solo, suponiendo una estructura propia y una descripción funcional del comportamiento de la red, era suficiente para forzar la formación de los mapas.

Este modelo tiene dos variantes, denominadas LVQ (Learning Vector Quantization) y TPM (Topology-Preserving Map) o SOM (Self-Organizing Map). Ambas se basan en el principio de formación de mapas topológicos para establecer características comunes entre las informaciones (vectores) de entrada a la red, aunque difieren en las dimensiones de éstos, siendo de una sola dimensión en el caso de LVQ, y bidimensional, e incluso tridimensional, en la red SOM.

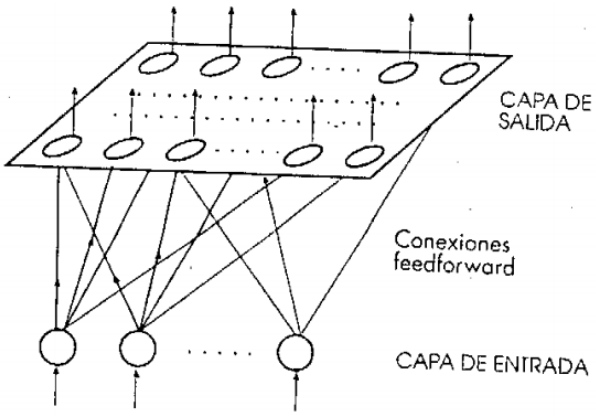
1. Arquitectura

La arquitectura de la versión original (LVQ) se trata de una red de **dos capas** con **N neuronas de entrada** y **M de salida**. Cada una de las N neuronas de entrada se conecta a las M de salida a través de conexiones hacia adelante (feedforward).



Entre las neuronas de la capa de salida puede decirse que existen **conexiones laterales implícitas** de inhibición (peso negativo), pues aunque no estén conectadas, cada una de estas neuronas va a tener cierta influencia sobre sus vecinas. El valor que se asigne a los pesos de las conexiones feedforward entre las capas de entrada y salida (w_{ji}) durante el proceso de aprendizaje de la red va a depender precisamente de esta interacción lateral. La influencia que una neurona ejerce sobre las demás es función de la distancia entre ellas. siendo muy pequeña cuando están muy alejadas.

Por otra parte, la versión SOM trata de de establecer una correspondencia entre los datos de entrada y un espacio bidimensional de salida, creando mapas topológicos de dos dimensiones, de tal forma que ante datos de entrada con características comunes, se deben activar neuronas situadas en zonas próximas de la capa de salida,



La interacción lateral entre las neuronas de la capa de salida sigue existiendo, aunque ahora hay que entender la distancia como una zona bidimensional que existe alrededor de cada neurona.

2. Funcionamiento

Cuando se presenta a la entrada una información $E_k = (e_1^{(k)}, \ldots, e_N^{(k)})$ cada una de las M neuronas de la capa de salida la recibe a través de las conexiones feedforward con peso w_{ji}. También estas neuronas reciben las correspondientes entradas debidas a las conexiones laterales con el resto de las neuronas de salida y cuya influencia dependerá de la distancia a la que se encuentren.

Así, la salida generada por una neurona de salida j ante un vector de entrada $E_k$sería: $s_j(t+1) = f(\sum_{i=1}^{N} \; w_{ji}\, e_i^{(k)} - \sum_{p=1}^{M} \; Int_{pj}\, s_p(t))$ donde Int_{pj}es una función del tipo sombrero mexicano que representa la influencia lateral de la neurona p sobre la neurona j.

La función de transferencia de las neuronas de salida (f) será del tipo continuo, lineal o sigmoidal, ya que esta red trabaja con valores reales.

Explicar el funcionamiento de SOM

Se trata de una red de tipo competitivo, que al presentar una entrada E_k, la red evoluciona hasta una situación estable en la que se activa solo una neurona de salida, la vencedora. Por ello, la formulación matemática de su funcionamiento puede simplificarse mediante la siguiente expresión, que representa cuál de las M neuronas se activa al introducir dicha información E_k:

$s_j = 1 \; si \; es \; el \; MIN|E_k - W_j| = MIN\sqrt{\sum_{i=1}^{N} \; \left(e_i^{(k)} - w_{ij}\right)^2}$ $y \; s_j = 0 \; en \; otro \; caso$ donde $|E_k - W_j|$ es una medida (por ejemplo, distancia euclídea) de la diferencia entre el vector de entrada $E_k = (e_1^{(k)}, \ldots, e_N^{(k)})$ y el vector de los pesos $W_j = (w_{j1}, \ldots, w_{jN})$ de las conexiones entre cada una de las neuronas de entrada y la neurona de salida j. En estos pesos se registran los datos almacenados en la red durante el proceso de aprendizaje. En la fase de funcionamiento, lo que se pretende es encontrar el dato aprendido más parecido al de entrada para, en consecuencia, averiguar qué neurona se activará y, sobre todo, en qué zona del espacio bidimensional de salida se encuentra.

Lo que hace la red, en definitiva, es realizar una **tarea de clasificación**, ya que la neurona de salida activada ante una entrada representa la clase a la que pertenece dicha información de entrada. Además, ante otra entrada parecida se activa la misma neurona de salida u otra cercana a la anterior por lo que se garantiza que las neuronas topológicamente próximas sean sensibles a entradas físicamente similares. Por esta causa, la red es especialmente útil para establecer relaciones, desconocidas previamente, entre conjuntos de datos.

3. Aprendizaje

El aprendizaje es de tipo **offline**, por lo que se distingue una etapa de aprendizaje y otra de funcionamiento. En la etapa de aprendizaje se fijan los valores de los pesos de las conexiones (feedforward) entre la capa de entrada y la de salida.

Esta red utiliza un aprendizaje **no supervisado** de **tipo competitivo y cooperativo**. Al presentar una entrada a la red, las neuronas de la capa de salida compiten entre sí y sólo una de ellas se activa. Luego, se actualizan los pesos de las conexiones tanto de la neurona vencedora como de sus vecinas.

En este modelo, el aprendizaje no concluye después de presentarle una vez todos los patrones de entrada, sino que habrá que repetir el proceso varias veces para refinar el mapa topológico de salida, de tal forma que cuantas más veces se presenten los datos, tanto más se reducirán las zonas de neuronas que se deben activar ante entradas parecidas, consiguiendo que la red pueda realizar una clasificación más selectiva.

Algoritmo de SOM. Explicar el algoritmo de aprendizaje de mapas auto-organizados. SOM: explicar el funcionamiento del algoritmo.

El algoritmo de aprendizaje utilizado para establecer los valores de los pesos de las conexiones entre las N neuronas de entrada y las M de salida es el siguiente:

1. Se inicializan los pesos (w_{ji}) con valores aleatorios pequeños y se fija la zona inicial de vecindad entre las neuronas de salida.
2. Se presenta a la red una información de entrada (la que debe aprender) en forma de vector $E_k = (e_1^{(k)}, \ldots, e_N^{(k)})$, cuyas componentes $e_i^{(k)}$ serán valores continuos.
3. Se determina la neurona vencedora de la capa de salida. Esta será aquella j cuyo vector de pesos W_j (vector cuyas componentes son los valores de los pesos de las conexiones entre esa neurona y cada una de las neuronas de la capa de entrada) sea el más parecido a la información de entrada E_k. Para ello, se calculan las distancias o diferencias entre ambos vectores, considerando una por una todas las neuronas de salida. Suele utilizarse la distancia euclídea o la siguiente expresión, que elimina la raíz cuadrada: $d_j = \sum_{i=1}^{N} \left(e_i^{(k)} - w_{ji}\right)^2 \quad 1 \le j \le M$
4. Una vez localizada la neurona vencedora (j'), se actualizan los pesos de las conexiones entre las neuronas de entrada y dicha neurona, así como los de las conexiones entre las de entrada y las neuronas vecinas de la vencedora. Con esto se consigue asociar la información de entrada con una zona de la capa de salida.
$$w_{ji}(t+1) = w_{ji}(t) + \alpha(t) \,.\, h \,.\, \left[e_i^{(k)} - w_{ji}(t)\right]$$
donde h es la función de vecindad cuyo valor desciende al alejarse de la neurona

vencedora y $\alpha(t)$ es el coeficiente de aprendizaje, con un valor entre 0 y 1, que decrece con el número de iteraciones del proceso de entrenamiento. Suele utilizarse alguna de las siguientes expresiones: $\alpha(t) = \frac{1}{t}$ $\alpha(t) = \alpha_1\left(1 - \frac{t}{\alpha_2}\right)$ siendo $\alpha_1$un valor de 0,1 o 0,2 y α_2 un valor próximo al número de iteraciones de aprendizaje (suele tomarse α_2=10.000)

5. El proceso se debe repetir, volviendo a presentar todo el juego de patrones de aprendizaje $E_1, E_2, \ldots$ hasta que los pesos de la red se estabilicen.

Una vez que la red se estabiliza, es decir, los valores de los pesos permanecen estables, dichos valores finales representan los centroides (media aritmética) de las instancias pertenecientes a cada una de las clases que la red ha establecido.

SOM: identificar fase cooperativa y fase competitiva en el algoritmo.

La fase competitiva corresponde al paso 3 del algoritmo, donde se calculan las distancias de los pesos respecto a la entrada y se determina la neurona vencedora. La fase cooperativa corresponde al paso 4, donde se actualizan los pesos de aquellas neuronas cercanas a la vencedora.

SOM: ¿Cómo se determina el número de entradas?

El número de entradas en la red SOM está dado por la cantidad de componentes que tienen los patrones a clasificar.

¿Cuál es el objetivo de la función de vecindad?

La función de vecindad tiene como propósito lograr que las neuronas próximas sean sensibles a entradas similares, de manera que la red sirva para establecer relaciones previamente desconocidas entre conjuntos de datos.

Función de vecindad para SOM

Una posible función de vecindad es $h(s_1, s_2) = \frac{1}{1+d(s_1,s_2)}$

Clustering

El clustering es un método de aprendizaje no supervisado que consiste en agrupar datos de acuerdo a sus valores, de manera que las instancias en el mismo clúster son similares entre sí. Para ello, se requiere de una función que mida qué tan similares son dos instancias, o que mida la distancia entre ellas. Existen dos tipos de clustering: **particional** y **jerárquico**.

K-Means

Algoritmo K-Means. Explicación.

K-means es el algoritmo más conocido de **clustering particional**. El mismo recibe como entradas el número k de clusters a crear y un conjunto de instancias D={$\underline{x}_1$, $\underline{x}_2$, …, $\underline{x}_n$), siendo $\underline{x}_i$=(x_{i1}, x_{i2}, …, x_{ir}) un vector en un espacio real con r dimensiones. Como resultado, produce k clusters cuyos centros se conocen como centroides, utilizando una función de distancia para particionar los datos. Los centroides representan al cluster y es la media de todas las instancias en dicho cluster.

Algorithm *k*-means(*k*, *D*)
1 choose *k* data points as the initial centroids (cluster centers)
2 **repeat**
3 **for** each data point **x** ∈ *D* do
4 compute the distance from **x** to each centroid;
5 assign **x** to the closest centroid // a centroid represents a cluster
6 **endfor**
7 re-compute the centroid using the current cluster memberships
8 **until** the stopping criterion is met

Inicialmente, el algoritmo selecciona aleatoriamente k instancias como las semillas de los centroides. Luego, calcula la distancia entre cada instancia $\underline{x}$ y cada centroide $\underline{m}_j$ como $dist(\underline{x}, \underline{m}_j) = \sqrt{(x_{i1} - m_{j1})^2 + (x_{i2} - m_{j2})^2 + \ldots + (x_{ir} - m_{jr})^2}$. Se asigna a la instancia el cluster correspondiente al centroide más cercano. Una vez que todas las instancias fueron asignadas, el centroide de cada cluster se recalcula usando las instancias en dicho cluster: $\underline{m}_j = \frac{1}{|C_j|} \sum_{x \in C_j} \underline{x}_i$ siendo $|C_j|$la cantidad de instancias en el cluster C_j.

Este proceso se repite hasta cumplir alguno de los criterios de parada:

1. No hay reasignaciones de instancias a otros clusters.
2. No hay cambios en los centroides.
3. El decremento en la suma del error cuadrático (SSE) es mínimo. $SSE = \sum_{j=1}^{k} \sum_{x \in C_j} dist(\underline{x}, \underline{m}_j)^2$ donde k es el número de clusters solicitados, C_j es el cluster j, $\underline{m}_j$es el centroide de C_j y $dist(\underline{x}, \underline{m}_j)^2$es la distancia entre una instancia $\underline{x}$ y el centroide $\underline{m}_j$

Un problema es que un cluster puede quedar vacío, sin instancias asignadas a él. Para solucionar esto, se selecciona alguna instancia como centroide de reemplazo. Se puede usar

la instancia más lejana de algún cluster grande (en caso de usar el SSE como criterio de parada, se puede tomar del cluster con el mayor SSE).

Disk-k-means

Algoritmo Disk-K-Means. Explicación.

Algorithm disk-*k*-means(k, D)

1 Choose k data points as the initial centriods $\mathbf{m}_j, j = 1, \ldots, k$;
2 **repeat**
3 initialize $\mathbf{s}_j \leftarrow \mathbf{0}, j = 1, \ldots, k$; // $\mathbf{0}$ is a vector with all 0's
4 initialize $n_j \leftarrow 0, j = 1, \ldots, k$; // n_j is the number of points in cluster j
5 **for** each data point $\mathbf{x} \in D$ **do**
6 $j \leftarrow \arg\min_{i \in \{1,2,\ldots k\}} dist(\mathbf{x}, \mathbf{m}_i)$;
7 assign $\mathbf{x}$ to the cluster j;
8 $\mathbf{s}_j \leftarrow \mathbf{s}_j + \mathbf{x}$;
9 $n_j \leftarrow n_j + 1$;
10 **endfor**
11 $\mathbf{m}_j \leftarrow \mathbf{s}_j / n_j, j = 1, \ldots, k$;
12 **until** the stopping criterion is met

Disk-K-Means es una variante del algoritmo original que no requiere cargar todo el dataset en memoria. El proceso escanea la información del disco t veces, siendo t el número de iteraciones para alcanzar la convergencia.

El algoritmo recibe como entradas el número k de clusters a crear y un conjunto de instancias D={$\underline{x_1}$, $\underline{x_2}$, …, $\underline{x_n}$), siendo $\underline{x_i}$=(x_{i1}, x_{i2}, …, x_{ir}) un vector en un espacio real con r dimensiones. Como resultado, produce k clusters cuyos centros se conocen como centroides, utilizando una función de distancia para particionar los datos. Los centroides representan al cluster y es la media de todas las instancias en dicho cluster.

Inicialmente, se seleccionan aleatoriamente k instancias como las semillas de los centroides $\underline{m_j}$. Se inicializan con 0 los vectores $\underline{s_j}$, usados para calcular incrementalmente la suma de las componentes de los vectores de cada cluster, y los valores n_j, que indican la cantidad de instancias asignadas a cada cluster. Luego, para cada instancia $\underline{x_i} \in D$:

1. Se calcula la distancia a cada centroide, asignando a dicha instancia el cluster correspondiente al centroide más cercano.

$$dist(\underline{x_i}, \underline{m_j}) = \sqrt{(x_{i1} - m_{j1})^2 + (x_{i2} - m_{j2})^2 + \ldots + (x_{ir} - m_{jr})^2}$$

2. Se suman las componentes de $\underline{x}$ al acumulado $\underline{s_j}$.
3. Se incrementa en 1 n_j (cantidad de instancias asignadas al cluster j)

Finalmente, se recalculan los centroides haciendo $\underline{m_j} = \underline{s_j} \,/\, n_j$

Ventajas:

1. Simplicidad y eficiencia
2. Es fácil de entender y de implementar

3. Su complejidad es O(t.k.n) donde t es el número de iteraciones, k el número de clusters y n la cantidad de instancias. Como k y t son, en general, mucho más pequeños que n, se lo puede considerar como un algoritmo lineal dependiente de n.

Desventajas:

1. El algoritmo sólo es aplicable a datasets donde está definida la noción de media. Una variante es el algoritmo k-modes.
2. El usuario debe especificar el número de clusters por adelantado.
3. El algoritmo es sensible a outliers (instancias muy lejanas a las demás, que suelen ser errores).
4. El algoritmo es sensible a las semillas iniciales de los centroides, de manera que la variación de estas puede dar lugar a clusters diferentes.
5. El algoritmo no es apropiado para encontrar clusters que no son hiper-elipsoides.

Para tratar los outliers se pueden eliminar las instancias con distancia superior a cierto valor o hacer una selección aleatoria de un subconjunto de instancias, calcular los centroides con ellas y luego asignar las instancias restantes a los clusters formados.

Heurística para la elección de centroides semillas.

Un método simple para la elección de semillas iniciales consiste en calcular la media de todo el dataset. Luego, la primera semilla se elige como la más lejana a la media, la segunda como la más lejana a la primera, y las siguientes como aquellas cuya suma de las distancias a las semillas ya seleccionadas sea máxima.

Una mejor alternativa para las semillas posteriores a la segunda sería calcular la distancia a las demás semillas ya seleccionadas y buscar aquella instancia que tenga el máximo producto de dichas distancias, de manera que si se está muy cerca de un centroide, el producto resultante no sea un valor alto. Con esto se reduciría la posibilidad de que la semilla seleccionada esté lejos de un centroide pero muy cerca de otro.

El algoritmo k-means:

- Es un método de clasificación local
- Es un método de clasificación lazy
- Es un algoritmo de aprendizaje supervisado
- **Ninguna de las anteriores**

K-Means:

- **Si se seleccionan las mismas semillas en K-Means y en Disk-K-Means, se obtienen los mismos resultados**
- **Se debe definir desde un principio el número de clases**

Clustering jerárquico

Clustering Jerárquico. Explicación. ¿Cómo se implementa?

El clustering jerárquico forma los clusters mediante una secuencia anidada de clusters en forma de árbol (también llamado dendograma). Las instancias se encuentran en las hojas del

árbol y un único cluster que cubre todas las instancias constituye la raíz. Cada nodo interno es un cluster que contiene clusters hijos.

Explicar clustering aglomerativo y divisivo. ¿Cuáles son las diferencias entre clustering aglomerativo y divisivo?

Existen dos tipos de clustering jerárquico:

- **Aglomerativo (bottom-up)**: construye el árbol desde abajo hacia arriba, mezclando los pares de clusters más cercanos para ascender un nivel. El proceso continúa hasta que todas las instancias fueron mezcladas en un único cluster que constituye la raíz del árbol.
- **Divisivo (top-down)**: inicia con todas las instancias en un clúster raíz y luego lo divide en un conjunto de clusters hijos. Cada uno de estos hijos es dividido recursivamente hasta que solo quedan clusters simples que contienen una única instancia.

Además del orden en que se efectúan, varían en la medición de las distancias y el criterio de enlace de los clusters: mientras que el aglomerativo enlaza los clusters más cercanos, el divisivo separa los más lejanos.

Los métodos aglomerativos son mucho más populares por lo que nos enfocaremos en ellos.

Algoritmo de clustering jerárquico

Algorithm Agglomerative(D)
1 Make each data point in the data set D a cluster,
2 Compute all pair-wise distances of $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n \in D$;
2 **repeat**
3 find two clusters that are nearest to each other;
4 merge the two clusters form a new cluster c;
5 compute the distance from c to all other clusters;
12 **until** there is only one cluster left

El algoritmo aglomerativo recibe como entrada el conjunto de instancias D y produce una jerarquía de clusters. Para ello, hace de cada instancia en D un cluster, y calcula la distancia entre cada par de instancias $\underline{x_i} \in D$. Luego, entra en un bucle hasta que quede un único cluster. En cada iteración:

1. Encuentra los 2 clusters más cercanos.
2. Mezcla los 2 clusters para formar un nuevo cluster c.
3. Calcula la distancia de c a los demás clusters.

De esta manera, el último cluster formado constituye la raíz del árbol.

Métodos utlizados para medir la distancia inter-cluster

Para calcular la distancia entre dos clusters existen diferentes métodos, entre ellos:

- **Single link**: la distancia entre dos clusters es la distancia entre los dos puntos más cercanos de ellos (uno de cada cluster). Une los clusters con la menor distancia entre sus puntos más cercanos. Es apropiado para encontrar clusters de forma no elíptica.

Puede ser sensible al ruido dando lugar a un efecto en cadena en el que las instancias erróneas unen dos clusters naturalmente separados. Tiene complejidad $O(n^2)$

- **Complete-link**: la distancia entre dos clusters es la distancia entre los dos puntos más alejados de ellos (uno de cada cluster). Une los clusters con la menor distancia entre sus puntos más alejados. Si bien no tiene el problema del efecto en cadena, puede ser sensible a outliers (instancias muy alejadas de las demás). Tiene complejidad $O(n^2 . log\ n)$
- **Average-link**: la distancia entre dos clusters es el promedio de las distancias entre todos los pares de instancias de dichos clusters. Es un punto medio entre la sensibilidad a outliers de complete-link y la tendencia al efecto cadena de single-link. Tiene complejidad $O(n^2 . log\ n)$

Otros métodos:

- Centroide: la distancia entre dos clusters es la distancia entre sus centroides (medias de todas las instancias en dicho cluster)
- Centinela: la distancia entre dos clusters es el incremento en la suma del error cuadrático (SSE) que resultaría de mezclar ambos clusters.

Ventajas:

- Permite usar cualquier función de distancia
- Permite al usuario explorar los clusters en el nivel de detalle que desee
- Se pueden encontrar clusters de cualquier forma

Desventajas:

- Los principales problemas de estos métodos son su complejidad computacional y sus requisitos de memoria que son, al menos, cuadráticos. Se puede usar sampling (tomar una muestra pequeña con la que formar los clusters y luego asignar a estos clusters las demás instancias) o métodos de scale-up (encontrar primero muchos clusters pequeños con un algoritmo eficiente y luego aplicar clustering jerárquico sobre dichos clusters).

Clustering jerárquico

- En Single Link la distancia entre dos clusters es la distancia mínima entre sus centroides.
- En Complete Link la distancia entre dos clusters es la distancia máxima entre sus centroides.
- En Average Link la distancia entre dos clusters es la distancia promedio entre sus centroides.
- **Ninguna es correcta.**

K-Nearest-Neighbors (KNN)

A diferencia de los métodos de aprendizaje que construyen un modelo general a partir de los datos de entrenamiento, el aprendizaje basado en instancia consiste simplemente en almacenar los datos de entrenamiento presentados. Cuando se presenta una nueva instancia de consulta, se recupera de memoria un conjunto de elementos similares que se utilizan para clasificar la misma. También se conocen como métodos perezosos (lazy) ya que demoran el procesamiento hasta que se presenta una nueva instancia.

Algoritmo

K-vecinos-próximos es el método basado en instancia más básico; este clasifica cada ejemplo no etiquetado a partir de la etiqueta más frecuente entre sus K vecinos más cercanos del conjunto de entrenamiento.

El mismo asume que todas las instancias corresponden a puntos en un espacio n-dimensional. Sea una instancia arbitraria $x = (a_1(x), a_2(x), \ldots, a_n(x))$ donde $a_r(x)$ indica el valor del atributo r de dicha instancia. La distancia entre dos instancias x_i y x_j está definida en términos de la distancia euclídea $d(x_i, x_j) = \sqrt{\sum_{r=1}^{n} (a_r(x_i) - a_r(x_j))^2}$

Algoritmo de aprendizaje: para cada ejemplo de entrenamiento $(\underline{x}, f(\underline{x}))$, añadir el ejemplo a la lista *training_examples*.

Algoritmo de clasificación: dada una instancia de consulta x_q a clasificar:
Sean $x_1, \ldots, x_k$ las k instancias de *training_examples* más cercanas a x_q, devolver:

$$f(x_q) \leftarrow argmax_{c \in C} \sum_{i=1}^{k} \delta(c, f(x_i))$$

donde $\delta(c, f(x_i)) = 1 \; si \; x_i \; pertenece \; a \; la \; clase \; c \; y \; \delta(c, f(x_i)) = 0 \; en \; otro \; caso$

El algoritmo recibe un conjunto de instancias de entrenamiento D, el número de vecinos K a considerar y la nueva instancia a clasificar x_q. Como resultado devuelve la clase a la que pertenece dicha instancia x_q. Para ello, calcula la distancia entre x_q y cada instancia en D, toma las K instancias más cercanas y asigna a x_q la clase más frecuente entre dichas instancias.

KNN con distancias ponderadas

KNN Ponderado. Explicación.

Un refinamiento destinado a mejorar las clasificaciones del algoritmo consiste en ponderar la contribución de cada uno de los K vecinos próximos en función a su distancia respecto de la nueva instancia a clasificar, otorgando un mayor peso a aquellos que se encuentran más cerca. Esto se puede lograr designando como clase de la nueva instancia aquella que corresponda a:

$$f(x_q) \leftarrow argmax_{c \in C} \sum_{i=1}^{k} \frac{\delta(c, f(x_i))}{d(x_q, x_i)^2}$$

donde x_q es la instancia de consulta, C es el conjunto de las diferentes clases en los ejemplos de entrenamiento, $d(x_q, x_i)$ es la distancia entre la instancia de consulta x_q y la instancia x_i y $\delta(c, f(x_i)) = 1\ si\ x_i\ pertenece\ a\ la\ clase\ c\ y\ \delta(c, f(x_i)) = 0\ en\ otro\ caso$

En caso de que la distancia entre la instancia de consulta x_q y alguna instancia x_i sea cero, se asigna a x_q la clase de x_i

Al ponderar las distancias, una posibilidad es, en lugar de considerar solo las K instancias más cercanas, tener en cuenta todas las instancias del conjunto de entrenamiento, ya que la influencia de las instancias muy lejanas será casi nula. Sin embargo, esto implicaría que el algoritmo sea más lento y pasaría de ser un método local a uno global.

Determinación del K óptimo

Explicar el método k-cross fold validation en KNN.

Un método para determinar el K óptimo es K-fold cross-validation. Sea D el conjunto de entrenamiento con N instancias, este método consiste en calcular las distancias entre cada par de instancias en D y luego, para cada K de 1 a N-1:

1. Dividir D en un conjunto de entrenamiento de tamaño N–1 y un conjunto de prueba de tamaño 1.
2. Clasificar la instancia del conjunto de prueba de acuerdo a las del conjunto de entrenamiento.
3. Comparar la clasificación obtenida con la clase conocida de dicha instancia, incrementando en 1 la cantidad de clasificaciones correctas para el valor actual de K si así lo fuera.
4. Repetir el proceso N veces de forma que cada instancia constituya el conjunto de prueba una vez.
5. Determinar la precisión del valor actual de K dividiendo la cantidad de clasificaciones correctas por N.

Finalmente, el K óptimo corresponde al valor de K con la mayor precisión.

Ventajas:

- El único parámetro a ajustar es K
- Es un método de implementación simple
- El costo computacional es relativamente bajo excepto por el cálculo del K óptimo
- Es un método robusto que no se ve afectado por errores de algunos datos dentro del conjunto de entrenamiento (ruido)
- No requiere fase de entrenamiento

Desventajas:

- Maldición de la dimensionalidad: se consideran todos los atributos de las instancias, por lo que si el objetivo depende sólo de algunos de ellos, el método probablemente dará lugar a malas clasificaciones.

- Si no se establece un método de desempate, no se puede realizar la clasificación cuando dentro de los K vecinos próximos hay igual cantidad de instancias de cada clase.
- Necesidad de calcular un valor de K óptimo

¿Cómo afecta la cantidad de muestras presentes en cada clase al método de clasificación KNN?

Por tratarse de un método local, KNN no se ve afectado por la cantidad de instancias en cada clase siempre y cuando la cantidad de instancias en la clase menos numerosa sea mayor a K/2.

Al implementar k vecinos, se tiene una clase A con p instancias y B con q instancias. Si se considera K=p+q, marque la/s correcta/s:

- **La nueva instancia es de clase A si p>q**
- **La nueva instancia es de clase B si p<q**
- No se puede determinar de antemano sin haber calculado las distancias con la nueva instancia
- **La nueva instancia no se puede clasificar si p=q**
- Ninguna de las anteriores

Al implementar k vecinos con distancia ponderada se tiene una clase A con p instancias y B con q instancias. Si se considera K=p+q, marque la/s correcta/s:

- La nueva instancia es de clase A si p>q
- La nueva instancia es de clase B si p<q
- La nueva instancia no se puede clasificar si p=q
- **Ninguna de las anteriores**

KNN Ponderado:

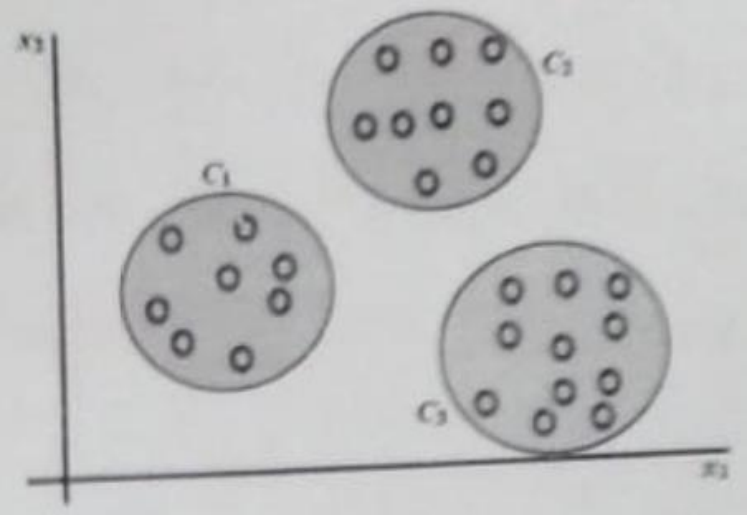
- Requiere una cantidad parecida de instancias en las clases
- Es necesario determinar de antemano el K óptimo
- Es un método de clasificación global
- **Ninguna de las anteriores**

KNN:

- **Es lazy**
- **Es un método local**
- Requiere una cantidad parecida de instancias en las clases.
- Ninguna es correcta.

Daba el siguiente conjunto de datos en R2, explique cómo implementar:

- Un clasificador MLP.
- Un clasificador NB.
- Un clasificador k-nn.



Implementaría un clasificador MLP con 3 capas: 2 neuronas de entrada (x1 y x2), 2 neuronas de capa oculta que determinarán las 2 rectas necesarias para separar las clases y 2 neuronas de salida que, de acuerdo a la ubicación respecto de esas rectas, indicarán la clase correspondiente. Para la salida utilizaría funciones de transferencias de tipo escalón, codificando, por ejemplo, las salidas 00 y 01 como clase 1, 10 como clase 2, y 11 como clase 3.

KNN con ponderación: proponga una medida de ponderación diferente a la del material de estudio, justifique su respuesta.

Diferencias K vecinos próximos y Naive Bayes

¿Qué método clasifica mejor, naive bayes o k-vecinos?

Compare los modelos SOM, Hopfield y Perceptrón simple, teniendo en cuenta los siguientes aspectos:

1. **Cantidad de entradas**
2. **Cantidad de salidas**
3. **Aprendizaje online/offline**
4. **Aprendizaje supervisado/no supervisado**
5. **Condición de finalización del aprendizaje**
6. **Función de transferencia de la/s salida/s**
7. **Limitaciones**
8. **Ejemplo de implementación**

Notas

Al explicar un algoritmo, empezar explicando qué recibe como entrada y qué retorna, y luego qué hace en el medio.

En Hopfield aclarar si la red se estabiliza o no y si la salida corresponde al patrón más cercano o no.